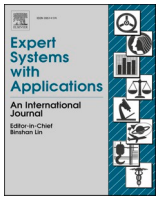
Expert Systems With Applications 257 (2024) 125026



Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa





Theoretical analysis and implementation of mandatory operations-based accelerated search in graph space for hybrid flow shop scheduling

Yuting Wang [a], Yuyan Han [a,*], Huan Li [a], Junqing Li [b], Kaizhou Gao [c], Yiping Liu [d]

[a] School of Computer Science, Liaocheng University, Liaocheng, China
[b] Department of Mathematics, Yunnan Normal University, Kunming, China
[c] Macau University of Science and Technology, Macau Institute of Systems Engineering, Macau, China
[d] College of Computer Science and Electronic Engineering, Hunan University, Changsha, China

ARTICLE INFO

Keywords:
Hybrid flow shop scheduling
Makespan
Mandatory operations
Accelerated local search
Effective insertion
Graph space

ABSTRACT

For the hybrid flow shop scheduling problem (HFSP), this study addresses its high computational burden associated with insertion operations due to over-much objective function calculations as the number of jobs increases. Unfortunately, the search solution space and existing accelerated local search methods exhibit limitations. We find that there are many operations on the critical path that do not affect the completion time, especially when there are multiple critical paths. However, deleting mandatory operations must make the original makespan value smaller. Moreover, only specific insertion positions have the potential to decrease the makespan of the changed sequence. In view of this, we emphasize the need for improved accelerated local search techniques tailored to overcome these limitations, and propose the theoretical analysis and methods of an accelerated local search based on effective insertion of mandatory operations in the graph for the HFSP with makespan criterion. More specifically, first, we represent a solution using a solution graph, and introduce four definitions related to mandatory operations in the graph space. Second, we propose five theorems and their corresponding proofs. Third, we present a mandatory operations-based accelerated iterated greedy algorithm (MOAIG) in the graph space according to the above theorems. Finally, we conduct many experiments on two well-known benchmarks (a total of 576 instances). Through statistical analysis, it becomes evident that the computational time required for calculating the makespan associated with operations-based insertion has decreased significantly. The development of the mandatory operations-based effective insertion accelerated local search is a promising approach to significantly mitigate the time complexity by reducing the numbers of operations perturbed and insertion positions involved in the calculation, and makes up for gaps in existing accelerated insertion methods for HFSP.

1. Introduction

In the traditional permutation flow shop scheduling problem (PFSP), jobs move through each stage with only one machine, following the same stage order. However, in practical manufacturing, the need arises to increase capacity or balance the capacity at various stages (Y. Han et al., 2024; Y. Wang et al., 2024; F. Zhao, Zhang, et al., 2023). This necessitates the acquisition of additional machines in some stages, resulting in the availability of more than one machine. Consequently, when jobs or products reach these stages, multiple parallel machines become an option for selection. Clearly, this production model can enhance production efficiency, increase capacity, and make the production process more adaptable. The extended problem mentioned above is a hybrid flow shop scheduling problem, known as HFSP, which encompasses machine selection and job scheduling considerations. The HFSP has been demonstrated to be strongly NP-hard, even when restricted to just two stages (Gupta, 1988; H.-X. Qin et al., 2022).

Due to the complexity of the scheduling problem, its optimal solution is unknown and difficult to obtain in a finite time. However, it can be approximated using heuristic methods or metaheuristic methods (Ribas et al., 2010; Y. Wang, Han, Wang, Pan, et al., 2023; F. Zhao et al., 2024; F. Zhao, Zhu, et al., 2023). So far, many methods have been used to solve the HFSP, such as heuristics (Dios et al., 2018), constructive heuristics (Fernandez-Viagas et al., 2018), the discrete artificial bee colony

* Corresponding author.
E-mail address: hanyuyan@lcu-cs.com (Y. Han).

https://doi.org/10.1016/j.eswa.2024.125026
Received 20 April 2024; Received in revised form 2 June 2024; Accepted 5 August 2024
Available online 9 August 2024
0957-4174/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

(DABC) algorithm (Pan et al., 2014), the discrete particle swarm optimization (DPSO) algorithm (Liao et al., 2012), the artificial immune system (AIS) algorithm (Engin & Döyen, 2004), the estimation of distribution algorithm (EDA) (Wang et al., 2012), and more. These solution representations used by the above algorithms can adopt only one job-based encoding, since the job sequencing rule and the machine assignment rule are given. If there are μ jobs, a job-based encoding can be represented as $(1, 2, \cdots, \mu)$. In this case, the maximum solution space of the HFSP is $\mu!$. At present, most algorithms in the existing literature are to explore the optimal solution in the space with $\mu!$. However, the search space does not contain all feasible solutions. Depending on the encoding method, we can classify the solution space into four categories (Fernandez-Viagas et al., 2019).

For the first category, both the job sequencing rule and the machine assignment rule are given, and the size of the solution space is $\mu!$. For the second category, only the machine assignment rule is given. In this case, a solution representation should include σ sequences (σ is the number of stages), each having μ jobs, and the maximum solution space of the HFSP is $(\mu!)^{\sigma}$. For the third category, only the job sequencing rule is given. In this case, a solution representation includes a set of $1+\sigma$ sequences, where one job sequence plus σ sequences that are the machine assignments of each job in each stage. The maximum solution space of the HFSP is $(\mu!)\prod_{s=1}^{\sigma}(\theta_s)^{\mu}$, where θ_s is the number of parallel machines in the s th stage. For the fourth category, there are no job sequencing rule and machine assignment rule. In this case, we can assign a sequence of jobs for each machine. Thus, a solution representation includes a set of $\sum_{s=1}^{\sigma}\theta_s$ sequences, and the maximum solution space of the HFSP is $(\mu!)^{\sigma}\prod_{s=1}^{\sigma}\binom{\mu+\theta_s-1}{\theta_s-1}$. It is obvious that the lengths of solution representation for the first three categories are equal but unequal length for the fourth solution representation.

Taking 5 jobs and 3 stages with each having 2 parallel machines as an example, from the first category to the fourth category, the solution spaces are $5! = 120$, $(5!)^3 = 1728000$, $(5!)\prod_{i=1}^{3}(2)^5 = 3932160$, and $(5!)^3\prod_{i=1}^{3}\binom{6}{1} = 373248000$, respectively.

It can be seen that the above four classifications have a direct impact on the size of the solution space searched. Optimizing solutions within the larger space can increase the chances of finding the optimal solution. The simpler the representation of a solution, the easier the algorithm design and implementation. Conversely, the more complex the representation of a solution, the more complex and challenging the implementation of the corresponding algorithmic strategies. Therefore, more research has been done on the first category of solution representation. For the HFSP, the real solution space is $(\mu!)^{\sigma}\prod_{s=1}^{\sigma}\binom{\mu+\theta_s-1}{\theta_s-1}$. However, so far, the algorithms mentioned in most of the literature only search for the optimal solution within $n!$ solution space. Compared to the real solution space, there are many unknown solution spaces to be explored. Currently, only a few literatures extend the solution space to $(\mu!)^{\sigma}$, $(\mu!)\prod_{s=1}^{\sigma}(\theta_s)^{\mu}$, or $(\mu!)^{\sigma}\prod_{s=1}^{\sigma}\binom{\mu+\theta_s-1}{\theta_s-1}$ (Fernandez-Viagas et al., 2019). This encourages us to expand the search space, thus increasing the chances of finding the optimal solution. In fact, the literature (Fernandez-Viagas et al., 2019) has shown that the optimal solutions searched in the solution spaces of the second and fourth types are basically the same. It indicates that the second type of search space ($(\mu!)^{\sigma}$) is sufficient for the HFPS problem. Therefore, we need to further expand the space based on the first category, which will be effective. Of course, expanding the space also increases the algorithm's time complexity, and we need to efficiently search for the optimal solution in the space size of $(\mu!)^{\sigma}$ or $(\mu!)^{\sigma}\prod_{s=1}^{\sigma}\binom{\mu+\theta_s-1}{\theta_s-1}$, which requires a balance between the size of the search solution space and the time complexity of the search algorithm.

In summary, with the above analyses, we can adopt the operations-based local search to expand the search solution space. However, the existing critical path-based local search methods have not yet been well studied to date. There is a phenomenon that some operations on the critical path do not affect the completion time, especially when there are multiple critical paths. But deleting mandatory operations must make the original makespan value smaller. Moreover, only specific insertion positions have the potential to decrease the makespan of the changed sequence. In view of this, we design an accelerated local search based on the effective insertion of mandatory operations to further explore the potential space.

The main contributions of this study are listed. (1) We represent a solution using a solution graph and give four definitions related to mandatory operations in the graph space. (2) By exploring the characteristic of multiple parallel machines, we propose an accelerated local search based on effective insertion of mandatory operations, in which five theorems and their proofs are proposed for the first time. (3) Based on the above theorems, a mandatory operations-based accelerated iterated greedy algorithm is designed. In contrast to other methods, the proposed MOAIG algorithm shows the superiority by reducing the number of operations perturbed and insertion involved in the calculation, leading to a significant reduction in the overall computation time. (4) We validated the theories and method proposed for the performance on a total of 576 test instances in many experiments. The results demonstrate that the proposed theories are accurate and feasible and that the proposed method can significantly reduce computational efforts.

2. Literature review

According to the size of the search solution space for the HFSP, we will categorically elaborate on the optimizing hybrid flow shop scheduling problems within the $\mu!$-sized solution space and beyond the $\mu!$-sized solution space, and the local search strategies for the hybrid flow shop scheduling problems, respectively.

2.1. Optimizing the hybrid flow shop scheduling problems within $\mu!$-sized solution space

When the job sequencing rule and the machine assignment rule are given, the size of the searching solution space is fixed at $\mu!$. The typical encoding and decoding ways are job-based encoding and the decoding of the first available machine (FAM), respectively. The whole scheduling process in all the stages depends on the above two rules, which limits the searching solution space, and leads to the most algorithms and strategies merely perturbing the job sequence within $n!$-sized solution space.

Fernandez-Viagas et al. considered the $\mu!$-sized solution space and adopted the FAM rule to assign jobs to machines. The authors designed four new heuristics, that is, two memory-based constructive heuristics and two constructive heuristics based on Johnson's algorithm (Fernandez-Viagas et al., 2018). Wang et al. adopted a job-based encoding and the first available machine-based machine allocation rule to solve the HFSP with makespan criterion. The authors proposed an effective EDA that can obtain a good job sequence by sampling for the probability model (Wang et al., 2013). Chou determined the job permutation in the first stage and then assigned jobs to each machine in the subsequent stages by multiple decoding methods. The author proposed the PSO algorithm to generate the new job sequence within the $\mu!$-sized space (Chou, 2013). Li et al. proposed two decoding methods, i.e., solution-dependent and random rules, for the job-based encoding rule, and designed a hybrid variable local search (HVNS) algorithm to disturb the job sequence within a search space having the size of $\mu!$ (Li et al., 2014). Marichelvam et al. presented a cuckoo search (CS) algorithm by incorporating the constructive NEH heuristic to minimize the makespan of

HFSP (Marichelvam et al., 2014). Cui and Gu employed the vector representation, and proposed an improved DABC (IABC) algorithm by combining a differential evolution and a variable local search to search good solutions (Cui & Gu, 2015).

For extended hybrid flow shop scheduling problems, most of the literature still conducts searches in the $\mu!$-sized solution space. Schulz et al. considered three objectives, i.e., the makespan, the total energy costs, and the peak power. The authors proposed three machine selection rules, that is, the earliest completion time, deterministic and leveling rules (Schulz et al., 2019). Marichelvam et al. considered the effect of human factors in HFSP, and proposed an improved PSO algorithm by integrating variable local search to minimize makespan (Marichelvam et al., 2020). Qin et al. adopted the job-based encoding rule and the FAM decoding rule to solve the energy-efficient distributed HFSP with blocking constraint. The authors proposed a Q-learning selection mechanism to store historical experiences (H. Qin et al., 2023). Various decoding methods can yield distinct makespan values for a specific job sequence, and the use of multiple decoding strategies can help to discover the optimal value. Consequently, some studies have proposed hybrid decoding strategies (W. Han et al., 2021; Paternina-Arboleda et al., 2008; Y. Wang, Wang, et al., 2023; Yu et al., 2020).

2.2. Optimizing the hybrid flow shop scheduling problems beyond the $\mu!$-sized solution space

There are three solution spaces that beyond the $\mu!$-sized solution space, i.e., $(\mu!)^{\sigma}\prod_{s=1}^{\sigma}\binom{\mu+\theta_s-1}{\theta_s-1}$, $(\mu!)^{\sigma}$, and $(\mu!)\prod_{s=1}^{\sigma}(\theta_s)^{\mu}$. The first one includes all feasible solutions. The last two are the subsets of the first, respectively. If jobs are scheduled between stages by using different rules, the good solution will be generated with high probability. Paternina-Arboleda et al. adopted different job sequencing rules and expanded the solution space to $(\mu!)^{\sigma}$ (Paternina-Arboleda et al., 2008). Vairaktarakis and Elhafsi adopted the dynamic programming (DP) algorithm to explore the good solution in the $(\mu!)^{\sigma}$-sized solution space (Vairaktarakis & Elhafsi, 2000). For the solution space with the same size, Lin et al. adopted job-based encoding at each stage and a hybrid decoding strategy in the proposed chaos-enhanced simulated annealing (CSA) algorithm. The authors applied a list scheduling, a permutation scheduling, and a non-permutation scheduling rule, respectively, to schedule the job sequence of each stage (Lin et al., 2021).

To expand the solution space to $(\mu!)^{\sigma}\prod_{s=1}^{\sigma}\binom{\mu+\theta_s-1}{\theta_s-1}$, Fan et al. adopted a tabu search (TS) based on a disjunctive graph representation a local search, in which they expand the searching space by moving a crucial operation in the critical path to another machine in the same stage (Fan et al., 2023). Zhao et al. designed a heterogeneous graph neural network and deep reinforcement learning to solve the HFSP. The authors adopted the graph-level representation vectors of heterogeneous graphs as input and proposed the operation-based and machine-based node embedding (Y. Zhao et al., 2024). Kuang et al. proposed a two-stage cross-neighborhood search algorithm (TSCNSA) to solve HFSP. In the first exploration stage, the authors employed a job-based encoding and two decoding methods to search the potential area. The size of this solution space is $\mu!$. In the exploitation stage, an operation-based encoding, a transformation a the job-based solution to an operation-based solution, and three local search operators are proposed for different solution spaces (Kuang et al., 2024). This expands the solution space to $(\mu!)^{\sigma}\prod_{s=1}^{\sigma}\binom{\mu+\theta_s-1}{\theta_s-1}$. To consider expanding the searching space and reducing the time complexity of optimization algorithms, Fernandez-Viagas removed the operations only in the critical path and reinserted them in the same stage by the proposed speed-up evaluation method (Fernandez-Viagas, 2022).

2.3. The local search strategies for the hybrid flow shop scheduling problems

Local search strategies can improve the development capability of algorithms. In the context of the hybrid flow scheduling problem, the space for local search can be divided into two categories, since it involves the sequencing of job and the selection of machines. The first category is the local perturbations within the $\mu!$-sized solution space. The second category is the local disturbances between operations on each machine at each stage, which expands the solution space into $(\mu!)^{\sigma}$, $(\mu!)\prod_{s=1}^{\sigma}(\theta_s)^{\mu}$, or $(\mu!)^{\sigma}\prod_{s=1}^{\sigma}\binom{\mu+\theta_s-1}{\theta_s-1}$.

Currently, most local search strategies proposed in the existing literature belong to the first category. Aqil and Allali proposed an iterative local search method by combining four neighborhood perturbation strategies (Aqil & Allali, 2020). Since the job sequence in all the stages are same order, and the machine selection rule is fixed, the perturbation space of the proposed local search is $\mu!$. Similarly, Shao et al. proposed nine neighbor local search methods to solve the multi-objective distributed HFSP (Shao et al., 2021). Zohal et al. adopted two neighbor structures based on insertion moves as the local search strategy to solve the HFSP with economic lot-sizing and sequencing (Zohali et al., 2019). Gümüşçü et al. proposed a chaotic local search consisting of inversion, insertion, and swap. These strategies generate feasible solutions within the $n!$-sized solution space (Gümüşçü et al., 2022). For the second category local search, it involves searching in the operation space, such as based on critical path local search (Fernandez-Viagas, 2022) and (Fan et al., 2023), it involves sequentially taking operations from the critical path and then attempting to insert them behind different jobs in the same stage of that operation. Except for operations in the critical path, the operations in the non-critical path also participate to be perturbed (Kuang et al., 2024) and (Lin et al., 2021). That is, we can randomly select an operation in a stage and then try to insert it behind different jobs in the same stage of that operation, followed by choosing the best position for insertion. From the existing literature on local search strategies designed for HFSP, they face the following three challenges.

(1) For the first type of local search strategy mentioned above, it merely perturbs the job scheduling sequence, with its disturbance space limited to $\mu!$ (Dios et al., 2018; Engin & Döyen, 2004; Fernandez-Viagas et al., 2018; Gupta, 1988; Liao et al., 2012; Pan et al., 2014; Ribas et al., 2010; Wang et al., 2012) and (Chou, 2013; Cui & Gu, 2015; W. Han et al., 2021; Li et al., 2014; Marichelvam et al., 2014, 2020; H. Qin et al., 2023; Schulz et al., 2019; S. Wang et al., 2013; Y. Wang, Wang, et al., 2023; Yu et al., 2020). However, for the HFSP, the true feasible solution space is $(\mu!)^{\sigma}\prod_{s=1}^{\sigma}\binom{\mu+\theta_s-1}{\theta_s-1}$. Therefore, most local search spaces are restricted and struggle to find the global optimal solution.
(2) Regarding the second category of the local search strategy (Fan et al., 2023; Fernandez-Viagas, 2022; Kuang et al., 2024; Lin et al., 2021; Y. Zhao et al., 2024), operations on the critical path directly affect the final completion time. Therefore, perturbation of operations on the critical path is essential. However, there are many operations on the critical path that do not affect the completion time, especially when there are multiple critical paths. Therefore, considering the algorithm's time complexity, selecting which operations to be perturbed is the second issue to be solved.
(3) When inserting suitable operations into different positions, existing methods attempt all positions (Fan et al., 2023; Fernandez-Viagas, 2022; Kuang et al., 2024; Lin et al., 2021; Y. Zhao et al., 2024), increasing the algorithm's time complexity. Furthermore, many insertion positions are ineffective. Thus,

choosing efficient insertion positions is the third issue to be solved.

To address these three issues of the local search method, this paper first conducts a local search on operations in the graph space, which can expand the search space, thereby solving the first problem. Next, we will choose mandatory operations for perturbation in the graph space, addressing the second issue. Finally, we will utilize the concept of *k-insertion* in the graph space to locate effective insertion positions and reduce the insertion of invalid positions, thus resolving the third problem.

In conclusion, the existing literature of searching the $\mu!$-sized solution space can reduce the search space and the time complexity of the algorithms. However, the found solution may be an approximate one rather than the optimal one. In contrast, the solution space beyond the $\mu!$ size can have a chance of finding the optimal solution. However, the space is very huge, especially the true feasible solution space with $(\mu!)^{\sigma}\prod_{s=1}^{\sigma}\binom{\mu+\theta_s-1}{\theta_s-1}$ size. Neither exact arithmetic nor brute-force enumeration is feasible. This requires the design of targeted and efficient search strategies, which enlarge the search space, on the one hand, and reduce the invalid search to reduce the time complexity, on the other hand. Therefore, this paper focuses on designing an accelerated local search based on the effective insertion of mandatory operations from the perspective of graph space.

3. Problem description and MILP model

HFSP is described as follows. μ jobs are processed at σ sequential stages according to the same sequence. For each stage s ($s = 1,2,\cdots,\mu$), there exist θ_s ($\theta_s \geq 1$) identical parallel machines. Furthermore, there is at least one stage containing more than one identical parallel machine. The k th machine at stage s is denoted as $m_{s,k}$, which means that the set of machines at stage s is $\Delta_s = \{m_{s,1}, m_{s,2}, \cdots, m_{s,\theta_s}\}$. Let $o_{j,s}$ denote the operation which corresponds to job j performed on stage s. The processing time of operation $o_{j,s}$ is denoted as $p_{j,s}$. Our optimization goal is to find a reasonable scheduling sequence with minimal makespan.

Typically, the following discussion of HFSP is based on the following five hypotheses (Y. Wang, Wang, et al., 2023).

1) At time 0, all jobs can be processed and all machines are free.
2) At any given time, a job can only be processed by one machine, and a machine can only process one job.
3) When a machine is processing a particular job, that job is to be processed continuously without preemption or interruption.
4) Each job goes through all the processing stages in turn and is processed by only one machine at a stage.
5) Skipping a stage or ending early is not allowed.

3.1. Mixed integer linear programming model

To solve this problem, we proposed a mixed integer linear programming (MILP) model as follows.

Notions:

μ: The number of jobs.
σ: The number of stages.
j: The index of jobs, $j \in \{1,2,\ldots,\mu\}$.
s: The index of stages, $s \in \{1,2,\ldots,\sigma\}$.
θ_s: The number of machines at stage s.
$m_{s,k}$: The k th machine at stage s.
Δ_s: The set of machines at stage s, $\Delta_s = \{m_{s,1}, m_{s,2}, \cdots, m_{s,\theta_s}\}$.
L: A big positive number.
$o_{j,s}$: the operation which corresponds to job j performed on stage s.
$p_{j,s}$: The processing time of job j at stage s.

Table 1
The processing time of all jobs at each stage.

Stage	Job							
	1	2	3	4	5	6	7	8
1	4	6	6	6	3	7	6	5
2	6	4	5	3	6	6	4	5
3	3	5	5	5	7	7	4	5

Decision variables:

c_{max}: The makespan of the sequence.
$c_{j,s}$: The completion time of operation $O_{j,s}$.
$y_{j,s,k}$: Decision variables, 1 if operation $O_{j,s}$ is processed on machine $m_{s,k}$ at stage s, 0 otherwise.
$z_{j,j',s}$: Decision variables, 1 if job j is at any position before job j' at stage s, 0 otherwise.

Objective:

$$Minimize C_{max} \tag{1}$$

Constraints:

$$\sum_{k=1}^{\theta_s} y_{j,s,k} = 1 \forall j \in \{1,2,\cdots,\mu\}, \forall s \in \{1,2,\cdots,\sigma\} \tag{2}$$

$$z_{j,j',s} + z_{j',j,s} \leq 1 \forall j,j' \in \{1,2,\cdots,\mu\}, j < j', \forall s \in \{1,2,\cdots,\sigma\} \tag{3}$$

$$z_{j,j',s} + z_{j',j,s} \geq y_{j,s,k} + y_{j',s,k} - 1 \forall j,j' \in \{1,2,\cdots,\mu\}, j < j', \forall s \in \{1,2,\cdots,\sigma\}, \forall k \in \{1,2,\cdots,\theta_s\} \tag{4}$$

$$c_{j,s} \geq p_{j,s} \forall j \in \{1,2,\cdots,\mu\}, \forall s \in \{1,2,\cdots,\sigma\} \tag{5}$$

$$c_{j',s} \geq c_{j,s} + p_{j',s} + (y_{j,s,k} + y_{j',s,k} + z_{j,j',s} - 3) \cdot L \forall j,j' \in \{1,2,\cdots,\mu\}, j \neq j', \forall s \in \{1,2,\cdots,\sigma\}, \forall k \in \{1,2,\cdots,\theta_s\} \tag{6}$$

$$c_{j,s} \geq c_{j,s-1} + p_{j,s} \forall j \in \{1,2,\cdots,\mu\}, \forall s \in \{2,3,\cdots,\sigma\} \tag{7}$$

$$c_{max} \geq c_{j,\sigma} \forall j \in \{1,2,\cdots,\mu\} \tag{8}$$

Eq. (1) is the optimization objective of the problem which is to minimize the makespan. Constraint (2) guarantees that a job is processed on only one machine at each stage. Constraint (3) refers to that $z_{j,j',s}$ and $z_{j',j,s}$ cannot equal 1 at the same time. Constraint (4) enforces that if jobs j and j' are processed on the same machine at stage s, one of $z_{j,j',s}$ and $z_{j',j,s}$ must be equal 1 and the other equal 0. Constraint (5) guarantees that the completion time of operation $o_{j,s}$ is larger than or equal to its processing time. For operations $o_{j,s}$ and $o_{j',s}$, if operation $o_{j,s}$ is processed on the same machine before operation $o_{j',s}$, the completion time of operation $o_{j',s}$ must be not less than the complete time of

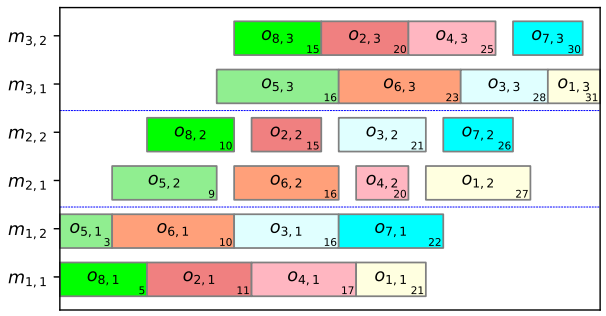


Fig. 1. The scheduling Gantt for an optimal solution.

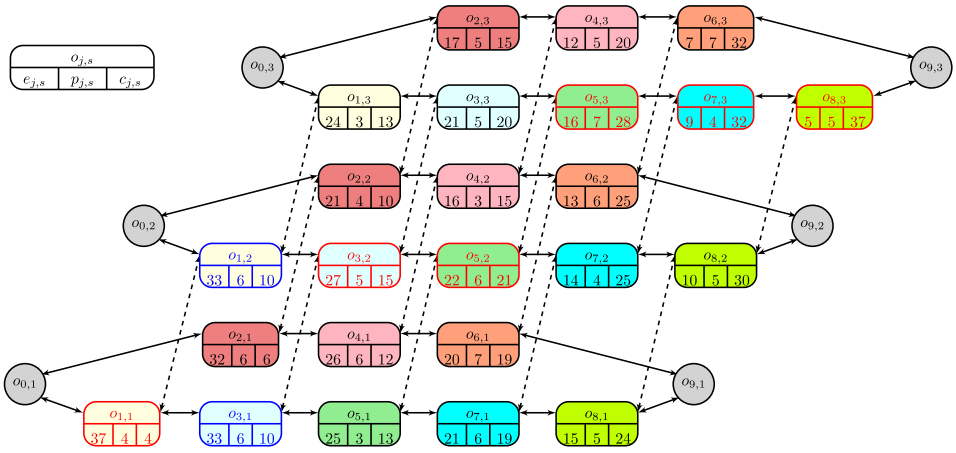


Fig. 2. The graphical representation of a solution, where the blue and red boxes indicate critical operations. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

operation $o_{j,s}$ plus $p_{j',s}$, ensured by Constraint (6).Constraint (7) states that the completion time of an operation for a job is greater than or equal to its processing time plus the completion time of the previous operation for that job. Constraint (8) guarantees that the makespan is equal to or larger than the completion time of all jobs at the last stage.

3.2. Example instance

The following is a specific example. Let $\mu = 2$, $\sigma = 3$, $\theta_1 = \theta_2 = \theta_3 = 2$. Table 1 reports the processing time of all jobs at each stage. The scheduling Gantt for an optimal solution solved using the Gurobi solver is shown in Fig. 1. From Fig. 1, we obtain $c_{max} = 31$.

4. Definitions, theorems, and method for the accelerated local search based on effective insertion of mandatory operations in graph space

Since the feasible solution space of the HFSP is $(\mu!)^{\sigma}\prod_{s=1}^{\sigma}\binom{\mu+\theta_s-1}{\theta_s-1}$, the approach of seeking the optimal solution solely through perturbing the job sequence proves to be overly restrictive. It is advisable to extend the local search strategy to perturb the operation at a stage. That is, we can move an operation from the current stage to a specific position on a machine within that stage. This local search strategy is referred to as *LS*. We find that the critical path (**Definite 2**) has a direct impact on the final completion time, and the critical path-based *LS* method is efficient (referring to Theorem 2). However, when there are multiple critical paths, the reassignments of some critical operations (**Definite 2**) situated on the critical path do not reduce the completion time. Furthermore, considering the algorithm's time complexity, we should only select and adjust these operations that really affect makespan. Here these operations are defined as mandatory operations (**Definite 4**).

Through the above analysis, we conclude that (1) the original makespan value maybe decreased after deleting operations on the critical path. However, deleting the operations in mandatory operations must make the original makespan value smaller (referring to Theorem 3). The method of obtaining mandatory operations is proposed in **Section 4.2.1**. (2) After inserting mandatory operations into different positions, there is a high computational burden associated with insertion operations due to over-much objective function calculations. To mitigate the time complexity, an accelerated evaluation method is proposed in **Section 4.2.2**. (3) During the process of mandatory operations-based local search, only specific insertion positions have the potential to decrease the makespan of the changed sequence. (referring to Theorem 4 and Theorem 5), so to further reduce the time complexity, the effective insertion positions are determined in **Section 4.2.3**, which can minimize the makespan compared to other insertion positions.

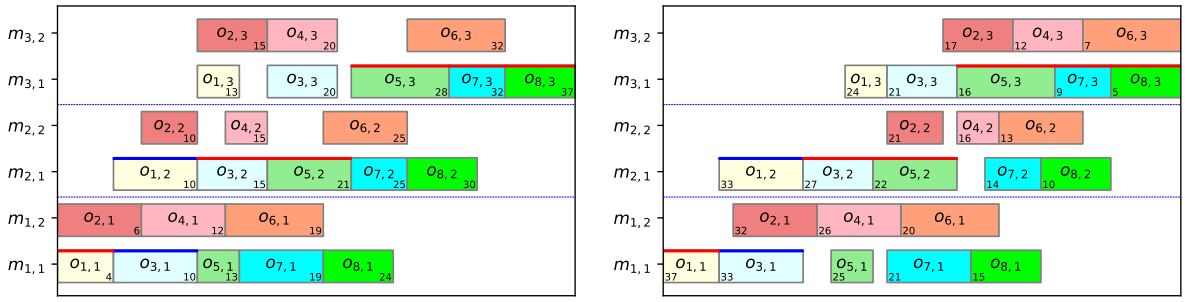
4.1. Definitions related to the accelerated local search based on effective insertion of mandatory operations

A solution to the hybrid flowshop scheduling problem is defined as follows: for each operation $o_{j,s}$ ($j = 1, 2, \cdots, \mu$, $s = 1, 2, \cdots, \sigma$) a unique machine is identified at stage s to process the operation, and the machine operation $o_{j,s}$ is noted as $\zeta_{j,s}$ ($\zeta_{j,s} \in \Delta_s$); furthermore, for each machine, the sequence in which the operations are processed is established.

We represent a solution using a *solution graph* that is abbreviated as G for convenience (Mastrolilli & Gambardella, 2000). In G, each node corresponds to an operation $o_{j,s}$, and the weight of each node equals the processing time $p_{j,s}$ of $o_{j,s}$. Within the solution graph, two virtual operations $o_{0,s}$ and $o_{\mu+1,s}$ are introduced to represent the start and end of processing at each stage s, and their processing times are equal to 0. Within the solution graph, there are two arcs between $o_{j,s-1}$ and $o_{j,s}$, where $j = 1,2,\cdots,\mu, s = 2,3,\cdots,\sigma$. To make the diagram clearer and more concise, the two arcs between $o_{j,s-1}$ and $o_{j,s}$ are represented as $o_{j,s-1} \leftrightarrow o_{j,s}$. Similarly, for the operations on the same stage, $o_{j,s}$ and $o_{j',s}$, $j = 1, 2, \cdots, \mu, j' = 1, 2, \cdots, \mu, s = 1, 2, \cdots, \sigma$, if $\zeta_{j,s} = \zeta_{j',s}$ and $o_{j',s}$ is the immediate successor of $o_{j,s}$, then two arcs ($o_{j,s} \leftrightarrow o_{j',s}$) exist between these two operations. If $o_{j,s}$ is the first operation on machine $\zeta_{j,s}$, then two arcs ($o_{0,s} \leftrightarrow o_{j,s}$) exist between $o_{0,s}$ and $o_{j,s}$. If $o_{j,s}$ is the last operation on machine $\zeta_{j,s}$, then two arcs exist ($o_{j,s} \leftrightarrow o_{\mu+1,s}$) between $o_{j,s}$ and $o_{\mu+1,s}$.

In G, the calculation of the forward completion time is from the first stage backward, and the calculation of the backward completion time is from the last stage forward. After calculating the forward and the backward completion times, the maximum completion time of the scheduling sequence can be determined, which is denoted as $c_{max}(G)$. The forward and backward complete times of each operation at each stage are calculated as follows, it should be noted that the time complexity of computing all $c_{j,s}$ or $e_{j,s}$ is $O(\mu\sigma)$.

(1) For the dummy operations $o_{0,s}$ and $o_{\mu+1,s}$, $c_{0,s}$=0 and$e_{\mu+1,s} = 0$, respectively, where $s = 1, 2, \cdots, \sigma$.
(2) At the first stage, each non-dummy operation, $o_{j,1}$, has three incoming arcs, $o_{j',1} \rightarrow o_{j,1}$, $o_{j,1} \leftarrow o_{j'',1}$ and $o_{j,1} \leftarrow o_{j,2}$. We can calculate the forward complete time of $o_{j,1}$, i.e., $c_{j,1} = c_{j',1} + p_{j,1}$, according to $o_{j',1} \rightarrow o_{j,1}$. Meanwhile, we can calculate the backward complete time of $o_{j,1}$, i.e., $e_{j,1} = max(e_{j'',1}, e_{j,2}) + p_{j,1}$, according to $o_{j,1} \leftarrow o_{j'',1}$ and $o_{j,1} \leftarrow o_{j,2}$.



(a) The forward scheduling (b) The backward scheduling

Fig. 3. The corresponding Gantt chart of the solution in Fig. 2, where the red and blue bars above the rectangle indicate critical operations. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

(3) At the last stage, each non-dummy operation, $o_{j,\sigma}$, has three incoming arcs, $o_{j,\sigma-1}\rightarrow o_{j,\sigma}$, $o_{j',\sigma}\rightarrow o_{j,\sigma}$ and $o_{j,\sigma}\leftarrow o_{j'',\sigma}$. We can calculate the forward complete time of $o_{j,\sigma}$, i.e., $c_{j,\sigma} = max(c_{j',\sigma}, c_{j,\sigma-1}) + p_{j,\sigma}$, according to $o_{j,\sigma-1}\rightarrow o_{j,\sigma}$ and $o_{j',\sigma}\rightarrow o_{j,\sigma}$. Meanwhile, we can calculate the backward complete time of $o_{j,\sigma}$, i.e., $e_{j,\sigma} = e_{j'',\sigma} + p_{j,\sigma}$, according to $o_{j,\sigma}\leftarrow o_{j'',\sigma}$.

(4) At other stages, each non-dummy operation, $o_{j,s}$ has four incoming arcs, $o_{j,s-1}\rightarrow o_{j,s}$, $o_{j',s}\rightarrow o_{j,s}$, $o_{j,s}\leftarrow o_{j'',s}$ and $o_{j,s}\leftarrow o_{j,s+1}$. We can calculate the forward complete time of $o_{j,s}$, i.e., $c_{j,s} = max(c_{j',s}, c_{j,s-1}) + p_{j,s}$, according to $o_{j,s-1}\rightarrow o_{j,s}$ and $o_{j',s}\rightarrow o_{j,s}$. Meanwhile, we can calculate the backward complete time of $o_{j,s}$, i.e., $e_{j,s} = max(e_{j'',s}, e_{j,s+1}) + p_{j,1}$, according to $o_{j,s}\leftarrow o_{j'',s}$ and $o_{j,s}\leftarrow o_{j,s+1}$.

To better understand the following definitions, we provide a graphical representation of a solution. Based on the example in Section 3.2, assume that the scheduling sequence is (1,2,3,4,5,6,7,8). After decoding, we obtain the graphical representation in the solution space, as shown in Fig. 2. Each node contains four information for an operation, namely, the operation name $o_{j,s}$, the backward completion time $e_{j,s}$, the processing time $p_{j,s}$, and the forward completion time $c_{j,s}$. The solid lines represent the scheduling relationships between operations on the same machine in the same stage. The dashed lines represent the scheduling relationships between operations in different stages. Fig. 3 gives the Gantt chart of the forward and the backward scheduling for the solution in Fig. 2.

Definition 1. (*(Length of a path). In G, the length of a path* $P = (o_{j,s}, \cdots, o_{j',s'})$ *is defined as the sum of processing times of all operations from the start point* $o_{j,s}$ *to the end point* $o_{j',s'}$*, denoted as* $L_G(P)$.) As shown in Fig. 2, $L_G(o_{0,1}\rightarrow o_{1,1}\rightarrow o_{3,1}\rightarrow o_{5,1}\rightarrow o_{7,1}) = 19$. In G, if there are R paths from $o_{j,s}$ to $o_{j',s'}$, among which T ($1 \le T \le R$) paths have the longest length and their length is denoted as $L_G(o_{j,s}\rightarrow o_{j',s'})$. Obviously, in G, the forward completion time of $o_{j,s}$ is equal to $L_G(o_{0,1}\rightarrow o_{j,s})$, i.e., $c_{j,s} = L_G(o_{0,1}\rightarrow o_{j,s})$. Similarly, the backward completion time of $o_{j,s}$ is the length of the longest path from $o_{\mu+1,\sigma}$ to $o_{j,s}$, denoted as $L_G(o_{j,s}\leftarrow o_{\mu+1,\sigma})$, it is obvious that $L_G(o_{j,s}\leftarrow o_{\mu+1,\sigma}) = e_{j,s}$. In Figs. 2 and 3(b), $L_G(o_{5,2}\leftarrow o_{9,3}) = e_{5,2} = 22$.

Definition 2. (*(Critical path, Critical operations). In G, each longest path from* $o_{0,1}$ *to* $o_{\mu+1,\sigma}$ *is defined as a critical path of G, where the i th longest path is denoted as* $P^i_G(o_{0,1}\rightarrow o_{\mu+1,\sigma})$*. The operations on these critical paths, excluding* $o_{0,1}$ *and* $o_{\mu+1,\sigma}$*, are defined as critical operations of G. For* $o_{j,s}$*,* $P^i_G(o_{0,1}\rightarrow o_{j,s}\rightarrow o_{\mu+1,\sigma})$ *is i th critical path that the longest path passing through* $o_{j,s}$ *from* $o_{0,1}$ *to* $o_{\mu+1,\sigma}$*, and its length is denoted as* $L_G(o_{0,1}\rightarrow o_{j,s}\rightarrow o_{\mu+1,\sigma})$.)
For example, in Fig. 2, there are two critical paths, i.e., $P^1_G(o_{0,1}\rightarrow o_{9,3}) = (o_{0,1}\rightarrow o_{1,1}\rightarrow o_{1,2}\rightarrow o_{3,2}\rightarrow o_{5,2}\rightarrow o_{5,3}\rightarrow o_{7,3}\rightarrow o_{8,3}\rightarrow o_{9,3})$ and $P^2_G(o_{0,1}\rightarrow o_{9,3}) = (o_{0,1}\rightarrow o_{1,1}\rightarrow o_{3,1}\rightarrow o_{3,2}\rightarrow o_{5,2}\rightarrow o_{5,3}\rightarrow o_{7,3}\rightarrow o_{8,3}\rightarrow o_{9,3})$. For P^1_G, its critical operations are $o_{1,1}$, $o_{1,2}$, $o_{3,2}$, $o_{5,2}$, $o_{5,3}$, $o_{7,3}$, and $o_{8,3}$. For P^2_G, its critical operations are $o_{1,1}$, $o_{3,1}$, $o_{3,2}$, $o_{5,2}$, $o_{5,3}$, $o_{7,3}$, and $o_{8,3}$.

Clearly, the length of any critical path in the graph is equal to the makespan, i.e., $c_{max}(G) = L_G(o_{0,1}\rightarrow o_{\mu+1,\sigma})$. For operation $o_{j,s}$, assuming that its preceding and succeeding operations on the same machine are $o_{j',s}$ and $o_{j'',s}$, respectively, the length of critical paths passing through $o_{j,s}$ can be calculated using Eq. (9), where $c_{j,0} = e_{j,\sigma+1} = 0, j = 1,2,\cdots,\mu$.

$$\begin{aligned} &L_G(o_{0,1}\rightarrow o_{j,s}\rightarrow o_{\mu+1,\sigma}) \\ &= max(L_G(o_{0,1}\rightarrow o_{j',s}) + L_G(o_{0,1}\rightarrow o_{j,s-1})) + p_{j,s} + max(L_G(o_{j'',s}\leftarrow o_{\mu+1,\sigma}) \\ &\quad + L_G(o_{j,s+1}\leftarrow o_{\mu+1,\sigma})) \\ &= max(c_{j',s}, c_{j,s-1}) + p_{j,s} + max(e_{j'',s}, e_{j,s+1}) \end{aligned} \tag{9}$$

Definition 3. (*(Critical operation subgraph). In G, the subgraph formed by nodes,* $o_{0,1}$*,* $o_{\mu+1,\sigma}$*, the critical operation nodes of G, and the arcs related to these nodes is referred to as the critical operation subgraph of G, denoted as* G^c.) Clearly, within the critical operation subgraph, there are no gaps between consecutive operations, which means that the completion time of the preceding operation in the critical operation subgraph is the same as the start time of the next operation in the critical operation subgraph. Fig. 4 gives the critical operation subgraph in Fig. 2.

Definition 4. (*(Mandatory operations of a path). In* G^c*, if the removal of a critical operation,* $o_{j,s}$*, results in the absence of a path from* $o_{0,1}$ *and* $o_{J+1,S}$*, then* $o_{j,s}$ *is referred to as a mandatory operation of the critical path in G.*)

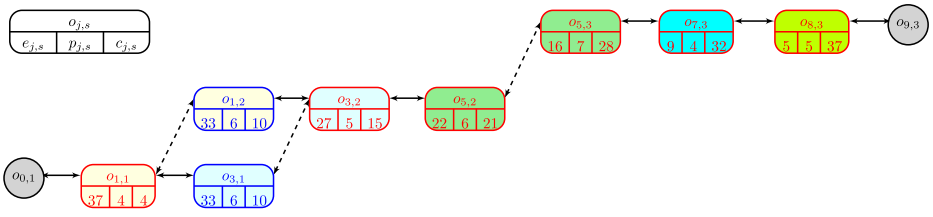


Fig. 4. The critical operation subgraph in Fig. 2.

From another perspective, the mandatory operations are the intersection of critical operations for all critical paths. In Fig. 4, for the two critical paths, $P_G^1(o_{0,1}\to o_{9,3})$ and $P_G^2(o_{0,1}\to o_{9,3})$, the intersection set of their critical operations is $\{o_{1,1}, o_{3,2}, o_{5,2}, o_{5,3}, o_{7,3}, \text{and } o_{8,3}\}$ highlighted in red.

4.2. The accelerated local search based on effective insertion of mandatory operations

4.2.1. Obtaining mandatory operations

As analyzed earlier, the critical path-based local search can affect the final completion time. However, when there are multiple critical paths, many operations on the critical path do not affect the completion time. Therefore, selecting which operations, i.e., mandatory operations, to be perturbed is a key issue to be solved. In this section, we first analyze why the mandatory operations make the original makespan value smaller by the corresponding theorem. Then, we give the method to obtain mandatory operations.

Theorem 1. (*An operation $o_{j,s}$ is a critical operation of graph G if and only if* $c_{j,s} + e_{j,s} - p_{j,s} = c_{max}(G)$.) **Proof**

An operation $o_{j,s}$ is a critical operation of graph G.

$\Leftrightarrow \exists P_G^i(o_{0,1}\to o_{\mu+1,\sigma})$ s.t. $o_{j,s} \in P_G^i(o_{0,1}\to o_{\mu+1,\sigma})$ ($P_G^i(o_{0,1}\to o_{\mu+1,\sigma})$ is a critical path of G)

$$\Leftrightarrow L_G(o_{0,1}\to o_{j,s}\to o_{\mu+1,\sigma}) = L_G(o_{0,1}\to o_{\mu+1,\sigma})$$

$$\Leftrightarrow L_G(o_{0,1}\to o_{j,s}\to o_{\mu+1,\sigma}) = c_{max}(G)$$

$$\Leftrightarrow L_G(o_{0,1}\to o_{j,s}) + L_G(o_{j,s}\leftarrow o_{\mu+1,\sigma}) - p_{j,s} = c_{max}(G)$$

$\Leftrightarrow c_{j,s} + e_{j,s} - p_{j,s} = c_{max}(G)$■

In our graph space, performing the *LS* method on $o_{j,s}$ involves first removing it from G corresponding to the solution. The graph after deleting $o_{j,s}$ is denoted as G^-. Then the graph after inserting $o_{j,s}$ into a specific position on a machine within the same stage is denoted as G^+. The maximum completion times of the scheduling solution in G^- and G^+ are denoted as $c_{max}(G^-)$ and $c_{max}(G^+)$, respectively.

Theorem 2. (*In G, when the LS method is applied to $o_{j,s}$, we have* $L_{G^+}(o_{0,1}\to o_{\mu+1,\sigma}) = \max(L_{G^-}(o_{0,1}\to o_{\mu+1,\sigma}), L_{G^+}(o_{0,1}\to o_{j,s}\to o_{\mu+1,\sigma}))$. *That is,* $c_{max}(G^+) = \max(c_{max}(G^-), L_{G^+}(o_{0,1}\to o_{j,s}\to o_{\mu+1,\sigma}))$.) **Proof**

We will provide separate proofs for two cases.

For the first case, inserting $o_{j,s}$ into G^- results in $o_{j,s}$ becoming a critical operation of G^+. In this scenario, a critical path in G^+ must include $o_{j,s}$. Thus, we have $L_{G^+}(o_{0,1}\to o_{\mu+1,\sigma}) = L_{G^+}(o_{0,1}\to o_{j,s}\to o_{\mu+1,\sigma})$. Furthermore, since $L_{G^+}(o_{0,1}\to o_{\mu+1,\sigma})$ must be greater than or equal to $L_{G^-}(o_{0,1}\to o_{\mu+1,\sigma})$, we have $L_{G^+}(o_{0,1}\to o_{\mu+1,\sigma}) = \max(L_{G^-}(o_{0,1}\to o_{\mu+1,\sigma}), L_{G^+}(o_{0,1}\to o_{j,s}\to o_{\mu+1,\sigma}))$.

For the second case, inserting $o_{j,s}$ into G^- results in $o_{j,s}$ becoming a non-critical operation of G^+. In this scenario, G^- and G^+ have identical critical paths. Therefore, we have$L_{G^+}(o_{0,1}\to o_{\mu+1,\sigma}) = L_{G^-}(o_{0,1}, o_{\mu+1,\sigma})$. Since $o_{j,s}$ does not become a critical operation of G^+, it implies that G^+ does not have any critical path passing through $o_{j,s}$. In this case, we have $L_{G^+}(o_{0,1}\to o_{j,s}\to o_{\mu+1,\sigma}) < L_{G^+}(o_{0,1}\to o_{\mu+1,\sigma})$. Thus, $L_{G^+}(o_{0,1}\to o_{\mu+1,\sigma}) = \max(L_{G^-}(o_{0,1}\to o_{\mu+1,\sigma}), L_{G^+}(o_{0,1}\to o_{j,s}\to o_{\mu+1,\sigma}))$.■

Theorem 3. (*In G, if $o_{j,s}$ is deleted, and it is not a mandatory operation, then* $L_{G^-}(o_{0,1}\to o_{\mu+1,\sigma}) = L_G(o_{0,1}\to o_{\mu+1,\sigma})$, *implying* $c_{max}(G^-) = c_{max}(G)$. *If $o_{j,s}$ is deleted, and it is a mandatory operation, then* $L_{G^-}(o_{0,1}\to o_{\mu+1,\sigma}) < L_G(o_{0,1}, o_{\mu+1,\sigma})$, *implying* $c_{max}(G^-) < c_{max}(G)$.) **Proof**

In G, if $o_{j,s}$ is not a mandatory operation, deleting it from G ensures that at least one critical path in G still exists in G^-. Thus, we have $L_{G^-}(o_{0,1}\to o_{\mu+1,\sigma}) = L_G(o_{0,1}\to o_{\mu+1,\sigma})$. If $o_{j,s}$ is a mandatory operation, it means that all critical paths in G include $o_{j,s}$. In this scenario, when $o_{j,s}$ is deleted from G, all critical paths in G don't exist in G^-. This implies that some non-critical paths in G become critical paths in G^-. Thus, we have $L_{G^-}(o_{0,1}\to o_{\mu+1,\sigma}) < L_G(o_{0,1}\to o_{\mu+1,\sigma})$. ■

Based on Theorem 2 and Theorem 3, when performing an *LS* method on operation $o_{j,s}$ in graph G, two case can be considered.

In the first case, if $o_{j,s}$ is not a mandatory operation, combining Theorem 2 and Theorem 3 yields $c_{max}(G^+) = \max(c_{max}(G^-), L_{G^+}(o_{0,1}\to o_{j,s}\to o_{\mu+1,\sigma})) = \max(c_{max}(G), L_{G^+}(o_{0,1}\to o_{j,s}\to o_{\mu+1,\sigma}))$. That is, $c_{max}(G^+) \geq c_{max}(G)$. This implies that performing an *LS* method on operation $o_{j,s}$ will not reduce the makespan of G^+ relative to G. Therefore, performing an *LS* method on operation $o_{j,s}$ is meaningless.

In the second case, if $o_{j,s}$ is a mandatory operation, combining Theorem 2, we have $c_{max}(G^+) = \max(c_{max}(G^-), L_{G^+}(o_{0,1}\to o_{j,s}\to o_{\mu+1,\sigma}))$, and combining Theorem 3, we have $c_{max}(G^-) < c_{max}(G)$. In this scenario, if $L_{G^+}(o_{0,1}\to o_{j,s}\to o_{\mu+1,\sigma})$ is less than $c_{max}(G)$, it can ensure that the *LS* method can reduce the makespan of G^+ compared to the makespan of G.

The following gives the process of generating mandatory operations of all critical operations in G. We first decode the current job sequence into the graph space by *DecodeToGraph*(π) outlined in **Algorithm 1**, where π is the job order obtained by perturbing the scheduling sequence. Next, in the solution space, we construct the critical operation subgraph G^c with a time complexity of $O(\mu\sigma)$ according to Theorem 1. Then, based on G^c, we have realized the nonrecursive algorithm (shown in **Algorithm 2**) for obtaining mandatory operations according to the recursive Tarjan's algorithm (Tarjan, 1971). We find all mandatory operations with a time complexity of $O(V(G^c)+A(G^c))$, where $V(G^c)$ and $A(G^c)$ represent the number of nodes and arcs in G^c. Line 3 in Algorithm 2, the concept of *discovery* refers to the order in which the nodes are visited by the depth-first search (DFS) traversal. It starts from 1 and is incremented as we visit new nodes. Parallelly, the "*low*" value for each node represents the minimum *discovery* number that can be reached from that node when doing a DFS, including via back edges. It helps to identify if a particular vertex can reach an ancestor in the DFS tree, indicating the presence of a cycle or a back edge. Line 13 in Algorithm 2, $visit[\langle o_{j,s}, o_{j',s'}\rangle]$ equals 0, indicating that the arc, $\langle o_{j,s}, o_{j',s'}\rangle$, has not been visited. Its value equals 1, indicating that the arc, $\langle o_{j,s}, o_{j',s'}\rangle$, has been visited.

Algorithm 1 *DecodeToGraph*(π)

Input: π, c_{max}
01: For each machine construct an empty doubly linked list with a head node and a tail node, these linked lists form the graph G
02: $c_{j,s}\leftarrow 0,\ j = 1,2,\cdots,\mu,\ s = 1,2,\cdots,\sigma$
03: $MIdle_{s,k}\leftarrow 0,\ s = 1,2,\cdots,\sigma,\ k = 1,2,\cdots,\theta_s$
04: **for** $j = 1$ **to** μ **do**
05: For $k = 1,2,\cdots,\theta_1$, find the k, denoted *sk*, that minimizes $MIdle_{1,k}$
06: Create a node corresponding to operation $o_{j,1}$
07: Insert this node into the tail of the linked list corresponding to machine *sk* at the first stage
08: $c_{j,1}\leftarrow MIdle_{1,sk} + p_{j,1}$
09: end for
10: **for** $s = 2$ **to** σ **do**
11: $\pi'\leftarrow$*Sort the jobs according to complete time* $p_{j,s-1}$
12: **for** $j = 1$ **to** μ **do**
13: For $k = 1,2,\cdots,\theta_s$, find the k, denoted *sk*, that minimizes $MIdle_{s,k}$
14: Create a node corresponding to operation $o_{j,s}$
15: Insert this node into the tail of the linked list corresponding to machine *sk* at stage s
16: $c_{j,s}\leftarrow max(MIdle_{s,sk}, c_{j,s-1}) + p_{j,s}$
17: **end for**
18: end for
19: Link the nodes representing the different operations of the same job in order of stages to construct μ doubly linked list
20: Calculate the backward completion time $e_{j,s}$ for each node in graph G
21: Calculate c_{max}
Output: G, c_{max}

Algorithm 2 *ObtainingMandatoryOperations*(G, c_{max})

Input: G, c_{max}

(*continued on next page*)

(*continued*)

Algorithm 2 *ObtainingMandatoryOperations*(G, c_{max})

$MO \leftarrow \varnothing$, and $time_{stamp} \leftarrow 1$ % *time_stamp* is a timestamp representing the current time
Construct the critical operation subgraph G^c according to Theorem 1
$discovery[o_{j,s}] \leftarrow 0$ and $low[o_{j,s}] \leftarrow 0$ for $\forall o_{j,s} \in G^c$
$visit[\langle o_{j,s}, o_{j',s'}\rangle] \leftarrow 0$ for $\forall \langle o_{j,s}, o_{j',s'}\rangle \in G^c$
Construct stack *st* to store the arc of the current node and its parent node
$discovery[o_{0,1}] \leftarrow time_stamp$, $low[o_{0,1}] \leftarrow time_stamp$, and $time_stamp \leftarrow time_stamp + 1$
$st.push(\langle null, o_{0,1}\rangle)$ % the arc $\langle null, o_{0,1}\rangle$ enter the stack, where *null* is the parent node of $o_{0,1}$
while $st.empty() \neq true$ **do**
  $\langle o_{j'',s''}, o_{j,s}\rangle \leftarrow st.top()$ % $o_{j''s''}$ is the parent node of $o_{j,s}$
  $\Psi \leftarrow \{\langle o_{j,s}, o_{j',s'}\rangle | \langle o_{j,s}, o_{j',s'}\rangle \in G^c \bigwedge visit[\langle o_{j,s}, o_{j',s'}\rangle] = 0 \bigwedge o_{j',s'} \neq o_{j'',s''}\}$
  if $\Psi \neq \varnothing$ **then**
    $\langle o_{j,s}, o_{j',s'}\rangle \leftarrow$ Randomly take an arc in Ψ
    $visit[\langle o_{j,s}, o_{j',s'}\rangle] \leftarrow 1$
    if $discovery[o_{j',s'}] = 0$ **then**
      $discovery[o_{j',s'}] \leftarrow time_stamp$, $low[o_{j',s'}] \leftarrow time_stamp$, and $time_stamp \leftarrow time_stamp + 1$
      $st.push(\langle o_{j,s}, o_{j',s'}\rangle)$
    else
      $low[o_{j,s}] \leftarrow min(low[o_{j,s}], discovery[o_{j',s'}])$
    end if
  else
    $st.pop()$
    if $st.empty() \neq true \bigwedge o_{j'',s''} \neq o_{0,1} \bigwedge discovery[o_{j'',s''}] \leq low[o_{j,s}]$ **then**
      $MO \leftarrow MO \cup \{o_{j'',s''}\}$
    end if
    $low[o_{j'',s''}] \leftarrow min(low[o_{j'',s''}], low[o_{j,s}])$
  end if
end while

Output: *MO*

4.2.2. The accelerated evaluation method for $L_{G^+}(o_{0,1} \rightarrow o_{j,s} \rightarrow o_{J+1,S})$

After generating mandatory operations, we can delete one of these mandatory operations, $o_{j,s}$, and attempt to insert $o_{j,s}$ into all possible positions. Finally, we find the position pos_b that minimizes $L_{G^+}(o_{0,1} \rightarrow o_{j,s} \rightarrow o_{\mu+1,\sigma})$. If the value of $L_{G^+}(o_{0,1} \rightarrow o_{j,s} \rightarrow o_{\mu+1,\sigma})$ is less than $c_{max}(G)$, then insert $o_{j,s}$ into position pos_b, otherwise, insert $o_{j,s}$ into its original position. Here, there is a high computational burden associated with insertion operations due to over-much objective function calculations. To mitigate the time complexity, an accelerated calculation of $L_{G^+}(o_{0,1} \rightarrow o_{j,s} \rightarrow o_{\mu+1,\sigma})$ is given as follows.

When $o_{j,s}$ is reinserted into G^-, its preceding and following operations are denoted as $o_{j',s}$ and $o_{j'',s}$, respectively. According to Eq. (9), we can determine $L_{G^+}(o_{0,1} \rightarrow o_{j,s} \rightarrow o_{\mu+1,\sigma}) = max(c^+_{j',s}, c^+_{j,s-1}) + p_{j,s} + max(e^+_{j'',s}, e^+_{j,s+1})$. Based on the characteristics of mandatory operations-based local search, we have $c^+_{j',s} = c^-_{j',s}$, $c^+_{j,s-1} = c^-_{j,s-1} = c_{j,s-1}$, $e^+_{j'',s} = e^-_{j'',s}$ and $e^+_{j,s+1} = e^-_{j,s+1} = e_{j,s+1}$. Therefore, Eq. (10) exists, where c^- and e^- represent the forward and backward completion time in G^-, and c^+ and e^+ represent the forward and backward completion time in G^+.

$$
\begin{aligned}
&L_{G^+}(o_{0,1} \rightarrow o_{j,s} \rightarrow o_{\mu+1,\sigma}) \\
&= max(c^-_{j',s}, c^-_{j,s-1}) + p_{j,s} + max(e^-_{j'',s}, e^-_{j,s+1}) \\
&= max(c^-_{j',s}, c_{j,s-1}) + p_{j,s} + max(e^-_{j'',s}, e_{j,s+1})
\end{aligned} \tag{10}
$$

In fact, the calculation of $L_{G^+}(o_{0,1} \rightarrow o_{j,s} \rightarrow o_{\mu+1,\sigma})$ can be done in $O(1)$ time complexity after updating the forward and backward completion times on the original machine. The reason is that after removing $o_{j,s}$ that is processed on machine $\zeta_{j,s}$ from G to obtain G^-, the forward completion time of operations after $o_{j,s}$ and the backward completion time of operations before $o_{j,s}$ on machine $\zeta_{j,s}$ have changed. However, on machines in Δ_s (the set of machines in stage s) other than $\zeta_{j,s}$, the forward completion time and the backward completion time of operations remain unchanged. Therefore, based on G, after removing $o_{j,s}$, it is only necessary to update the forward completion time of operations after $o_{j,s}$ and the backward completion time of operations before $o_{j,s}$ on machine $\zeta_{j,s}$.

In addition, after removing $o_{j,s}$, the time complexity of updating the changed the forward completion time and the backward completion time on machine $\zeta_{j,s}$ is $O(n(\zeta_{j,s}))$, where $n(\zeta_{j,s})$ represents the number of operations processed on machine $\zeta_{j,s}$.

4.2.3. The effective insertion method

As mentioned above, mandatory operations can be attempted for insertion at all positions within the current stage. However, only specific positions have the potential to decrease makespan. In this subsection, we will solve how to obtain effective insertion position.

When $o_{j,s}$ is reinserted into a machine $m_{s,k}$ ($m_{s,k} \in \Delta_s$) in G^-, there are $n(m_{s,k})+1$ positions available for insertion on $m_{s,k}$, where $n(m_{s,k})$ is the number of operations processed on machine $n(m_{s,k})$. The index of each position is represented as ω, ($\omega = 1, \cdots, n(m_{s,k}) + 1$). For machine $m_{s,k}$, operations preceding and following position ω are denoted as $o_{j(\omega-1,k),s}$ and $o_{j(\omega,k),s}$, respectively, where $j(\omega-1,k)$ and $j(\omega,k)$ represent the jobs corresponding to the preceding and following operations at position ω. If $\omega = 1$, then $j(\omega-1,k) = 0$, indicating $o_{j(w-1,k),s}$ is a virtual operation. If $\omega = n(m_{s,k}) + 1$, then $j(\omega,k) = \mu + 1$, indicating $o_{j(\omega,k),s}$ is a virtual operation.

Hence, there are two crucial positions indexes that need to be determined, one is the rightmost position index on machine $m_{s,k}$ satisfying $c^-_{j(\omega-1,k),s} \leq c^-_{j,s-1}$ in the forward scheduling, and the other is the leftmost position index on machine $m_{s,k}$ satisfying $e^-_{j(\omega,k),s} \leq e^-_{j,s+1}$ in the backward scheduling. The two important position indexes are denoted as α, and β, respectively, which will serve as essential references for selecting effective positions. Eqs (11) and (12) are used to calculate α and β, respectively.

$$\alpha = max(\omega | c^-_{j(\omega-1,k),s} \leq c^-_{j,s-1}, \omega = 1, \cdots, n(m_{s,k}) + 1) \tag{11}$$

$$\beta = min(\omega | e^-_{j(\omega,k),s} \leq e^-_{j,s+1}, \omega = 1, \cdots, n(m_{s,k}) + 1) \tag{12}$$

Theorem 4. (*On $m_{s,k}$, if $\alpha < \beta$, for any position γ to the left of position α, it holds that $max(c^-_{j(\alpha-1,k),s}, c^-_{j,s-1}) + p_{j,s} + max(e^-_{j(\alpha,k),s}, e^-_{j,s+1}) < max(c^-_{j(\gamma-1,k),s}, c^-_{j,s-1}) + p_{j,s} + max(e^-_{j(\gamma,k),s}, e^-_{j,s+1})$; for any position δ to the right of position B, it holds that $max(c^-_{j(\beta-1,k),s}, c^-_{j,s-1}) + p_{j,s} + max(e^-_{j(\beta,k),s}, e^-_{j,s+1}) < max(c^-_{j(\delta-1,k),s}, c^-_{j,s-1}) + p_{j,s} + max\left(e^-_{j(\delta,k),s}, e^-_{j,s+1}\right)$.*) **Proof**

For any position γ to the left of position α, we have $\gamma < \alpha < \beta$. According to Eq. (11), we obtain $c^-_{j(\alpha-1,k),s} \leq c^-_{j,s-1}$ and $c^-_{j(\gamma-1,k),s} \leq c^-_{j,s-1}$, so $\max(c^-_{j(\alpha-1,k),s}, c^-_{j,s-1}) = \max(c^-_{j(\gamma-1,k),s}, c^-_{j,s-1}) = c^-_{j,s-1}$. According to Eq. (12), we obtain $e^-_{j(\alpha,k),s} > e^-_{j,s+1}$ and $e^-_{j(\gamma,k),s} > e^-_{j,s+1}$, so $\max(e^-_{j(\alpha,k),s} + e^-_{j,s+1}) = e^-_{j(\alpha,k),s}$ and $\max(e^-_{j(\gamma,k),s} + e^-_{j,s+1}) = e^-_{j(\gamma,k),s}$. Thus, we have $\max(c^-_{j(\alpha-1,k),s}, c^-_{j,s-1}) + p_{j,s} + \max(e^-_{j(\alpha,k),s}, e^-_{j,s+1}) = c^-_{j,s-1} + p_{j,s} + e^-_{j(\alpha,k),s}$ and $\max(c^-_{j(\gamma-1,k),s}, c^-_{j,s-1}) + p_{j,s} + \max(e^-_{j(\gamma,k),s}, e^-_{j,s+1}) = c^-_{j,s-1} + p_{j,s} + e^-_{j(\gamma,k),s}$. Since $\gamma < \alpha$, we have $e^-_{j(\alpha,k),s} < e^-_{j(\gamma,k),s}$, which implies $c^-_{j,s-1} + p_{j,s} + e^-_{j(\alpha,k),s} < c^-_{j,s-1} + p_{j,s} + e^-_{j(\gamma,k),s}$. Hence, we conclude that $\max(c^-_{j(\alpha-1,k),s}, c^-_{j,s-1}) + p_{j,s} + \max(e^-_{j(\alpha,k),s}, e^-_{j,s+1}) < \max(c^-_{j(\gamma-1,k),s}, c^-_{j,s-1}) + p_{j,s} + \max(e^-_{j(\gamma,k),s}, e^-_{j,s+1})$.

Similarly, it can be proven that for any position δ to the right of position B, we have $max(c^-_{j(\beta-1,k),s} + c^-_{j,s-1}) + p_{j,s} + max(e^-_{j(\beta,k),s} + e^-_{j,s+1}) <$

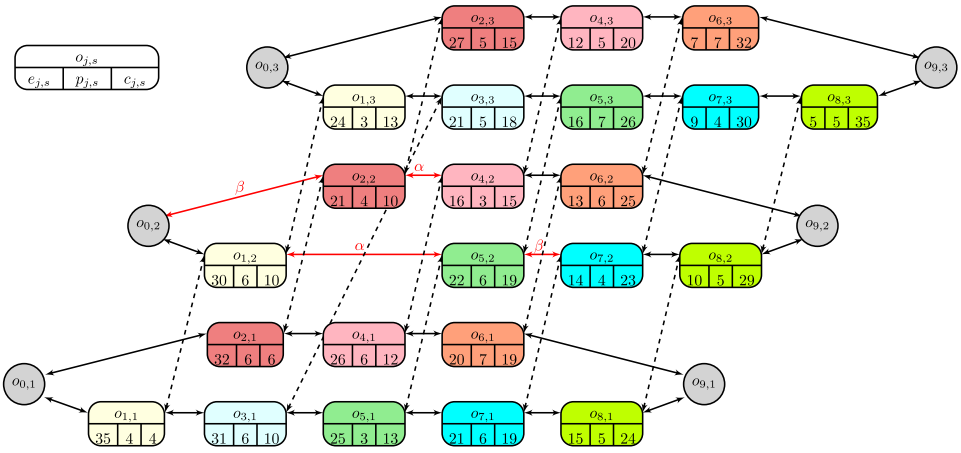
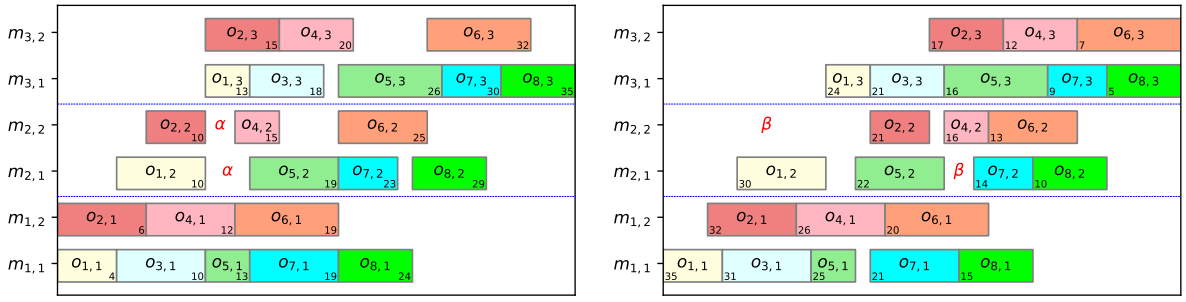


Fig. 5. The solution graph after deleting the operation $o_{3,2}$.



(a) The forward scheduling (b) The backward scheduling

Fig. 6. The Gantt chart after deleting operation $o_{3,2}$.

$max(c^-_{j(\delta-1,k),s}, c^-_{j,s-1}) + p_{j,s} + max\left(e^-_{j(\delta,k),s}, e^-_{j,s+1}\right)$. ■

In summary, according to Theorem 4, when $o_{j,s}$ is reinserted into a machine $m_{s,k}$ in G^-, if $\alpha < \beta$, then positions to the left of α are not superior to position α, and positions to the right of β are not superior to position β. In this case, it is sufficient to try all positions from α to β.

Theorem 5. (*On $m_{s,k}$, if $\beta \le \alpha$, for any position η between position β and position α (including α and β), we have* $max(c^-_{j(\eta-1,k),s}, c^-_{j,s-1}) + p_{j,s} + max(e^-_{j(\eta,k),s} + e^-_{j,s+1}) = c^-_{j,s-1} + p_{j,s} + e^-_{j,s+1}$; *for any position λ to the left of position β or to the right of position α, we have* $max(c^-_{j(\lambda-1,k),s}, c^-_{j,s-1}) + p_{j,s} + max(e^-_{j(\lambda,k),s} + e^-_{j,s+1}) > c^-_{j,s-1} + p_{j,s} + e^-_{j,s+1}$.) Proof

For any position η between β and α (including α and β), we have $\beta \le \eta \le \alpha$. According to Eq. (11), we obtain $c^-_{j(\eta-1,k),s} \le c^-_{j,s-1}$, so $max(c^-_{j(\eta-1,k),s}, c^-_{j,s-1}) = c^-_{j,s-1}$. According to Eq. (12), we obtain $e^-_{j(\eta,k),s} \le e^-_{j,s+1}$, so $max(e^-_{j(\eta,k),s} + e^-_{j,s+1}) = e^-_{j,s+1}$. Thus, we have $max(c^-_{j(\eta-1,k),s}, c^-_{j,s-1}) + p_{j,s} + max(e^-_{j(\eta,k),s} + e^-_{j,s+1}) = c^-_{j,s-1} + p_{j,s} + e^-_{j,s+1}$.

For any position λ to the left of β or to the right of position α, consider first that λ is to the left of β, and we have $\lambda < \beta \le \alpha$. According to Eq. (11), we obtain $c^-_{j(\lambda-1,k),s} \le c^-_{j,s-1}$, so $max(c^-_{j(\lambda-1,k),s}, c^-_{j,s-1}) = c^-_{j,s-1}$. According to Eq. (12), we obtain $e^-_{j(\lambda,k),s} > e^-_{j,s+1}$, so $max(e^-_{j(\lambda,k),s} + e^-_{j,s+1}) = e^-_{j(\lambda,k),s} > e^-_{j,s+1}$. Thus, $max(c^-_{j(\lambda-1,k),s}, c^-_{j,s-1}) + p_{j,s} + max(e^-_{j(\lambda,k),s} + e^-_{j,s+1}) > c^-_{j,s-1} + p_{j,s} + e^-_{j,s+1}$. Then consider that λ is to the right of α, which leads to a similar conclusion. ■

In summary, according to Theorem 5, when $o_{j,s}$ is reinserted into a machine $m_{s,k}$ in G^-, if $\beta \le \alpha$, any position between β and α can achieve the same objective value. Other positions are not superior to those between β and α. According to Theorem 4 and Theorem 5, we can reduce the attempts on certain positions when $o_{j,s}$ is reinserted into G^-, thus reducing the time complexity of the algorithm. To efficiently determine positions A and β, we propose three strategies.

Strategy 1: If operation $o_{j,s}$ is the first one in its stage, then set $\alpha = 1$. If it is the last operation in its stage, then set $\beta = n(m_{s,k}) + 1$.

Strategy 2: When operation $o_{j,s}$ is reinserted to $m_{s,k}$ and $m_{s,k} \neq \zeta_{j,s}$, we need to calculate the forward completion time, $c_{s,k}$ of the last operation on machine $m_{s,k}$, and the backward completion time, $e_{s,k}$ of the first operation on machine $m_{s,k}$. If $c_{s,k} - c^-_{j,s-1} \ge e_{s,k} - e^-_{j,s+1}$, it implies that positions α and β are more likely to appear on the left side of the machine. We can determine positions α and β from the leftmost side to the right. If $c_{s,k} - c^-_{j,s-1} < e_{s,k} - e^-_{j,s+1}$, it implies that positions α and β are more likely to appear on the right side of the machine. We can determine positions α and β from the rightmost side to the left. As shown in Figs. 5 and 6, when operation $o_{3,2}$ is reinserted on machine $m_{2,2}$, we have $c_{s,k} - c^-_{j,s-1} = 25 - 10 = 15 > e_{s,k} - e^-_{j,s+1} = 21 - 21 = 0$. From the Figs. 5 and 6, positions α and β are on the left side of machine $m_{2,2}$. When they are determined from the leftmost side to the right, if position β is found first, the conditions of Theorem 5 are already satisfied, and the search is terminated. If position α is found first, continue to the right until position β is determined, satisfying the conditions of Theorem 4, and then terminate the search. When positions α and β are determined from the rightmost side to the left, if position α is found first, the conditions of Theorem 5 are already satisfied, and the search is terminated. If position β is found first, continue to the right until position α is determined, satisfying the conditions of Theorem 4, and then terminate the search.

Strategy 3: When operation $o_{j,s}$ is reinserted to $m_{s,k}$ and $m_{s,k} = \zeta_{j,s}$. The position of operation $o_{j,s}$ on machine $m_{s,k}$ is denoted as ρ. If $c^-_{j(\rho-1,k),s} \le c^-_{j,s-1}$, then $\rho \le \alpha$ must hold true. In this case, position α needs

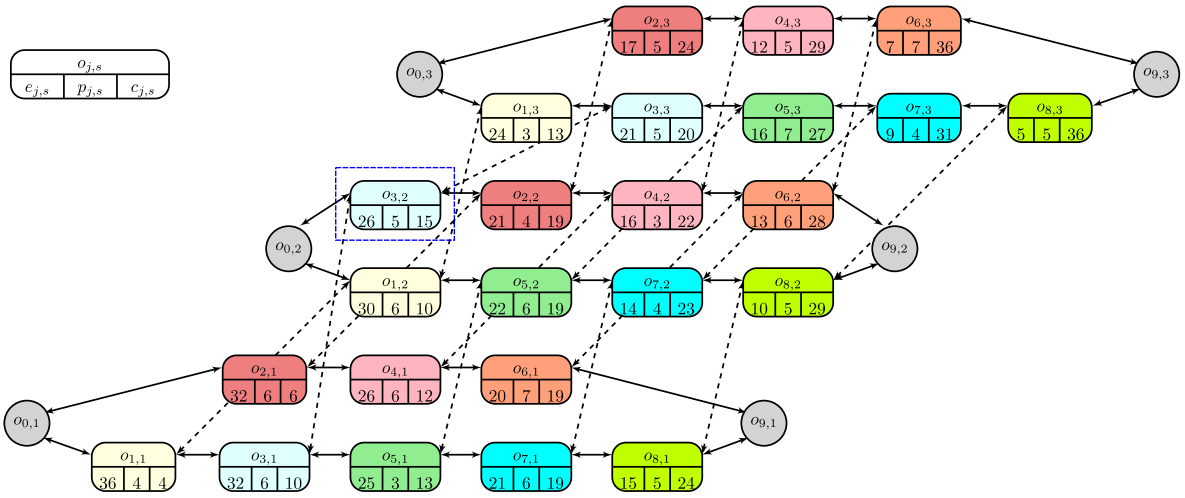
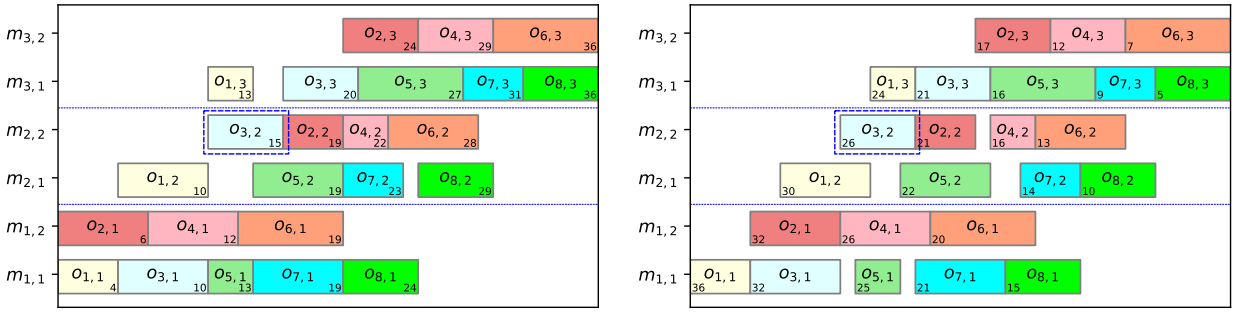


Fig. 7. The solution graph after reinserting operation $o_{3,2}$ to the best position, where the blue rectangle represents its reinserting position. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)



(a) The forward scheduling (b) The backward scheduling

Fig. 8. The Gantt chart after reinserting operation $o_{3,2}$, where the blue rectangle represents its reinserting position. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

to be determined from position ρ towards the right. At the same time, the completion time of the operation is updated when searching for position α. If $e^-_{j(\rho,k),s} \le e^-_{j,s+1}$, then $\alpha < \rho$ must hold true, and position α is determined from position ρ towards the left. If $e^-_{j(\rho,k),s} \le e^-_{j,s+1}$, then $\beta \le \rho$ must hold true. In this case, position β needs to be determined from position ρ towards the left. At the same time, the reverse completion time of the operation is updated when searching for position β. Otherwise, if $\rho < \beta$, then position β is determined from position ρ towards the right. As shown in Figs. 5 and 6, when $o_{3,2}$ is reinserted on machine $m_{2,1}$, $c^-_{j(\rho-1,k),s} = 10 \le c^-_{j,s-1} = 10$, indicating $\rho \le \alpha$. Thus, position α is determined from position ρ towards the right. Also, $e^-_{j(\rho,k),s} = 22 > e^-_{j,s+1} = 21$, indicating $\rho < \beta$. Therefore, position β is determined from position ρ towards the right.

After implementing the above strategies, we find the best insertion position of $o_{3,2}$, as shown in Figs. 7 and 8. The blue rectangle represents its reinserting position. The makespan is reduced from 37 to 36.

4.2.4. The algorithm frame of the accelerated local search based on effective insertion of mandatory operations

Based on the above theorems and methods, Algorithm 3 gives the complete algorithm framework for the accelerated local search based on the effective insertion of mandatory operations. For convenience, the proposed local search is simply denoted as *MOEIALS*. In *MOEIALS* method, we first generate the set of mandatory operations of G by **Algorithm 2**, and then lines 5–34 implement the insertion of mandatory operations-based the effective position, and the accelerated evaluation method is used to calculate the makespan value. First, we select an operation from the set of mandatory operations and obtain the corresponding job and stage information, i.e., $o_{j,s}$, associated with the selected operation. Then, the selected operation, $o_{j,s}$, is removed from graph G. Lines 9–13 determine the position interval of effective insertion. If operation $o_{j,s}$ is reinserted into a machine ($m_{s,k}$) other than the original machine ($\zeta_{j,s}$) at the same stage, the positions α and β are determined according to strategies 1 and 2 proposed in Section 4.2.3. If operation $o_{j,s}$ is reinserted into the original machine ($\zeta_{j,s}$), the positions α and β are determined according to strategies 1 and 3 proposed in Section 4.2.3. At the same time, the forward and backward completion time of the operation is updated when searching for position. After determining α and β, if $\beta \le \alpha$ is satisfied and $c_{j,s-1} + p_{j,s} + e_{j,s+1} < min_path$, then update min_path. Meanwhile, if only position α or β is determined, then $pos_b \leftarrow \alpha$ or $pos_b \leftarrow \beta$. If both positions α and β are determined, pos_b is set to a random position between α. if $c_{j,s-1} + p_{j,s} + e_{j,s+1} \ge min_path$. Line 21 is to try all possible positions, p, $\alpha \le p \le \beta$, and records the value that minimize $L_{G,p}(o_{0,1} \to o_{j,s} \to o_{\mu+1,\sigma})$ as L^*_G. The corresponding best position is denoted p^*. Lines 22–24 are used to update min_path and pos_b. If $min_path < ls_c_{max}$, the operation $o_{j,s}$ is reinserted into position pos_b, and update ls_c_{max}, the parameter $improvement$, the forward and backward completion time of each operation in G. Otherwise, the operation $o_{j,s}$ is reinserted into the original position. The above operation is repeated until no better solution is found after trying the local search at all

mandatory operations.

Algorithm 3 *MOEIALS*(G, c_{max})

Input: G, c_{max}
01: $ls_c_{max} \leftarrow c_{max}$, and *improvement*$\leftarrow$*true*
02: **while** *improvement* **do**
03: *improvement*$\leftarrow$*flase*
04: $MO \leftarrow ObtainingMandatoryOperations(G, c_{max})$ % **Algorithm 2**
05: **for** $i = 1$ to *sizeof*(MO) **do**
06: $j \leftarrow MO_i$ and $s \leftarrow MO_i$
07: Remove $o_{j,s}$ from G, and $min_path \leftarrow INT_MAX$
08: **for** $k = 1$ to τ_s **do**
09: **if** $m_{s,k} \neq \mu_{j,s}$ **then**
10: Determine positions α and β by strategies 1 and 2 % **Section 4.2.3**
11: **else**
12: Determine positions α and β by strategies 1 and 3, and update the forward and backward completion time % **Section 4.2.3**
13: **end if**
14: **if** $\beta \leq \alpha$ **then**
15: **if** $c_{j,s-1} + p_{j,s} + e_{j,s+1} < min_path$ **then**
16: $:min_path \leftarrow c_{j,s-1} + p_{j,s} + e_{j,s+1}$
17: $pos_b \leftarrow \alpha$ or $pos_b \leftarrow \beta$ when only position α or β is determined
18: $pos_b \leftarrow$ a random position between β and α when both positions α an β are determined
19: **end if**
20: **else**
21: $L_G^* = min_{\alpha \leq p \leq \beta} L_{G,p}(o_{0,1} \rightarrow o_{j,s} \rightarrow o_{\mu+1,\sigma})$ and $p^* = arcmin_{\alpha \leq p \leq \beta} L_{G,p}(o_{0,1} \rightarrow o_{j,s} \rightarrow o_{\mu+1,\sigma})$
22: **if** $L_G^* < min_path$ **then**
23: $min_path \leftarrow L_G^*$ and $pos_b \leftarrow p^*$
24: **end if**
25: **end if**
26: **end for**
27: **if** $min_path < ls_c_{max}$ **then**
28: Insert $o_{j,s}$ into pos_b
29: Update the forward and backward completion time of each operation, and obtain ls_c_{max}
30: *improvement*$\leftarrow$*true*, and **break**
31: **else**
32: Insert $o_{j,s}$ into the original position
33: Restore the forward and backward completion times of the operations being updated on machine $\mu_{j,s}$
34: **end if**
35: **end for**
36: **end while**
Output: ls_c_{max}

5. Mandatory operations-based accelerated iterated greedy algorithm in the graph space

To assess the performance of the proposed accelerated local search based on effective insertion of mandatory operations, we integrate the proposed *MOEIALS* method into the modified iterated greedy algorithm (Öztop et al., 2019), forming a Mandatory Operations-based Accelerated Iterated Greedy algorithm, namely *MOAIG*, for solving the HFSP with makespan criterion. The overall *MOAIG* procedure is described in Algorithm 4. Initially, we generate an initial order of jobs by using NEH heuristic outlined in Algorithm 5. Then, the initial sequence is applied a *ImprovementInsertionLocalSearch*(π, c_{max}) method to generate a better solution by inserting each job into the best position. The specific procedure is detailed in Algorithm 6.

Inside the loop, we first implement *Destruction*(π, d) (Algorithm 7) and *Construction*(π, EJ) (Algorithm 8), in which d jobs are randomly chosen from π without repetition, and they are reinserted into the best position, respectively. After destruction and construction, two local search methods, *RISLocalSearch* and *RSSLocalSearch*, are randomly selected to further improve the quality of the current solution. Algorithm 9 is the Reference Insertion Scheme (*RIS*) local search proposed by Pan et al. (Pan et al., 2008). Instead of randomly selecting, the *RIS* local search guides job removal by using a reliable reference solution. Algorithm 10 is the Reference Swap Scheme (*RSS*) local search proposed by Tasgetiren et al. (Tasgetiren et al., 2016). Like *RIS*, the search process is guided by the reference solution. Initially, the referenced job is generated and exchanged with all other jobs in that solution. The above operations involve perturbing the job sequence in the schedule. However, once each job is assigned to a machine, perturbations between machines are not implemented, which limits the search solution space. Therefore, line 2 presents decoding the job sequence into a graph space (Algorithm 1), followed by the implementation of the proposed *MOEIALS* method (Algorithm 3) within this graph space to further expand the search space.

After *MOEIALS*, if the solution obtained is at least as good as the old solution, it is accepted. Otherwise, the new solution is accepted using a simulated annealing-like acceptance criterion with a constant temperature T. If $ls_c_{max} < c_{max}^{best}$, update the global optimal solution π^{best} and the global minimal makespan c_{max}^{best}.

Algorithm 4 *MOAIG*(d, lsp, τ)

Input: *parameters* d, p *and* τ
01: $(\pi, c_{max}) \leftarrow NEH()$ % **Algorithm 5**
02: $(\pi, c_{max}) \leftarrow ImprovementInsertionLocalSearch(\pi, c_{max})$ % **Algorithm 6**
03: $(\pi^{best}, c_{max}^{best}) \leftarrow (\pi, c_{max})$, $(\pi^{old}, c_{max}^{old}) \leftarrow (\pi, c_{max})$
04: **while** *time* $\leq t \times \mu^3 \times \sigma(ms)$ **do**
05: $(\pi, EJ) \leftarrow Destruction(\pi, d)$ % **Algorithm 7**
06: $(\pi, c_{max}) \leftarrow Construction(\pi, EJ)$ % **Algorithm 8**
07: **if** $rand(0, 1) < lsp$ **then**
08: $(\pi, c_{max}) \leftarrow RISLocalSearch(\pi, c_{max}, \pi^{best})$ % **Algorithm 9**
09: **else**
10: $(\pi, c_{max}) \leftarrow RSSLocalSearch(\pi, c_{max}, \pi^{best})$ % **Algorithm 10**
11: **end if**
12: $(G, c_{max}) \leftarrow DecodeToGraph(\pi)$ % **Algorithm 1**
13: $ls_c_{max} \leftarrow MOEIALS(G, c_{max})$ % **Algorithm 3**
14: **if** $c_{max} \leq c_{max}^{old}$ **then**
15: $(\pi^{old}, c_{max}^{old}) \leftarrow (\pi, c_{max})$
16: **else**
17 **if** $rand(0, 1) \leq exp\{-(c_{max} - c_{max}^{old})/T\}$ **then** $\%T = 0.4 \times \sum_{s=1}^{\sigma} \sum_{j=1}^{\mu} p_{j,s} / 10 \times \mu \times \sigma$
18: $(\pi^{old}, c_{max}^{old}) \leftarrow (\pi, c_{max})$
19: **else**
20: $(\pi, c_{max}) \leftarrow (\pi^{old}, c_{max}^{old})$
21: **end if**
22: **end if**
23: **if** $ls_c_{max} < c_{max}^{best}$ **then**
24: $(\pi^{best}, c_{max}^{best}) \leftarrow (\pi, ls_c_{max})$
25: **end if**
26: **end while**
Output: c_{max}^{best}

Algorithm 5 *NEH*()

001: $tp_j \leftarrow \sum_{s=1}^{\sigma} p_{j,s}$, $j = 1, 2, \cdots, \mu$
02: $\pi^{seed} \leftarrow$ Sort the jobs according to descending tp_j
03: $\pi \leftarrow \{\pi_1^{seed}\}$
04: **for** $j = 2$ to μ **do**
05: Test π_j^{seed} at all possible positions of π
06: Insert π_j^{seed} at the position resulting in lowest makespan c_{max}
07: end for
Output: π, c_{max}

Algorithm 6 *ImprovementInsertionLocalSearch*(π, c_{max})

Input: π, c_{max}
01: $\pi^{old} \leftarrow \pi$
02: *improvement*$\leftarrow$*true*
03: **while** *improvement* **do**
04: *improvement*$\leftarrow$*false* and $\pi^{old} \leftarrow \pi$
05: **for** $j = 1$ to μ **do**

(*continued on next page*)

(*continued*)

Algorithm 6 *ImprovementInsertionLocalSearch*(π, c_{max})

06: Record the position of π_j^{old} in π as *pos*, remove π_j^{old} from π
07: Test π_j^{old} at all possible positions of π
08: Record makespan at the best position as c'_{max}, corresponding to position *pos*′
09: **if** $c'_{max} < c_{max}$ **then**
10: Insert π_j^{old} to position *pos*′ of π
11: $c_{max} \leftarrow c'_{max}$ and *improvement*←*true*
12: **else**
13: Restore π_j^{old} to position *pos* of π
14: **end if**
15: **end for**
16: **end while**
Output: π, c_{max}

Algorithm 7 *Destruction*(π, d)

Input: π, d
01: $EJ \leftarrow \{\}$
05: **for** $j = 1$ to d **do**
06: Randomly remove a job from π and append it to *EJ*
11: end for
Output: π, EJ

Algorithm 8 *Construction*(π, EJ)

Input: π, d
01: **for** $j = 1$ **to** *sizeof*(EJ) **do**
02: Test EJ_j at all possible positions of π
03: Insert EJ_j at the position resulting in lowest makespan c_{max}
04: **end for**
Output: π, c_{max}

Algorithm 9 *RISLocalSearch*$(\pi, c_{max}, \pi^{best})$

Input: π, c_{max}, π^{best}
01: *iter*←1, *j*←1
02: **while** *iter* $\le \mu$ **do**
03: Find π_j^{best} in π and record the position as *pos*, remove π_{pos} from π
04: Test π_{pos} at all possible positions of π
05: Record makespan at the best position as c'_{max}, corresponding to position *pos*′
06: **if** $c'_{max} < c_{max}$ **then**
07: Insert π_{pos} to position *pos*′ of π
08: $c_{max} \leftarrow c'_{max}$ and *iter*←1
09: **else**
10: Restore π_{pos} to position pos of π, and *iter*←*iter* + 1
11: **end if**
12: $j \leftarrow (j+1)\%\mu$
13: **end while**
Output: π, c_{max}

Algorithm 10 *RSSLocalSearch*$(\pi, c_{max}, \pi^{best})$

Input: π, c_{max}, π^{best}
01: *iter*←1, *j*←1
02: **while** *iter* $\le \mu$ **do**
03: Find π_j^{best} in π and record the position as *pos*
04: Test swap π_{pos} with all other jobs in π
05: Record the lowest makespan as c'_{max}, corresponding to the job to be swapped is π_{swap}
06: **if** $c'_{max} < c_{max}$ **then**
07: Swap π_{pos} with π_{swap} in π
08: $c_{max} \leftarrow c'_{max}$ and *iter*←1
09: **else**
10: *iter*←*iter* + 1
11: **end if**
12: $j \leftarrow (j+1)\%\mu$

(*continued on next column*)

(*continued*)

Algorithm 10 *RSSLocalSearch*$(\pi, c_{max}, \pi^{best})$

13: **end while**
Output: π, c_{max}

6. Experimental results and analysis

6.1. Experimental environment and test instances

For the experimental environment, all algorithms are written by using the C++ language and conducted on a PC with an Intel Core i7 12,700 processor and 16 GB of DDR5 RAM. The operating system is Windows 11 Professional 64-bit.

Jose's benchmark test dataset: According to the characteristics of HFSP, we adopt 480 test instances (240 small-size and 240 large-size instances) proposed by Fernandez-Viagas and Framinan (Fernandez-Viagas & Framinan, 2020). In (Fernandez-Viagas & Framinan, 2020), substantial experimental results demonstrate that the designed 480 test instances exhibit superior performance across various metrics, highlighting the inadequacies of using randomly generated instances as benchmarks for the problem. The 240 small-size instances are generated by using the following specific combinations of μ (the number of jobs), σ (the number of stages), where $\mu \in \{10, 15, 20, 25, 30, 35\}$ and $\sigma \in \{5, 10, 15, 20\}$. Each of the 24 combinations (6×4) mentioned above consists of 10 instances with the same scale size, resulting in a total of 240 instances. The difference between the 240 large-size instances and the 240 small-size instances is only in the number of jobs, i.e., $\mu \in \{40, 80, 120, 160, 200, 240\}$. All the 480 test instances can be found in https://github.com/wangyuting9836/HFSP_MOAIG.

Liao's benchmark test dataset: We adopt the 96 instances proposed by Carlier and Neron (Carlier & Neron, 2000). The 96 instances are generated by using the following specific combinations of the number of jobs (i.e., *j*10 and *j*15), the number of stages (i.e., *c*5 and *c*10), and the number of identical machines at each stage (five types of configurations, i.e., '*a*', '*b*', '*c*', '*d*', and '*e*'). The notation '*a*' indicates the presence of a single machine at the intermediate stage and three machines at all other stages. '*b*' indicates the presence of a single machine at the first stage and three machines at all other stages. '*c*' indicates the presence of three machines at all stages. '*d*' indicates the presence of two machines at the intermediate stage and three machines at all other stages. '*e*' indicates the presence of two machines at the intermediate stage, a single machine at the last stage and three machines at all other stages. All 96 test instances can be found in https://github.com/wangyuting9836/HFSP_MOAIG.

The selection of comparative algorithms: We select eight comparative algorithms. More precisely, Öztop et al. proposed four metaheuristics based on iterated greedy algorithm and one variable block insertion heuristic in 2019, namely IG_{RS}, IG_{GR}, IG_{ALL}, *IGT*, and *VBIH*, respectively (Öztop et al., 2019). Their experimental results demonstrated that the five proposed algorithms are better than DABC (Pan et al., 2014) and IDABC (Cui & Gu, 2015). In 2021, Lin et al. proposed a chaos-enhanced simulated annealing (*CSA*) algorithm to minimize makespan of HFSP (Lin et al., 2021). In 2022, Fernandez-Viagas proposed a speed-up IG (*IGwS*) algorithm to minimize the makespan of HFSP (Fernandez-Viagas, 2022). In 2024, Kuang et al. designed a two-stage cross-local search algorithm (*TSCNSA*) to solve HFSP (Kuang et al., 2024). These comparative algorithms and our algorithm are non-population and are used to solve the HFSP with makespan criterion. To facilitate the implementation of all algorithms, the C++ source codes of nine algorithms can be found in https://github.com/wangyuting9836/HFSP_MOAIG. Please download them yourself.

To assess the performance of all algorithms, the runtime is uniformly used as the termination condition for the experiments in this section. Its value is $t \times \mu^3 \times \sigma$, where t is a freely definable parameter, measured in milliseconds, μ and σ are the numbers of jobs and stages in the test

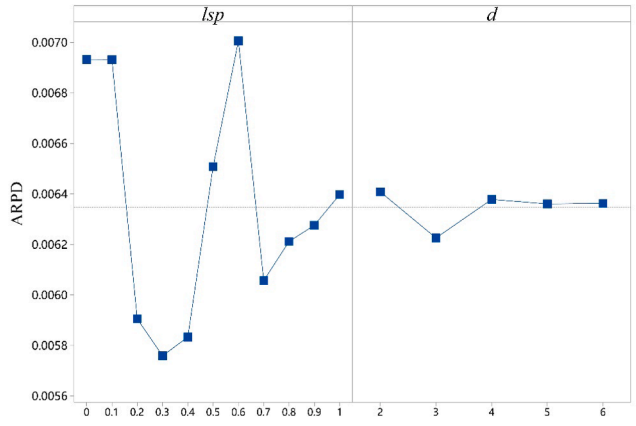


Fig. 9. Main effects plot for parameters in MOAIG.

instance, respectively. The relative percentage deviation (*RPD*) is employed as the evaluation indicator (Y. Wang, Han, Wang, Tasgetiren, et al., 2023). Its computation for each instance is given in Eq. (13).

$$RPD_i = (c^i_{max} - makespan_{best})/makespan_{best} \tag{13}$$

In Eq. (13), c^i_{max} is the makespan value obtained by the i th algorithm, where i belongs to *IGRS* (Öztop et al., 2019), *IGGR* (Öztop et al., 2019), *IGALL* (Öztop et al., 2019), *IGT* (Öztop et al., 2019), *VBIH* (Öztop et al., 2019), *CSA* (Lin et al., 2021), *IGwS* (Fernandez-Viagas, 2022), *TSCNSA* (Kuang et al., 2024), and *MOAIG*. The $makespan_{best}$ and $makespan_{worst}$ denote the best and the worst makespan found by all algorithms, respectively. It is unequivocal that the value of RPD_i ranges from 0 to 1. A performance is deemed superior as RPD_i approaches 0, on the contrary, the tendency 1 indicates deteriorating performance. In addition, we give the average RPD_i (denoted as $ARPD_i = \sum_{r=1}^{rep} RPD_i^r/rep$), the minimal RPD_i (denoted as $MINRPD_i = min(RPD_i^1, RPD_i^2, \cdots, RPD_i^{rep})$), and the maximal RPD_i (denoted as $MAXRPD_i = max(RPD_i^1, RPD_i^2, \cdots, RPD_i^{rep})$), where *rep* is the number of independent runs.

Table 2
Comparison between three compared local search and the proposed *MOEIALS*.

μ	σ	makespan				Running time (milliseconds)			
		LSwS	*LS*	*ALS*	*MOEIALS*	*LSwS*	*LS*	*ALS*	*MOEIALS*
10	5	477	477	477	**474**	**0**	**0**	***0***	**0**
10	10	**939**	**939**	**939**	941	**0**	**0**	***0***	**0**
10	15	1059	1059	1059	**1058**	**0**	1.6	***0***	**0**
10	20	**1525**	**1525**	**1525**	**1525**	**0**	1.5	***0***	**0**
15	5	**604**	**604**	**604**	605	**0**	**0**	***0***	1.6
15	10	1072	**1071**	**1071**	1072	**0**	1.5	***0***	**0**
15	15	**1284**	**1284**	**1284**	1286	**0**	6.3	***0***	**0**
15	20	**1667**	1669	1669	1672	**0**	10.9	*3.1*	**0**
20	5	774	777	777	**770**	**0**	1.6	***0***	**0**
20	10	1110	**1104**	**1104**	1110	**0**	4.7	***0***	**0**
20	15	1495	**1494**	**1494**	1495	**0**	10.9	***0***	**0**
20	20	**1666**	**1666**	**1666**	**1666**	**0**	34.3	*1.6*	**0**
25	5	877	**874**	**874**	878	**0**	3.2	***0***	**0**
25	10	**1191**	1193	1193	**1191**	**0**	9.4	***0***	1.6
25	15	1479	1480	1480	**1478**	**0**	37.4	*3.1*	**0**
25	20	1718	**1714**	**1714**	1719	**0**	65.6	*1.6*	**0**
30	5	922	922	922	**917**	**0**	4.7	***0***	**0**
30	10	1283	1285	1285	**1280**	**0**	18.7	*1.6*	**0**
30	15	**1714**	1718	1718	1715	**0**	74.9	*4.8*	**0**
30	20	2126	**2114**	**2114**	2126	**0**	131.4	*6.1*	**0**
35	5	1093	**1087**	**1087**	1092	**0**	7.8	*1.6*	**0**
35	10	1525	1526	1526	**1522**	**0**	35.9	*3.2*	**0**
35	15	**1851**	1853	1853	**1851**	**0**	92.1	*4.7*	**0**
35	20	2102	2101	2101	**2099**	**0**	192.3	*12.4*	**0**
40	5	1115	1110	1110	**1109**	**0**	9.4	***0***	**0**
40	10	1592	1590	1590	**1588**	**0**	45.4	*1.5*	**0**
40	15	1819	1820	1820	**1814**	1.5	136.1	*4.6*	**0**
40	20	**2237**	2238	2238	**2237**	**0**	352.9	*15.8*	**0**
80	5	2220	2220	2220	**2216**	**0**	164	*1.6*	1.6
80	10	**2264**	2266	2266	**2264**	**0**	543.8	*14*	**0**
80	15	2834	**2829**	**2829**	2831	**1.6**	1546.8	*37.5*	3.2
80	20	3039	3041	3041	**3033**	1.6	3063.8	*72.1*	**0**
120	5	3159	3160	3160	**3157**	**1.6**	398.5	*7.7*	**1.6**
120	10	3506	3507	3507	**3504**	1.6	1984.4	*34.3*	**0**
120	15	3829	**3826**	**3826**	3827	7.8	9122	*139*	**0**
120	20	4229	**4224**	**4224**	**4224**	12.6	24134.4	*334.2*	**0**
160	5	4356	**4348**	**4348**	4355	1.6	1534.4	*20.2*	**0**
160	10	**4636**	4642	4642	**4636**	3.1	4351.6	*59.3*	**1.6**
160	15	4749	4747	4747	**4744**	12.5	19431.4	*206.1*	**0**
160	20	4773	**4769**	**4769**	**4769**	27.9	76112.6	*781.4*	**3.1**
200	5	5248	**5244**	**5244**	5246	1.5	1982.8	*18.8*	**0**
200	10	**5554**	5555	5555	5555	4.6	9564.1	*89.2*	**3.1**
200	15	5751	**5747**	**5747**	5749	20.3	51554.7	*420.4*	**1.5**
200	20	5746	**5741**	**5741**	5746	37.7	132170.4	*1148.2*	**0**
240	5	6267	**6260**	**6260**	6266	1.5	3237.7	*29.5*	**0**
240	10	**7122**	7126	7126	7123	12.6	18473.4	*131.2*	**1.5**
240	15	6784	**6779**	**6779**	**6779**	17.3	78170.3	*559.3*	**1.6**
240	20	**6993**	6994	6994	6994	67.3	186354.6	*1381.3*	**0**
mean		2737	**2736**	**2736**	**2736**	4.92	13024.71	*115.65*	**0.46**

6.2. Parameter calibration

The proposed MOAIG algorithm has two important parameters, namely d and lsp. The first is the number of destruction jobs, and the second signifies the probability of executing the different swap-based and insertion-based *RLS* methods. Grounded in the existing literature, we employed the Design of Experiments (DOE) and Analysis of Variance (ANOVA) methodologies to analyze the experimental data for each parameter. We consider five levels for d, i.e., $d \in [2, 3, 4, 5, 6]$ and eleven levels for lsp, i.e., $lsp \in [0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1]$, resulting in a total of 55 parameter combinations. We select Jose's benchmark test dataset for the parameter calibration under the termination condition of $30*\mu*\sigma$ milliseconds. Each parameter combination independently runs 5 times, and their average *RPD* values are calculated to reduce the impact of randomness in the algorithm. Fig. 9 shows the 95 % Least Significant Difference (LSD) related to the two parameters.

For the parameter d in the destructive reconstruction process, being too small will limit the exploration of the local solution. Conversely, too large may introduce excessive disruption and reconstruction, which can increase computational overhead. For the parameter lsp, it has the most significant impact on the algorithm. Setting an appropriate lsp value is crucial for balancing the performance of different *RLS* methods. $lsp = 0$ indicates that only the swap-based *RLS* method is adopted, while $lsp = 1$, only the insertion-based *RLS* method is adopted. In general, the insertion-based local search performs better than the swap-based local search; however, the former takes a little more time than the latter. Therefore, through parameter calibration, we find that $lsp = 0.3$ has a better *ARPD* value than others. Based on the analysis conducted above, the two parameters are set as $d = 3$ and $lsp = 0.3$.

6.3. Verify the performance of the accelerated local search based on effective insertion of mandatory operations

The most important innovation of this paper is to propose *MOEIALS*. To verify its effectiveness, this paper selects three exiting local search methods as comparison. The first is the critical path-based local search, called *LSwS* (Fernandez-Viagas, 2022). The second is the classic local search method *LS*, referred to (Fernandez-Viagas, 2022), which reinserts every operation in every stage, and each insertion has to calculate the value of the objective function from scratch. It greatly improves the time complexity of the algorithm. Therefore, to reduce its time complexity, we integrate the accelerated evaluation method proposed in Section 4.2.2 into *LS*, and obtain the third compared method, *ALS*. To implement the four local search methods and easily verify their performances, all methods start with a same random sequence. Then a solution is decoded into a graph, and a local search is performed directly until the solution cannot be improved. Algorithm 11 is the framework for testing the performance of four local search methods. At this time, we can obtain the minimum objective function value and the time taken to invoke each local search as the evaluation indicators. Table 2 reports the experimental results of all the comparison algorithms grouped by 10 instances with the same scale size on Jose's benchmarks.

Algorithm 11 *Algorithmfortestingfourlocalsearchmethods*

```
π←randomjobsequence
(G, c_max)←DecodeToGraph(π) % Algorithm 1
LSwS_c_max←LSwS(G, c_max)
(G, c_max)←DecodeToGraph(π) % Algorithm 1
LS_c_max←LS(G, c_max)
(G, c_max)←DecodeToGraph(π) % Algorithm 1
ALS_c_max←ALS(G, c_max)
(G, c_max)←DecodeToGraph(π) % Algorithm 1
MOEIALS_c_max←MOEIALS(G, c_max) % Algorithm 3
```

Output: $LSwS_c_{max}, LS_c_{max}, ALS_c_{max}, MOEIALS_c_{max}$

In Table 2, for the average makespan values of each $\mu \times \sigma$ instance grouped by 10 instances with the same scale size, all the algorithms can obtain better solutions due to they can expand the search space by perturbing the operations in the graph. In comparison, our *MOEIALS* method can obtain the minimal mekspan values on 26/48 instances, while there are 15/48, 22/48, 22/48 smallest values obtained by *LSwS*, *LS*, and *ALS*, respectively. We focus more on the second evaluation indicator, *Running time*. From the average running time on all 480 instances, our *MOEIALS* method (0.46 ms) is smaller than *LSwS* (4.92 ms), *ALS*(115.65 ms), and *LS* (13024.7 ms). It clearly shows that the proposed *MOEIALS* method can reduce the computation time. Compared to the *LS* method, *ALS* the average values are reduced by 99 % (as shown in italics in Table 2). The only difference between *LS* and *ALS* methods is that the latter incorporates our accelerated evaluation method and does not need to calculate its makespan value from scratch when the operation is reinserted. This distinction drastically reduces the overall computation time. Our algorithm's superiority derives from its implementation of an accelerated local search method based on mandatory operations effective insertion. In contrast to other methods that select the operations on the critical path or on each stage to reinsert all possible locations, leading to more extensive computation times, our approach only considers the operations on the mandatory operation set, and reinserts them to the effective insertion positions that are the ones where the makespan is less than or equal to the other insertion positions. The superiority of the proposed method *MOEIALS* can be further demonstrated in Table 3.

Table 3
Running time results of the variance analysis.

Algorithms	Ranks	CN	Min	Max	Mean	Std. Dev
MOEIALS	**1.83**	480	**0.00**	**16.00**	**0.46**	**2.65**
LSwS	1.98	480	0.00	156.00	4.92	14.65
ALS	2.57	480	0.00	2813.00	115.63	331.70
LS	3.62	480	0.00	378031.00	13024.71	41168.03
P-value	**0.000**					

6.4. Verify the performance of the proposed MOAIG algorithm

To evaluate the effectiveness of our proposed *MOAIG* algorithm, we compare it with *IGRS*, *IGGR*, *IGALL*, *IGT*, and *VBIH* proposed in 2019 (Öztop et al., 2019), *CSA* proposed in 2021 (Lin et al., 2021), *IGWS* proposed in 2022 (Fernandez-Viagas, 2022), and *TSCNSA* proposed in 2024 (Kuang et al., 2024). The above nine algorithms are non-population and are used to solve the HFSP with makespan criterion. Thus, it is reasonable and fair to choose the above algorithms. All algorithms have been tested on the 48 instance sets that are grouped by 10 instances with the same scale size on Jose's benchmarks. Tables 4–6 show the *MAXRPD*, *MINRPD*, and *ARPD* (*MAX*, *MIN*, *AVG*, for short) values obtained by each algorithm on each scale of instance under the termination times, $0.003 \times \mu^3 \times \sigma$, $0.006 \times \mu^3 \times \sigma$, $0.01 \times \mu^3 \times \sigma$ milliseconds, respectively, with the best value marked in bold.

In Table 4, the RPD values of *MAX*, *MIN*, *AVG* obtained by *IGRS*, *IGGR*, *IGALL*, *IGT*, *VBIH*, *CSA*, *IGWS*, *TSCNSA*, and MOAIG have been reported under the same termination time $0.003 \times \mu^3 \times \sigma$ milliseconds.

From the number of the minimal *MAX*, *MIN*, *AVG* values in Table 4, *MOAIG* outperforms other algorithms on 27/48, 23/48, 27/48 instance sets, followed that *TCANSA* (6/48,13/48,6/48), *IGWS* (9/48, 8/48,10/48), *CSA* (4/48,8/48,4/48), *IGT* (4/48,9/48,3/48), *IGGR* (0, 5/48, 1/48), *VBIH* (0, 5/48,0), *IGALL* (0,4/48,0,) and *IGRS* (0, 3/48, 0). Overall, with respect to the mean values of *MAX*, *MIN*, *AVG* for 480 instances obtained by *MOAIG* (0.965, 0.236, 0.586) is significantly lower than that *IGWS* (1.142, 0.348, 0.730), *IGT* (1.161, 0.417, 0.773), *CSA* (1.696, 0.692, 1.169), *TCANSA* (2.572, 0.840, 1.665), *IGRS* (1.329, 0.470, 0.868), *IGGR* (1.330, 0.507, 0.899), *IGALL* (1.255, 0.504, 0.877), and *VBIH* (1.248, 0.477, 0.844). These results demonstrate the superior performance of *MOAIG*. This can be attributed to mandatory operations-based accelerated local search. In fact, the framework of these comparison algorithms is basically the same, including the initialization and

Table 4
Average *RPD* values of *MAX*, *MIN*, *AVG* of nine algorithms grouped by 10 instances with the same scale size on Jose's benchmarks under $t = 0.003$.

μ	σ	*IGWS*			*IGT*			*CSA*			*TCANSA*			*IGRS*			*IGGR*			*IGALL*			*VBIH*			*MOAIG*		
		MAX	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*
10	5	**0.254**	0.078	**0.113**	0.749	0.600	0.630	0.668	0.104	0.341	1.647	**0.000**	0.765	1.052	0.600	0.696	0.749	0.600	0.635	0.718	0.600	0.624	0.875	0.600	0.661	0.971	0.292	0.428
10	10	**0.372**	0.321	0.341	0.489	0.375	0.398	0.779	**0.111**	**0.296**	1.467	0.155	0.651	0.426	0.375	0.396	0.529	0.426	0.447	0.375	0.375	0.375	0.514	0.375	0.403	0.428	0.375	0.386
10	15	0.537	0.180	**0.327**	0.909	0.517	0.698	0.763	0.219	0.447	1.354	**0.126**	0.583	1.084	0.517	0.746	1.121	0.638	0.893	0.859	0.517	0.637	1.264	0.517	0.718	**0.419**	0.374	0.399
10	20	**0.244**	0.135	**0.165**	0.822	0.442	0.538	1.362	0.265	0.752	1.715	**0.132**	0.772	0.657	0.442	0.527	0.796	0.442	0.606	0.642	0.442	0.515	0.810	0.442	0.562	0.324	0.135	0.173
15	5	1.466	0.576	0.973	**1.147**	0.708	**0.847**	1.159	0.613	0.866	2.041	**0.256**	1.086	1.560	0.613	0.978	1.558	0.661	1.061	1.349	0.745	0.978	1.299	0.690	0.942	1.272	0.599	0.926
15	10	1.102	0.327	0.706	1.199	0.659	0.847	1.142	0.451	0.721	1.966	**0.286**	1.168	1.302	0.685	0.936	1.342	0.602	0.890	1.069	0.557	0.823	1.148	0.648	0.915	**1.007**	0.381	**0.684**
15	15	1.033	0.352	0.631	1.121	0.476	0.757	1.643	0.471	0.945	1.832	0.495	1.160	1.241	0.457	0.796	1.112	0.483	0.790	1.173	0.557	0.829	1.053	0.587	0.812	**0.810**	**0.223**	**0.510**
15	20	1.168	0.385	0.814	1.476	0.738	1.027	1.637	0.412	0.953	2.076	**0.343**	1.110	1.630	0.805	1.233	1.662	0.845	1.161	1.417	0.778	1.062	1.604	0.821	1.137	**0.992**	0.379	**0.694**
20	5	1.543	0.606	1.080	1.303	0.499	0.903	2.003	0.942	1.500	2.418	0.700	1.489	1.789	0.431	1.051	1.620	0.670	1.106	1.305	0.499	0.971	1.480	0.691	1.076	**1.103**	**0.348**	**0.782**
20	10	1.805	0.395	1.098	1.468	0.386	0.887	1.470	0.520	1.070	2.763	0.524	1.785	1.663	0.533	1.040	1.441	0.494	0.964	1.521	0.372	0.966	1.573	0.474	1.038	**1.339**	**0.359**	**0.878**
20	15	1.400	0.409	0.848	1.352	0.367	**0.789**	1.510	0.483	0.942	2.761	0.577	1.573	1.550	0.374	0.945	1.478	0.365	0.859	1.359	0.538	0.916	1.537	0.689	1.077	**1.304**	**0.331**	0.813
20	20	1.490	0.364	0.917	1.291	0.375	0.844	1.760	0.392	1.072	2.785	0.702	1.662	1.248	0.425	0.805	1.584	0.461	0.986	1.394	**0.307**	0.894	1.618	0.533	1.016	**1.207**	0.327	**0.737**
25	5	0.852	0.409	0.659	0.896	**0.133**	0.474	1.520	0.837	1.138	1.759	0.441	1.085	0.928	0.361	0.684	0.951	0.242	0.623	0.869	0.299	0.574	0.823	0.222	0.547	**0.692**	0.206	**0.453**
25	10	1.463	0.546	0.988	1.496	0.634	1.049	1.508	0.510	1.030	2.612	0.733	1.575	1.894	0.592	1.175	1.670	0.664	1.091	1.592	0.721	1.111	1.729	0.723	1.268	**1.333**	**0.241**	**0.772**
25	15	2.175	0.708	1.459	1.977	0.773	1.350	2.389	0.977	1.647	4.001	1.268	2.650	2.069	0.880	1.432	2.077	0.668	1.371	1.949	0.758	1.384	2.099	0.883	1.491	**1.465**	**0.412**	**0.975**
25	20	1.601	0.563	1.085	1.730	0.727	1.198	1.928	0.624	1.318	3.432	0.642	1.785	1.685	0.487	1.033	1.754	0.643	1.146	1.607	0.420	1.032	1.744	0.616	1.215	**1.382**	**0.363**	**0.821**
30	5	1.325	0.513	0.967	1.252	0.369	0.768	2.025	1.190	1.645	2.885	0.766	1.745	1.439	0.385	0.862	1.271	**0.270**	0.760	1.269	0.475	0.853	1.474	0.374	0.836	**0.885**	0.310	**0.580**
30	10	1.727	0.571	1.143	**1.475**	**0.330**	0.991	1.998	1.074	1.452	3.669	0.826	2.266	1.822	0.653	1.241	1.834	0.542	1.218	1.858	0.721	1.290	1.837	0.423	1.215	1.591	0.416	**0.928**
30	15	1.850	0.519	1.192	1.739	**0.404**	1.108	2.197	0.749	1.515	3.785	0.990	2.271	1.992	0.628	1.224	1.845	0.432	1.148	1.879	0.695	1.280	1.917	0.711	1.345	**1.666**	0.409	**1.011**
30	20	2.309	0.891	1.567	**1.530**	0.624	1.083	2.704	0.958	1.914	3.499	1.134	2.264	1.996	0.729	1.368	1.838	0.752	1.239	1.775	0.849	1.285	1.863	1.024	1.382	1.547	**0.244**	**0.937**
35	5	0.976	0.439	0.697	0.847	0.270	0.564	1.386	0.836	1.140	2.209	0.513	1.311	0.931	0.269	0.594	1.094	0.227	0.636	0.933	0.350	0.583	0.955	0.347	0.671	**0.756**	**0.126**	**0.415**
35	10	1.420	0.537	0.945	1.218	0.339	0.742	1.480	0.803	1.155	2.292	**0.151**	1.122	1.551	0.526	0.952	1.231	0.389	0.785	1.573	0.334	0.936	1.335	0.409	0.906	**1.131**	0.200	**0.697**
35	15	1.788	0.656	1.243	**1.407**	**0.248**	**0.911**	2.177	0.845	1.483	3.544	0.576	2.266	1.737	0.685	1.161	1.580	0.563	1.097	1.776	0.631	1.243	1.950	0.827	1.340	1.478	0.509	1.012
35	20	2.020	0.690	1.313	1.672	**0.323**	1.032	2.149	0.948	1.580	3.015	0.868	1.933	1.807	0.528	1.162	1.844	0.650	1.201	1.904	0.593	1.216	1.862	0.805	1.282	**1.552**	0.382	**0.917**
40	5	0.912	0.474	0.656	0.951	0.311	0.581	1.491	0.915	1.199	2.013	0.485	1.226	1.165	0.301	0.694	0.961	0.190	**0.539**	0.979	0.278	0.627	0.963	0.307	0.656	**0.836**	**0.104**	0.560
40	10	1.326	0.516	0.925	1.430	0.390	0.884	1.734	0.961	1.340	3.227	1.009	2.034	1.711	0.496	1.114	1.570	0.342	0.966	1.585	0.529	1.081	1.681	0.463	1.087	**1.289**	**0.273**	**0.722**
40	15	2.198	0.511	1.305	1.735	0.454	1.076	2.308	0.849	1.551	3.702	0.993	2.315	2.013	0.722	1.314	1.959	0.551	1.193	1.714	0.548	1.140	2.263	0.850	1.516	**1.585**	**0.369**	**0.929**
40	20	1.824	0.497	1.185	1.932	0.363	1.092	2.491	0.992	1.735	4.105	1.022	2.613	1.788	0.463	1.131	1.876	0.509	1.144	1.749	0.625	1.197	2.071	0.848	1.439	**1.544**	**0.193**	**0.887**
80	5	0.036	**0.000**	0.012	0.031	**0.000**	0.015	0.030	0.005	0.022	**0.016**	**0.000**	**0.004**	0.141	**0.000**	0.049	0.046	**0.000**	0.017	0.025	**0.000**	0.013	0.031	**0.000**	0.010	0.069	0.005	0.050
80	10	**1.169**	0.287	0.779	1.251	0.268	0.780	1.528	0.601	1.057	4.370	2.014	2.949	1.904	0.654	1.224	1.736	0.626	1.167	1.756	0.579	1.232	1.804	0.424	1.150	1.488	**0.123**	**0.773**
80	15	1.646	0.338	0.936	1.554	0.435	0.948	2.231	0.762	1.409	3.924	1.525	2.691	2.144	0.738	1.349	1.794	0.604	1.190	1.593	0.444	1.043	1.909	0.625	1.195	**1.258**	**0.233**	**0.744**
80	20	2.394	0.664	1.508	1.739	0.506	1.125	4.000	1.259	2.466	5.707	2.143	3.829	2.051	0.618	1.310	1.871	**0.355**	1.152	2.023	0.496	1.306	2.078	0.899	1.446	**1.697**	0.418	**1.001**
120	5	0.006	**0.000**	0.003	0.007	**0.000**	0.003	**0.003**	**0.000**	**0.001**	0.006	**0.000**	0.001	0.028	0.003	0.013	0.030	**0.000**	0.007	0.016	**0.000**	0.004	0.026	**0.000**	0.009	0.010	**0.000**	0.003
120	10	**0.171**	0.053	**0.095**	0.268	**0.012**	0.140	0.373	0.108	0.202	0.537	0.239	0.382	0.466	0.093	0.231	0.452	0.110	0.259	0.482	0.136	0.278	0.387	0.093	0.226	0.331	0.046	0.162
120	15	**1.496**	**0.321**	**0.912**	1.832	0.555	1.139	2.778	0.839	1.785	5.387	2.664	3.966	2.032	0.708	1.332	2.345	0.876	1.543	2.045	0.828	1.437	1.994	0.649	1.260	1.618	**0.499**	1.003
120	20	2.322	0.443	1.318	1.742	0.528	1.105	4.751	2.181	3.371	5.476	2.787	4.119	2.236	0.992	1.588	2.462	0.952	1.649	2.122	0.630	1.415	2.211	0.778	1.505	**1.565**	**0.225**	**0.926**
160	5	0.008	**0.000**	0.005	0.027	0.002	0.012	0.005	**0.000**	0.003	**0.002**	**0.000**	**0.000**	0.070	**0.000**	0.023	0.032	0.002	0.011	0.014	0.002	0.006	0.021	0.002	0.009	0.030	0.005	0.014
160	10	0.030	0.002	**0.011**	0.141	0.005	0.054	**0.025**	**0.002**	0.012	0.050	0.002	0.023	0.224	0.022	0.098	0.150	0.012	0.060	0.137	0.012	0.064	0.115	0.005	0.048	0.102	0.014	0.045
160	15	1.421	**0.198**	**0.726**	1.805	0.797	1.276	3.242	0.995	2.134	4.991	2.650	3.880	2.192	0.751	1.492	2.382	1.114	1.722	2.217	0.915	1.548	1.873	0.626	1.196	**1.379**	**0.277**	0.763
160	20	1.767	0.375	1.072	2.035	0.688	1.356	4.349	2.065	3.189	5.897	3.110	4.373	2.449	0.890	1.663	2.486	0.934	1.719	2.386	1.174	1.808	1.942	0.742	1.304	**1.433**	**0.151**	**0.809**
200	5	0.024	0.004	0.011	0.025	0.012	0.015	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	0.062	**0.000**	0.029	0.025	0.012	0.016	0.027	0.012	0.016	0.031	**0.000**	0.014	0.049	0.014	0.030
200	10	0.053	**0.006**	0.026	0.113	0.027	0.065	0.056	0.018	0.039	**0.030**	**0.012**	**0.020**	0.218	0.031	0.089	0.158	0.025	0.083	0.124	0.042	0.074	0.076	0.026	0.047	0.130	0.041	0.069
200	15	**0.227**	**0.010**	**0.111**	0.586	0.199	0.383	0.734	0.137	0.430	0.797	0.426	0.580	0.836	0.251	0.478	0.732	0.265	0.462	0.620	0.182	0.411	0.489	**0.094**	0.293	0.518	0.108	0.310
200	20	1.766	0.294	0.956	2.452	1.154	1.857	4.636	2.307	3.253	5.623	2.503	4.070	2.252	0.860	1.535	2.904	1.567	2.235	2.676	1.438	2.039	1.704	0.475	1.081	**1.041**	**0.133**	**0.586**
240	5	**0.000**	**0.000**	**0.000**	0.003	**0.000**	0.001	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	0.045	0.002	0.018	0.002	**0.000**	0.000	0.002	**0.000**	0.001	0.005	**0.000**	0.001	0.025	**0.000**	0.007
240	10	0.067	0.019	0.036	0.118	0.037	0.073	0.072	**0.002**	0.042	**0.054**	0.013	**0.030**	0.173	0.023	0.094	0.109	0.047	0.078	0.114	0.044	0.075	0.086	0.024	0.047	0.118	0.037	0.074
240	15	**0.028**	0.015	**0.024**	0.186	0.082	0.119	0.231	**0.015**	0.103	0.286	0.028	0.136	0.296	0.054	0.170	0.248	0.039	0.125	0.194	0.051	0.113	0.120	0.026	0.065	0.293	0.039	0.114
240	20	2.020	0.533	1.156	3.175	1.858	2.545	4.982	2.868	3.865	5.726	3.495	4.598	2.243	0.914	1.574	3.542	2.483	3.082	3.475	2.103	2.802	1.678	0.504	1.076	**1.280**	**0.096**	**0.627**
Mean		1.142	0.348	0.730	1.161	0.417	0.773	1.696	0.692	1.169	2.572	0.840	1.665	1.329	0.470	0.868	1.330	0.507	0.899	1.255	0.504	0.877	1.248	0.477	0.844	**0.965**	**0.236**	**0.586**

Table 5
Average *RPD* values of *MAX*, *MIN*, *AVG* of nine algorithms grouped by 10 instances with the same scale size on Jose's benchmarks under $t = 0.006$.

μ	σ	*IGWS*			*IGT*			*CSA*			*TCANSA*			*IGRS*			*IGGR*			*IGALL*			*VBIH*			*MOAIG*		
		MAX	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*
10	5	**0.322**	0.238	**0.255**	0.787	0.761	0.766	0.527	**0.160**	0.362	1.756	0.281	0.958	0.787	0.761	0.771	0.787	0.761	0.766	0.761	0.761	0.761	0.813	0.761	0.776	0.576	0.322	0.451
10	10	**0.384**	0.371	0.373	0.425	0.425	0.425	0.662	0.148	**0.359**	1.416	**0.086**	0.615	0.476	0.425	0.436	0.476	0.425	0.446	0.425	0.425	0.425	0.476	0.425	0.436	0.425	0.425	0.425
10	15	0.479	0.214	**0.300**	0.874	0.551	0.670	0.538	0.229	0.411	1.212	**0.156**	0.683	1.062	0.551	0.741	1.051	0.551	0.755	0.574	0.551	0.555	0.898	0.551	0.639	**0.441**	0.408	0.428
10	20	**0.175**	**0.135**	**0.143**	0.476	0.442	0.469	0.852	0.177	0.352	1.407	0.201	0.682	0.616	0.442	0.518	0.769	0.442	0.564	0.442	0.442	0.442	0.482	0.442	0.450	0.298	**0.135**	0.168
15	5	1.214	0.465	0.829	0.916	0.656	0.771	**0.897**	0.542	**0.679**	1.637	**0.377**	0.929	1.297	0.615	0.839	1.268	0.682	0.882	0.990	0.706	0.825	1.025	0.638	0.826	1.121	0.578	0.801
15	10	**0.981**	0.352	0.662	0.998	0.547	0.797	1.014	0.332	0.643	1.821	0.411	1.059	1.239	0.735	0.916	1.119	0.547	0.836	1.063	0.634	0.845	1.153	0.647	0.890	0.994	**0.232**	**0.633**
15	15	0.867	0.381	0.583	0.942	0.488	0.686	0.905	0.460	0.681	2.081	0.705	1.300	0.903	0.505	0.696	1.008	0.495	0.678	0.959	0.479	0.688	0.936	0.512	0.726	**0.659**	**0.233**	**0.444**
15	20	1.204	0.370	0.740	1.272	0.845	1.043	1.623	0.634	1.015	2.265	0.513	1.389	1.612	0.853	1.180	1.415	0.832	1.066	1.300	0.842	1.077	1.565	0.956	1.170	**0.935**	**0.335**	**0.632**
20	5	1.189	0.556	0.856	1.131	0.302	0.694	1.659	0.934	1.285	3.234	0.768	1.911	1.462	0.318	0.860	1.237	0.514	0.878	1.227	0.315	0.814	1.269	0.559	0.938	**0.950**	**0.146**	**0.627**
20	10	1.534	0.363	0.904	**1.194**	0.407	0.798	1.458	0.721	1.045	3.197	0.848	1.857	1.431	0.412	0.938	1.423	0.360	0.882	1.342	0.538	0.914	1.481	0.488	0.961	1.257	**0.255**	**0.788**
20	15	1.245	**0.241**	**0.757**	1.193	0.417	0.771	1.461	0.470	0.935	2.948	0.522	1.585	**1.147**	0.457	0.842	1.208	0.378	0.771	1.193	0.562	0.859	1.416	0.620	0.983	1.294	0.391	0.811
20	20	1.245	0.293	0.723	1.209	0.418	0.806	1.689	0.543	1.110	2.979	0.871	1.876	**1.054**	0.383	0.717	1.327	0.415	0.867	1.157	0.360	0.793	1.461	0.467	0.929	1.093	**0.278**	**0.660**
25	5	0.978	0.520	0.716	**0.673**	**0.202**	**0.424**	1.197	0.416	0.883	1.767	0.706	1.253	0.865	0.244	0.574	0.849	0.217	0.527	0.823	0.325	0.510	0.704	0.286	0.500	0.691	0.315	0.494
25	10	1.288	0.439	0.945	1.346	0.405	0.847	1.322	0.410	0.931	2.754	0.745	1.605	1.211	0.558	0.855	1.325	0.554	0.910	1.427	0.680	1.036	1.588	0.764	1.213	**1.089**	**0.318**	**0.686**
25	15	1.824	0.568	1.283	1.614	0.420	1.032	1.963	0.844	1.464	3.610	1.055	2.380	1.716	0.725	1.231	1.709	0.615	1.127	1.685	0.711	1.198	2.132	0.857	1.434	**1.285**	**0.353**	**0.855**
25	20	1.626	0.638	1.072	1.535	0.639	1.100	1.876	0.719	1.268	3.008	1.126	2.090	1.564	0.605	1.063	1.566	0.640	1.105	1.615	**0.280**	0.979	1.848	0.698	1.310	**1.415**	0.373	**0.839**
30	5	1.179	0.528	0.914	1.092	**0.269**	0.665	1.569	0.957	1.314	3.064	0.910	1.887	1.123	0.425	0.775	1.184	0.340	0.747	1.097	0.477	0.751	1.110	0.455	0.809	**0.801**	0.369	**0.569**
30	10	1.383	0.572	0.990	1.319	0.332	0.913	1.893	0.716	1.282	3.400	1.149	2.160	1.608	0.433	1.011	1.646	0.424	0.984	1.509	0.566	1.038	1.487	0.530	1.030	**1.117**	**0.267**	**0.720**
30	15	1.757	0.688	1.173	1.610	0.541	1.063	2.229	0.713	1.543	4.101	1.250	2.582	1.878	0.513	1.186	1.586	0.439	0.988	1.715	0.691	1.217	1.953	0.802	1.349	**1.477**	**0.300**	**0.881**
30	20	2.145	0.703	1.453	**1.517**	0.648	1.066	2.765	0.729	1.641	3.347	1.217	2.201	1.770	0.440	1.130	1.632	0.739	1.190	1.802	0.781	1.312	1.928	0.848	1.365	1.521	**0.315**	**0.881**
35	5	0.911	0.389	0.637	0.587	0.151	0.354	1.042	0.522	0.784	1.929	0.391	1.148	0.778	0.230	0.490	0.751	0.125	0.452	0.749	0.219	0.485	0.758	0.301	0.536	**0.519**	**0.109**	**0.322**
35	10	1.117	0.490	0.770	**0.917**	0.246	0.582	1.260	0.571	0.959	2.853	0.627	1.364	1.274	0.327	0.744	1.146	0.238	0.667	1.172	0.417	0.793	1.201	0.340	0.807	0.988	**0.189**	**0.525**
35	15	1.850	0.699	1.287	1.470	0.379	0.891	1.886	0.809	1.341	3.456	1.213	2.381	1.792	0.721	1.161	1.555	0.519	1.012	1.607	0.671	1.195	1.784	0.718	1.307	**1.430**	**0.302**	**0.883**
35	20	2.049	0.789	1.354	**1.447**	0.380	0.925	2.307	0.826	1.550	3.799	0.897	2.274	1.539	0.634	1.080	1.732	0.614	1.140	1.729	0.638	1.207	1.864	0.784	1.326	1.605	**0.378**	**0.915**
40	5	0.807	0.499	0.659	0.711	0.204	0.449	1.157	0.641	0.906	1.812	0.525	1.178	0.799	**0.074**	0.443	0.865	0.168	0.463	0.872	0.314	0.566	0.830	0.309	0.539	**0.658**	0.102	**0.408**
40	10	1.178	0.466	0.841	1.056	0.353	0.675	1.541	0.622	1.087	3.302	1.050	2.179	1.309	0.380	0.846	1.268	0.285	0.712	1.258	0.529	0.871	1.170	0.378	0.845	**0.955**	**0.261**	**0.607**
40	15	1.918	0.383	1.150	1.473	0.392	0.939	2.121	0.829	1.527	4.077	1.282	2.680	1.868	0.672	1.174	1.650	0.667	1.124	1.661	0.543	1.087	1.960	0.887	1.411	**1.262**	**0.377**	**0.795**
40	20	1.885	0.671	1.241	1.614	0.424	1.041	3.176	1.082	1.996	4.072	1.353	2.647	1.789	0.403	1.117	**1.582**	0.500	1.022	1.780	0.674	1.201	1.962	0.875	1.418	1.650	**0.304**	**0.904**
80	5	0.010	**0.000**	0.003	0.031	**0.000**	0.010	0.021	**0.000**	0.010	**0.000**	**0.000**	**0.000**	0.079	**0.000**	0.029	0.031	**0.000**	0.010	0.020	**0.000**	0.006	0.015	**0.000**	0.003	0.060	**0.000**	0.028
80	10	1.316	0.637	0.970	**1.152**	0.250	0.721	1.413	0.317	0.870	4.099	1.952	2.967	1.526	0.625	1.007	1.509	0.422	0.959	1.384	0.718	1.083	1.621	0.692	1.126	1.293	**0.151**	**0.694**
80	15	1.384	0.515	0.939	1.422	0.286	0.831	2.054	0.522	1.179	3.835	1.458	2.646	1.554	0.625	1.067	1.542	0.465	1.012	1.511	0.430	0.982	1.804	0.729	1.207	**1.188**	**0.240**	**0.707**
80	20	2.510	0.750	1.516	1.689	0.414	**0.967**	3.360	1.267	2.135	5.165	1.972	3.589	**1.667**	0.769	1.209	1.944	**0.313**	1.134	1.896	0.577	1.266	1.988	0.968	1.466	1.714	0.347	1.007
120	5	0.003	**0.000**	0.002	0.003	**0.000**	0.001	0.006	**0.000**	0.001	0.003	**0.000**	0.001	0.010	**0.000**	0.004	0.010	**0.000**	0.002	0.013	**0.000**	0.003	0.013	**0.000**	0.004	**0.000**	**0.000**	**0.000**
120	10	**0.115**	0.033	**0.086**	0.216	**0.010**	0.110	0.260	0.110	0.173	0.523	0.250	0.360	0.381	0.076	0.191	0.298	0.133	0.190	0.352	0.117	0.216	0.290	0.106	0.169	0.218	0.036	0.097
120	15	1.307	0.367	0.841	**1.282**	**0.248**	**0.759**	3.100	0.989	1.929	4.899	1.950	3.467	1.475	0.603	1.007	1.581	0.566	1.028	1.401	0.690	1.030	1.874	0.790	1.258	1.490	0.301	0.790
120	20	2.281	0.589	1.334	**1.298**	0.309	0.821	5.481	2.084	3.595	5.261	2.954	4.082	1.975	0.800	1.371	2.158	0.640	1.321	1.775	0.612	1.179	1.994	0.805	1.422	1.482	**0.256**	**0.814**
160	5	**0.000**	**0.000**	**0.000**	0.010	0.002	0.005	0.005	**0.000**	0.002	**0.000**	**0.000**	**0.000**	0.040	**0.000**	0.012	0.010	0.002	0.005	0.005	**0.000**	0.003	0.007	0.002	0.005	0.027	**0.000**	0.009
160	10	**0.010**	**0.002**	**0.005**	0.058	0.005	0.024	0.032	0.005	0.015	0.018	**0.002**	0.012	0.140	0.005	0.048	0.033	0.005	0.016	0.101	0.005	0.047	0.057	0.005	0.024	0.064	0.005	0.024
160	15	1.156	0.232	**0.633**	1.204	0.244	0.762	3.567	1.264	2.306	5.092	2.770	3.902	1.922	0.576	1.208	1.550	0.681	1.120	1.660	0.546	1.053	1.814	0.688	1.204	**1.083**	**0.129**	0.634
160	20	1.787	0.203	0.955	1.386	0.377	0.857	4.668	2.418	3.447	5.758	2.792	4.280	2.167	0.579	1.384	1.554	0.393	1.072	1.618	0.586	1.061	1.956	0.516	1.215	**1.223**	**0.161**	**0.664**
200	5	0.012	0.002	0.006	0.020	**0.000**	0.011	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	0.043	**0.000**	0.017	0.016	0.012	0.012	0.021	0.012	0.014	0.016	**0.000**	0.010	0.039	0.012	0.021
200	10	0.036	**0.000**	**0.015**	0.066	0.023	0.038	0.030	0.012	0.021	**0.022**	0.010	**0.015**	0.150	0.020	0.062	0.080	0.021	0.045	0.073	0.006	0.046	0.062	0.023	0.039	0.064	0.023	0.041
200	15	**0.224**	**0.044**	**0.134**	0.457	0.096	0.231	0.611	0.222	0.403	0.841	0.321	0.592	0.636	0.167	0.392	0.453	0.192	0.313	0.411	0.134	0.270	0.457	0.124	0.294	0.397	0.091	0.235
200	20	1.781	0.278	0.902	1.336	0.260	0.833	4.632	2.102	3.311	5.413	2.724	4.055	1.655	0.575	1.102	1.672	0.630	1.166	1.553	0.495	1.069	1.694	0.498	1.027	**0.781**	**0.026**	**0.402**
240	5	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	0.020	**0.000**	0.011	0.002	**0.000**	**0.000**	0.002	**0.000**	0.001	**0.000**	**0.000**	**0.000**	0.005	**0.000**	0.001
240	10	**0.040**	0.017	0.026	0.087	0.026	0.050	0.063	**0.000**	0.035	0.043	0.002	**0.018**	0.121	0.020	0.061	0.090	0.027	0.055	0.080	0.019	0.043	0.068	0.019	0.036	0.081	0.025	0.047
240	15	**0.028**	0.013	**0.021**	0.096	0.024	0.056	0.166	0.008	0.074	0.172	**0.002**	0.074	0.203	0.039	0.102	0.062	0.013	0.036	0.086	0.024	0.047	0.088	0.021	0.049	0.199	0.020	0.079
240	20	1.980	0.520	1.121	1.535	0.456	0.989	5.040	2.831	4.011	6.195	3.316	4.680	1.955	0.723	1.276	2.148	0.957	1.581	1.877	0.779	1.271	1.658	0.512	1.084	**1.085**	**0.061**	**0.546**
Mean		1.057	0.359	0.69	0.932	0.318	0.62	1.606	0.644	1.101	2.576	0.911	1.7	1.118	0.418	0.748	1.081	0.395	0.722	1.037	0.435	0.732	1.14	0.487	0.803	**0.854**	**0.214**	**0.521**

Table 6
Average *RPD* values of *MAX*, *MIN*, *AVG* of nine algorithms grouped by 10 instances with the same scale size on Jose's benchmarks under $t = 0.01$.

μ	σ	*IGWS*			*IGT*			*CSA*			*TCANSA*			*IGRS*			*IGGR*			*IGALL*			*VBIH*			*MOAIG*		
		MAX	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*
10	5	**0.369**	0.285	**0.302**	0.807	0.807	0.807	0.426	0.173	0.302	1.526	**0.146**	0.862	0.807	0.807	0.807	0.807	0.807	0.807	0.807	0.807	0.807	0.833	0.807	0.812	0.499	0.369	0.473
10	10	**0.422**	0.411	0.420	0.476	0.476	0.476	0.681	0.212	**0.398**	1.541	**0.137**	0.695	0.476	0.476	0.476	0.476	0.476	0.476	0.476	0.476	0.476	0.476	0.476	0.476	0.476	0.476	0.476
10	15	**0.396**	0.259	**0.298**	0.788	0.596	0.689	0.509	0.286	0.376	1.841	**0.233**	0.840	0.907	0.596	0.712	0.924	0.596	0.753	0.596	0.596	0.596	0.596	0.596	0.596	0.487	0.431	0.462
10	20	**0.182**	0.182	**0.182**	0.523	0.490	0.516	0.679	0.196	0.383	1.321	**0.114**	0.600	0.663	0.490	0.559	0.630	0.490	0.574	0.490	0.490	0.490	0.497	0.490	0.491	0.243	0.182	0.195
15	5	1.136	0.647	0.888	0.992	0.795	0.905	0.990	0.662	**0.808**	2.421	**0.483**	1.341	1.190	0.797	0.942	1.188	0.844	0.968	1.068	0.845	0.938	1.080	0.820	0.940	**0.958**	0.711	0.826
15	10	0.977	0.522	0.782	1.100	0.668	0.914	0.932	0.530	**0.689**	1.724	0.440	1.008	1.281	0.810	0.982	1.194	0.690	0.903	1.035	0.765	0.910	1.254	0.765	0.998	**0.923**	**0.403**	0.699
15	15	0.790	0.422	0.578	0.867	0.523	0.641	0.993	0.395	0.667	2.265	0.571	1.318	0.900	0.542	0.677	0.941	0.563	0.689	0.853	0.540	0.679	0.955	0.545	0.743	**0.700**	**0.300**	**0.488**
15	20	1.097	**0.291**	0.632	1.176	0.853	0.980	1.316	0.419	0.760	2.550	0.482	1.338	1.272	0.778	1.041	1.346	0.840	1.025	1.172	0.812	0.964	1.304	0.867	1.049	**0.843**	0.304	**0.570**
20	5	1.142	0.553	0.848	0.945	0.337	0.585	1.620	0.812	1.165	2.941	0.653	1.672	1.288	0.358	0.830	1.120	0.421	0.783	1.104	0.259	0.684	1.159	0.540	0.861	**0.925**	**0.203**	**0.579**
20	10	1.360	0.492	0.924	**1.066**	0.521	**0.799**	1.496	0.723	1.062	3.020	1.166	2.073	1.514	**0.414**	0.954	1.238	0.491	0.883	1.159	0.506	0.881	1.230	0.603	0.930	1.275	0.416	0.810
20	15	**1.064**	**0.312**	**0.720**	1.188	0.461	0.782	1.323	0.532	0.867	2.653	0.837	1.745	1.130	0.480	0.841	1.109	0.455	0.735	1.205	0.524	0.838	1.190	0.616	0.928	1.139	0.416	0.770
20	20	1.231	0.391	0.831	1.319	0.559	0.929	1.673	0.786	1.220	3.290	0.853	1.947	**1.150**	0.512	0.845	1.388	0.567	0.951	1.243	0.418	0.851	1.466	0.636	1.053	1.160	**0.342**	**0.751**
25	5	0.907	0.413	0.638	**0.545**	**0.158**	**0.354**	0.950	0.491	0.694	2.164	0.607	1.290	0.689	0.214	0.456	0.717	0.204	0.461	0.651	0.282	0.442	0.634	0.244	0.451	0.663	0.273	0.447
25	10	1.201	0.390	0.878	1.082	0.380	0.739	1.306	0.501	0.941	2.617	0.380	1.350	1.114	0.478	0.762	1.127	0.368	0.730	1.283	0.570	0.938	1.325	0.672	1.026	**0.896**	**0.368**	**0.644**
25	15	1.642	0.539	1.111	1.468	0.371	0.908	2.028	0.726	1.402	3.559	0.936	2.293	1.664	0.697	1.145	1.662	0.497	1.059	1.649	0.733	1.203	1.883	0.771	1.304	**1.322**	**0.298**	**0.870**
25	20	1.594	0.556	1.045	1.505	0.578	1.026	1.945	0.845	1.403	3.472	0.968	2.191	1.365	0.495	0.902	1.373	0.502	0.971	1.600	**0.387**	0.945	1.835	0.688	1.299	**1.226**	0.479	**0.801**
30	5	1.113	0.465	0.822	0.793	**0.136**	**0.463**	1.484	0.645	1.053	2.416	0.901	1.674	0.925	0.274	0.614	0.972	0.357	0.642	1.009	0.381	0.691	0.968	0.384	0.668	**0.786**	0.234	0.518
30	10	1.210	0.545	0.880	1.193	0.186	0.686	1.445	0.473	0.997	3.492	0.647	1.953	1.255	0.300	0.756	1.191	0.385	0.798	1.350	0.430	0.877	1.378	0.356	0.871	**0.954**	**0.164**	**0.584**
30	15	1.432	0.500	0.967	1.415	0.456	0.850	2.013	0.754	1.322	3.153	0.912	2.000	1.501	0.436	0.987	1.354	**0.229**	0.798	1.475	0.583	1.064	1.550	0.745	1.163	**1.218**	0.285	**0.725**
30	20	2.007	0.659	1.327	1.380	0.647	0.988	2.530	0.709	1.555	3.267	1.136	2.202	1.661	0.452	1.017	1.648	0.671	1.135	1.725	0.815	1.250	1.783	0.743	1.233	**1.307**	**0.291**	**0.776**
35	5	0.873	0.448	0.678	0.577	**0.097**	**0.332**	0.921	0.448	0.707	2.319	0.530	1.298	0.676	0.229	0.417	0.663	0.219	0.413	0.770	0.307	0.538	0.754	0.278	0.505	**0.526**	0.201	0.368
35	10	1.021	0.452	0.749	0.878	0.348	0.584	1.071	0.562	0.844	2.347	0.598	1.409	1.097	0.378	0.698	1.088	0.343	0.685	1.015	0.372	0.704	1.058	0.358	0.719	**0.776**	**0.223**	**0.444**
35	15	1.972	0.783	1.327	1.567	**0.517**	1.002	2.320	0.898	1.601	3.779	1.392	2.432	1.805	0.696	1.178	1.501	0.525	**0.988**	1.734	0.757	1.321	1.834	0.837	1.356	**1.449**	0.533	1.009
35	20	1.780	0.925	1.326	**1.468**	**0.420**	**0.921**	2.273	0.790	1.510	4.086	0.722	2.293	1.640	0.638	1.148	1.594	0.664	1.095	1.556	0.660	1.160	1.965	0.859	1.393	1.536	0.575	0.996
40	5	0.759	0.498	0.635	**0.531**	0.148	0.338	0.866	0.354	0.678	1.821	0.678	1.172	0.582	**0.033**	0.332	0.597	0.122	0.333	0.693	0.294	0.492	0.645	0.166	0.404	0.548	0.101	**0.309**
40	10	1.070	0.434	0.767	0.837	**0.167**	0.512	1.199	0.410	0.798	3.067	1.165	2.061	0.999	0.192	0.646	0.962	0.301	0.602	1.055	0.405	0.733	1.089	0.382	0.755	**0.709**	0.248	**0.462**
40	15	1.739	0.368	1.057	1.419	0.443	0.870	2.270	0.920	1.582	3.623	0.828	2.122	1.617	0.529	1.101	1.470	0.538	1.028	1.658	0.507	1.118	1.879	0.782	1.298	**1.188**	**0.312**	**0.802**
40	20	1.941	0.748	1.246	1.633	0.597	1.091	3.444	1.047	2.057	4.297	1.268	2.680	1.756	0.426	1.145	1.691	0.592	1.104	1.812	0.704	1.213	1.916	0.728	1.377	**1.307**	**0.315**	**0.820**
80	5	0.005	**0.000**	0.001	0.026	**0.000**	0.006	0.005	**0.000**	0.002	**0.005**	**0.000**	**0.001**	0.035	**0.000**	0.011	0.020	**0.000**	0.008	0.015	**0.000**	0.003	0.010	**0.000**	0.002	0.044	**0.000**	0.011
80	10	1.479	0.870	1.197	**1.191**	0.418	**0.805**	1.639	0.432	1.013	4.634	2.468	3.494	1.329	0.608	0.978	1.452	0.555	0.997	1.596	0.783	1.210	1.741	0.796	1.221	1.285	**0.371**	0.827
80	15	1.405	0.649	1.021	1.442	0.388	0.880	1.646	**0.333**	0.955	4.222	1.963	2.961	1.487	0.638	1.068	1.444	0.534	0.992	1.437	0.608	0.993	1.765	0.703	1.252	**1.056**	0.435	**0.757**
80	20	2.015	0.797	1.431	**1.495**	**0.383**	**0.909**	3.449	0.920	2.052	6.101	2.637	4.079	1.898	0.579	1.156	1.799	0.402	1.132	1.691	0.712	1.128	1.760	0.998	1.411	1.653	0.499	1.037
120	5	0.003	**0.000**	0.002	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	0.010	**0.000**	0.003	0.010	**0.000**	0.002	0.003	**0.000**	0.001	0.003	**0.000**	0.001	**0.000**	**0.000**	**0.000**
120	10	**0.109**	0.039	0.077	0.200	**0.017**	0.086	0.347	0.150	0.227	0.425	0.200	0.295	0.217	0.056	0.132	0.186	0.073	0.127	0.199	0.107	0.147	0.254	0.062	0.125	0.191	**0.017**	**0.074**
120	15	1.317	0.454	0.868	**1.084**	**0.187**	**0.638**	3.365	1.041	2.069	5.754	2.831	4.178	1.447	0.470	0.958	1.442	0.447	0.942	1.378	0.595	0.984	1.652	0.815	1.210	1.306	0.270	0.767
120	20	1.986	0.512	1.209	**1.220**	**0.199**	**0.709**	4.479	1.737	3.093	5.745	2.781	4.039	1.798	0.615	1.172	1.848	0.398	1.007	1.659	0.442	1.022	1.767	0.856	1.270	1.395	0.287	0.790
160	5	**0.000**	**0.000**	**0.000**	0.005	**0.000**	0.002	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	0.017	**0.000**	0.006	0.010	0.002	0.004	0.005	**0.000**	0.003	0.007	0.002	0.005	0.015	**0.000**	0.006
160	10	**0.005**	0.002	0.004	0.046	0.005	0.018	0.033	0.002	0.012	**0.005**	**0.000**	**0.003**	0.106	0.002	0.032	0.024	0.005	0.010	0.093	0.002	0.037	0.041	0.005	0.015	0.031	0.005	0.013
160	15	1.174	0.348	0.719	1.143	0.166	0.614	3.434	1.387	2.392	4.894	2.440	3.676	1.721	0.561	1.087	1.359	0.408	0.876	1.248	0.541	0.873	1.615	0.592	1.115	**0.943**	**0.091**	**0.544**
160	20	1.753	0.312	1.007	1.290	0.318	0.797	4.538	2.283	3.549	5.542	3.025	4.226	1.792	0.489	1.221	1.655	0.354	1.001	1.256	0.487	0.862	1.723	0.560	1.162	**0.962**	**0.097**	**0.554**
200	5	0.010	0.002	0.005	0.018	**0.000**	0.010	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	0.031	**0.000**	0.011	0.014	0.012	0.012	0.014	0.012	0.012	0.014	**0.000**	0.010	0.026	0.012	0.014
200	10	**0.020**	**0.000**	**0.008**	0.040	0.023	0.030	0.046	0.004	0.020	0.022	0.008	0.013	0.097	0.020	0.043	0.062	0.002	0.037	0.060	0.006	0.041	0.050	0.020	0.032	0.062	0.020	0.037
200	15	**0.190**	**0.019**	**0.118**	0.370	0.090	0.191	0.595	0.228	0.356	0.690	0.334	0.500	0.552	0.164	0.320	0.384	0.065	0.233	0.365	0.138	0.230	0.404	0.148	0.278	0.342	0.078	0.205
200	20	1.817	0.318	0.951	1.206	0.230	0.659	4.635	2.525	3.500	5.452	2.806	4.031	1.558	0.638	1.043	1.501	0.498	0.960	1.404	0.324	0.830	1.624	0.677	1.070	**0.753**	**0.172**	**0.446**
240	5	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	0.015	**0.000**	0.005	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
240	10	0.035	0.017	0.023	0.087	0.014	0.034	0.040	0.002	0.016	**0.030**	**0.000**	**0.008**	0.119	0.020	0.058	0.070	0.016	0.035	0.065	0.015	0.031	0.055	0.019	0.032	0.059	0.020	0.035
240	15	**0.029**	0.015	**0.021**	0.067	0.010	0.042	0.103	0.011	0.049	0.101	**0.006**	0.041	0.132	0.041	0.073	0.058	0.010	0.030	0.054	0.026	0.039	0.077	0.018	0.040	0.135	0.013	0.052
240	20	1.968	0.544	1.135	1.371	0.367	0.858	5.396	3.138	4.290	6.025	3.643	4.738	1.775	0.599	1.175	1.773	0.829	1.250	1.479	0.584	1.035	1.690	0.651	1.150	**1.053**	**0.119**	**0.576**
Mean		0.995	0.383	0.680	0.872	0.324	0.583	1.551	0.635	1.072	2.629	0.936	1.711	1.022	0.396	0.698	0.981	0.383	0.668	0.964	0.429	0.693	1.058	0.493	0.773	**0.778**	**0.249**	**0.509**

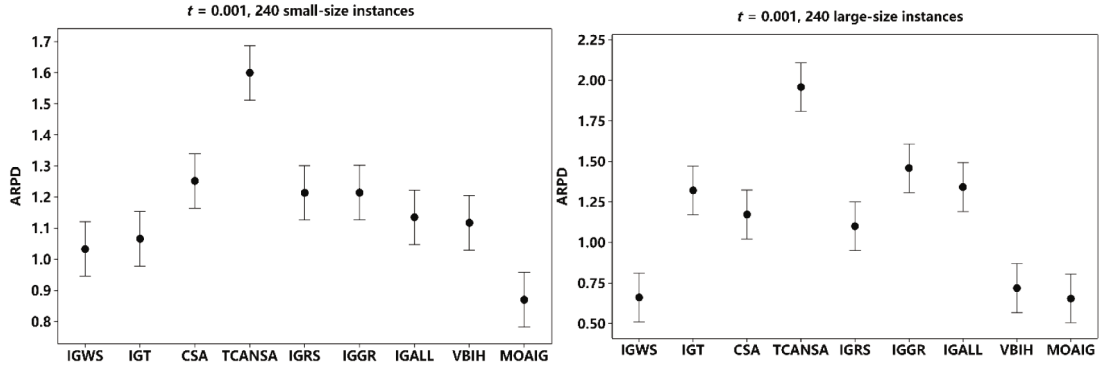
Table 7
Average *RPD* values of *MAX*, *MIN*, *AVG* obtained by nine algorithms on Liao's benchmarks under $t = 0.01$.

instances	*IGWS*			*IGT*			*CSA*			*TCANSA*			*IGRS*			*IGGR*			*IGALL*			*VBIH*			*MOAIG*		
	MAX	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*
j10c5a2	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c5a3	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c5a4	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c5a5	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c5a6	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c5b1	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c5b2	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c5b3	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c5b4	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c5b5	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c5b6	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c5b7	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c5c1	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	1.471	**0.000**	0.588	1.471	**0.000**	1.176	**0.000**	**0.000**	**0.000**	1.471	**0.000**	0.294	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c5c2	**0.000**	**0.000**	**0.000**	1.351	**0.000**	0.541	1.351	**0.000**	0.541	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	1.351	**0.000**	0.270	1.351	**0.000**	0.541	1.351	**0.000**	0.811	**0.000**	**0.000**	**0.000**
j10c5c4	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	1.515	**0.000**	0.909	1.515	**0.000**	0.606	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c5c5	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c5c6	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	1.449	**0.000**	0.290	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c5d1	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c5d2	**1.370**	1.370	1.370	**1.370**	1.370	1.370	**1.370**	**0.000**	0.548	**1.370**	**0.000**	**0.274**	**1.370**	1.370	1.370	**1.370**	1.370	1.370	**1.370**	1.370	1.370	**1.370**	1.370	1.370	**1.370**	1.370	1.370
j10c5d3	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c5d4	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	1.429	**0.000**	0.286	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c5d5	1.515	**0.000**	1.212	1.515	**0.000**	0.606	3.030	**0.000**	1.515	3.030	1.515	2.727	1.515	**0.000**	0.606	1.515	**0.000**	0.303	**0.000**	**0.000**	**0.000**	1.515	**0.000**	1.212	**0.000**	**0.000**	**0.000**
j10c5d6	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c5d7	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c10a1	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c10a2	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c10a3	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c10a4	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c10a5	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c10a6	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c10b1	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c10b2	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c10b3	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c10b4	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c10b5	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c10b6	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c10c1	**0.877**	0.877	0.877	**0.877**	0.877	0.877	**0.877**	0.877	0.877	**0.877**	**0.000**	**0.351**	**0.877**	0.877	0.877	**0.877**	0.877	0.877	**0.877**	0.877	0.877	**0.877**	0.877	0.877	**0.877**	0.877	0.877
j10c10c2	1.724	1.724	1.724	2.586	2.586	2.586	1.724	**0.000**	**0.690**	**0.862**	0.862	0.862	2.586	2.586	2.586	2.586	2.586	2.586	2.586	2.586	2.586	2.586	2.586	2.586	1.724	1.724	1.724
j10c10c3	**0.862**	**0.000**	**0.172**	**0.862**	**0.000**	0.345	**0.862**	**0.000**	0.517	**0.862**	**0.000**	**0.172**	**0.862**	**0.000**	**0.172**	**0.862**	**0.000**	0.517	**0.862**	**0.000**	**0.172**	**0.862**	**0.000**	0.690	**0.862**	**0.000**	**0.172**
j10c10c4	**0.840**	0.840	**0.840**	**0.840**	0.840	**0.840**	**0.840**	0.840	**0.840**	2.521	**0.000**	1.008	**0.840**	0.840	**0.840**	**0.840**	0.840	**0.840**	**0.840**	0.840	**0.840**	**0.840**	0.840	**0.840**	0.840	**0.840**	**0.840**
j10c10c5	**0.000**	**0.000**	**0.000**	0.800	0.800	0.800	1.600	0.800	0.960	**0.000**	**0.000**	**0.000**	0.800	0.800	0.800	0.800	0.800	0.800	0.800	0.800	0.800	0.800	0.800	0.800	**0.000**	**0.000**	**0.000**
j10c10c6	1.923	1.923	1.923	1.923	1.923	1.923	1.923	0.962	1.154	**0.962**	**0.000**	**0.769**	1.923	1.923	1.923	1.923	1.923	1.923	1.923	1.923	1.923	1.923	1.923	1.923	1.923	1.923	1.923
j10c10d1	0.901	0.901	0.901	0.901	0.901	0.901	0.901	**0.000**	0.180	**0.000**	**0.000**	**0.000**	0.901	0.901	0.901	0.901	0.901	0.901	0.901	0.901	0.901	0.901	0.901	0.901	0.901	0.901	0.901
j10c10d2	**0.847**	0.847	0.847	**0.847**	0.847	0.847	**0.847**	**0.000**	**0.169**	**0.847**	**0.000**	**0.169**	**0.847**	0.847	0.847	**0.847**	0.847	0.847	**0.847**	0.847	0.847	**0.847**	0.847	0.847	**0.847**	0.847	0.847
j10c10d3	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j10c10d4	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**	**0.877**
j10c10d5	0.935	**0.000**	0.374	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	0.935	**0.000**	0.374	**0.000**	**0.000**	**0.000**
j10c10d6	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	0.952	**0.000**	0.381	0.952	**0.000**	0.190	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5a1	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5a2	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5a3	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5a4	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**

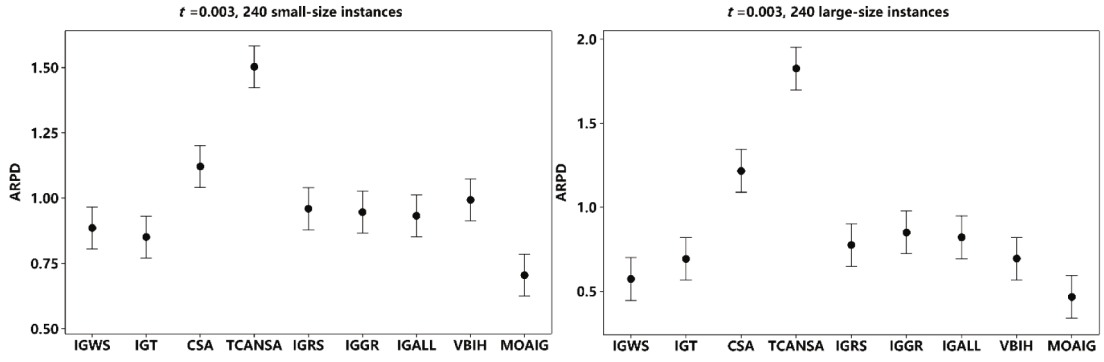
(*continued on next page*)

Table 7 (*continued*)

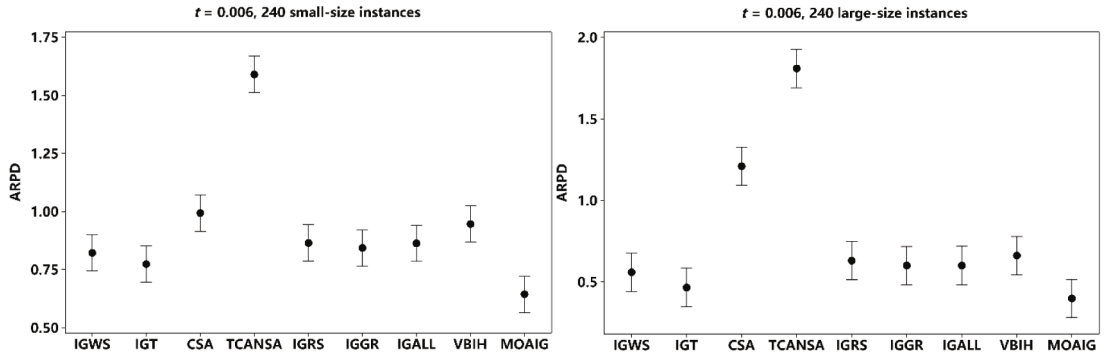
instances	*IGWS*			*IGT*			*CSA*			*TCANSA*			*IGRS*			*IGGR*			*IGALL*			*VBIH*			*MOAIG*		
	MAX	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*	*MAX*	*MIN*	*AVG*
j15c5a5	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5a6	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5b1	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5b2	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5b3	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5b4	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5b5	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5b6	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5c1	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	2.353	1.176	2.118	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5c2	**1.111**	1.111	1.111	**1.111**	1.111	1.111	**1.111**	1.111	1.111	**1.111**	**0.000**	**0.222**	**1.111**	1.111	1.111	**1.111**	**0.000**	0.889	**1.111**	1.111	1.111	**1.111**	**0.000**	0.889	**1.111**	**0.000**	0.889
j15c5c3	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	1.149	**0.000**	0.920	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5c4	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	1.124	1.124	1.124	1.124	**0.000**	0.225	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	1.124	**0.000**	0.225	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5c5	1.351	**0.000**	0.811	**0.000**	**0.000**	**0.000**	4.054	2.703	3.784	1.351	**0.000**	0.541	1.351	**0.000**	0.270	1.351	**0.000**	0.541	1.351	**0.000**	0.541	1.351	**0.000**	0.270	**0.000**	**0.000**	**0.000**
j15c5c6	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5d1	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5d2	**0.000**	**0.000**	**0.000**	1.190	**0.000**	0.238	2.381	1.190	2.143	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	1.190	**0.000**	0.238	1.190	**0.000**	0.238	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5d3	1.220	**0.000**	0.732	1.220	**0.000**	0.732	2.439	2.439	2.439	1.220	**0.000**	0.244	1.220	1.220	1.220	1.220	**0.000**	0.732	1.220	**0.000**	0.732	1.220	**0.000**	0.732	**0.000**	**0.000**	**0.000**
j15c5d4	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	2.381	1.190	2.143	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5d5	**1.266**	**0.000**	1.013	**1.266**	**0.000**	0.759	3.797	1.266	2.532	**1.266**	**0.000**	0.506	**1.266**	**0.000**	0.759	**1.266**	**0.000**	1.013	**1.266**	**0.000**	0.759	**1.266**	**0.000**	0.506	**1.266**	**0.000**	**0.253**
j15c5d6	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	2.469	1.235	2.222	**0.000**	**0.000**	**0.000**	1.235	**0.000**	0.247	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5e1	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5e2	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5e3	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5e4	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5e5	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c5e6	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c10a1	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c10a2	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c10a3	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c10a4	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c10a5	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c10a6	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c10b1	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c10b2	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c10b3	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c10b4	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c10b5	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c10b6	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
j15c10c1	**0.746**	**0.000**	0.299	**0.746**	**0.000**	0.448	1.493	0.746	1.343	**0.746**	**0.000**	0.597	**0.746**	**0.000**	0.299	**0.746**	**0.000**	**0.149**	**0.746**	**0.000**	0.448	**0.746**	**0.000**	0.448	**0.746**	**0.000**	0.448
j15c10c2	**1.527**	0.763	**0.916**	**1.527**	0.763	**0.916**	3.053	3.053	3.053	3.053	0.763	1.679	**1.527**	0.763	1.069	**1.527**	0.763	1.374	**1.527**	0.763	**0.916**	**1.527**	0.763	1.221	**1.527**	**0.000**	1.069
j15c10c3	**0.690**	**0.690**	**0.690**	**0.690**	**0.690**	**0.690**	2.069	1.379	1.517	**0.690**	**0.690**	**0.690**	**0.690**	**0.690**	**0.690**	**0.690**	**0.690**	**0.690**	**0.690**	**0.690**	**0.690**	**0.690**	**0.690**	**0.690**	**0.690**	**0.690**	**0.690**
j15c10c4	**0.781**	0.781	0.781	**0.781**	0.781	0.781	3.125	2.344	2.500	1.563	**0.000**	**0.625**	**0.781**	0.781	0.781	**0.781**	0.781	0.781	**0.781**	0.781	0.781	**0.781**	0.781	0.781	**0.781**	0.781	0.781
j15c10c5	**0.709**	**0.000**	0.426	**0.709**	**0.000**	0.284	2.128	0.709	1.560	**0.709**	**0.000**	0.426	**0.709**	**0.000**	0.284	**0.709**	**0.000**	0.284	**0.709**	**0.000**	**0.142**	**0.709**	**0.000**	**0.142**	**0.709**	**0.000**	0.284
j15c10c6	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	1.515	**0.000**	0.758	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**	**0.000**
Mean	0.230	0.132	0.186	0.250	0.150	0.192	0.607	0.271	0.441	0.317	**0.049**	0.159	0.250	0.162	0.193	0.279	0.138	0.199	0.260	0.150	0.191	0.261	0.138	0.206	**0.178**	0.113	**0.145**



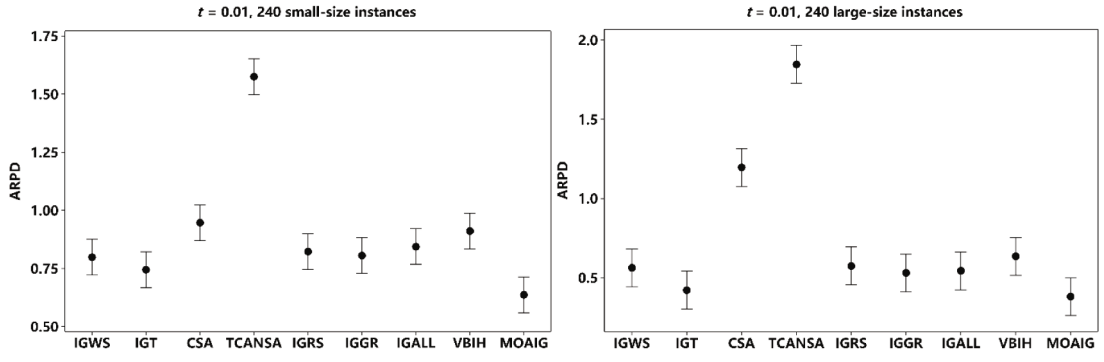
(a) Means plots of all algorithms at $t = 0.001$



(b) Means plots of all algorithms at $t = 0.003$



(c) Means plots of all algorithms at $t = 0.006$



(d) Means plots of all algorithms at $t = 0.01$

Fig. 10. Means plots of all algorithms at different time for Jose's instances including 240 small-size and 240 large-size instances.

destruction reconstruction strategies. However, the main difference between them is the local search strategy. Our local search is based on the idea of effective insertion of mandatory operations. This strategy can not only expand the search space, but also reduce the number of unnecessary insertions and operations, so it improves the efficiency of the algorithm and reduces the time complexity of the algorithm.

The superiority of the proposed *MOAIG* method can be further demonstrated in Tables 5 and 6 which reports the experimental results of

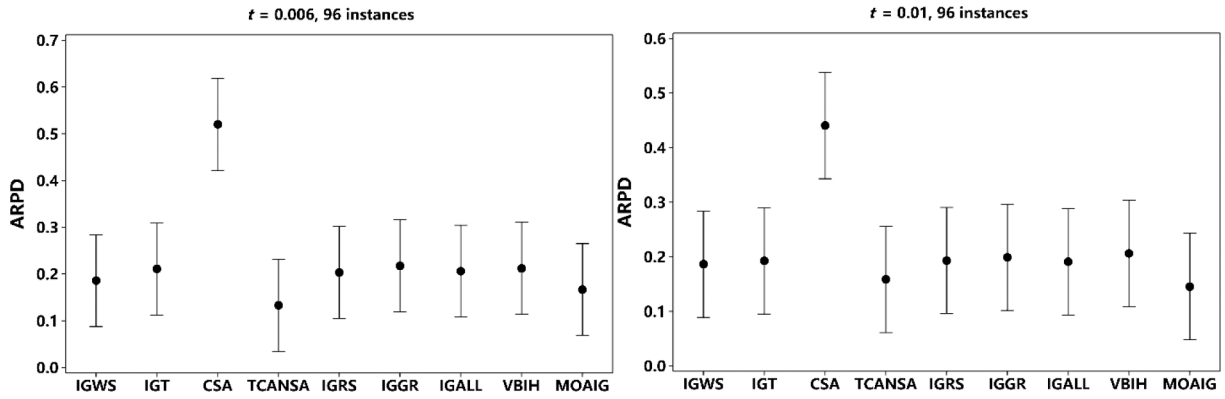


Fig. 11. Means plots of all algorithms at different time for Liao's instances including 96 instances.

all comparison algorithms at $0.006 \times \mu^3 \times \sigma$, $0.01 \times \mu^3 \times \sigma$ milliseconds, respectively, on Jose's benchmark test dataset. In Table 5, our *MOAIG* method can obtain the more minima of *MAX* (21/48), *MIN* (30/48), and *AVG* (28/48) values. It is the best among *IGWS* (10/48, 9/48, 8/48), *IGT* (9/48, 8/48, 4/48), *CSA* (3/48, 7/48, 4/48), *TCANSA* (4/48, 10/48, 6/48), *IGRS* (3/48, 6/48, 0), *IGGR* (1/48, 4/48, 1/48), *IGALL* (0, 5/48, 0), and *VBIH* (1/48, 4/48, 1/48). From the average value of *MAX*, *MIN*, and *AVG* values on all the test dataset in Jose's benchmark, our *MOAIG* method (0.854, 0.214, 0.521) is smaller than *IGWS* (1.057, 0.359, 0.69), *IGT* (0.932, 0.318, 0.62), *CSA* (1.606, 0.644, 1.101), *TCANSA* (2.576, 0.911, 1.7), *IGRS* (1.118, 0.418, 0.748), *IGGR* (1.081, 0.395, 0.722), *IGALL* (1.037, 0.435, 0.732), and *VBIH* (1.14, 0.487, 0.803). Similarly, In Table 6, from the average value of *MAX*, *MIN*, and *AVG* values on all the Jose's test dataset, our *MOAIG* algorithm (0.778, 0.249, 0.509) is smaller than *IGWS* (0.995, 0.383, 0.680), *IGT* (0.872, 0.324, 0.583), *CSA* (1.551, 0.635, 1.072), *TCANSA* (2.629, 0.936, 1.711), *IGRS* (1.022, 0.396, 0.698), *IGGR* (0.981, 0.383, 0.668), *IGALL* (0.964, 0.429, 0.693), and *VBIH* (1.058, 0.493, 0.773). Thus, the performance of accelerated evaluation based on mandatory operations insertion can be further demonstrated in Table 6.

In addition, Table 7 reports the experimental results of all the comparison algorithms under the same termination time, $0.01 \times \mu^3 \times \sigma$ milliseconds, on Liao's benchmark test dataset (including 96 instances). The experimental results show that all algorithms can obtain good results on most of the instances (about 60/96), and their *MAX*, *MIN*, and *AVG* values are 0. This is because the instances are small-scale problems, and all algorithms can find better solutions in an acceptable time. However, there are still a few instances where the performance of each algorithm is greatly different, among which our *MOAIG* algorithm has the best overall performance. From the average value of *MAX*, *MIN*, and *AVG* values on all the Liao's test dataset, our *MOAIG* algorithm (0.178, 0.113, 0.1459) is smaller than *IGWS* (0.23, 0.132, 0.186), *IGT* (0.25, 0.15, 0.192), *CSA* (0.607, 0.271, 0.441), *TCANSA* (0.317, 0.049, 0.159), IGRS (0.25, 0.162, 0.193), *IGGR* (0.279, 0.138, 0.199), *IGALL* (0.26, 0.15, 0.1913), and *VBIH* (0.261, 0.138, 0.206). Thus, the superiority of the proposed *MOAIG* algorithm can be further demonstrated in Table 7.

To further analyze the performance of the proposed *MOAIG* algorithm on the *ARPD* indicator, we have also performed the statistical analysis of the eight confidence interval figures. Means and interactions plots with 95 % LSD intervals are done to represent the average level and overall performance of the algorithms. Figs. 10 and 11 illustrate the interactions between nine algorithms to solve Jose's and Liao's instances, respectively. From the values of confidence interval center, the results obtained by the *MOAIG* algorithm with respect to *ARPD* indicator are significantly lower than these of eight comparative algorithms. Thus, the performance of the proposed algorithm is better than those of *IGWS*, *IGT*, *CSA*, *TCANSA*, *IGRS*, *IGGR*, *IGALL*, and *VBIH*. These results demonstrate that our algorithm exhibits a pronounced superiority over the comparison methods across all test instances, demonstrating its effectiveness in solving this problem. Moreover, the convergence curves have been shown in Fig. 12. We select 240 small-size and 240 large-size instances within a time limit of $0.01 \times \mu^3 \times \sigma$ milliseconds. From Fig. 12, it can be observed that the *MOAIG* algorithm outperforms other algorithms both in terms of convergence speed and convergence level.

To further validate the performance of the proposed mandatory operations-based effective insertion local search, we give an example of 30 jobs and 5 stages, and provide, in the form of Gantt chart, the changes in the mandatory operations and the reduction in makespan values after implementing this strategy. In the Gantt chart, different colors represent different operations, and "$m_{s,k}$" denotes the completion time of the operation on machine "k" in stage "s". The blue dashed ellipses indicate the selected mandatory operation and its processing machine, while the blue dashed rectangles represent operations whose completion times are reduced after applying the proposed local search. Fig. 13(a) is the best solution found by *IGGR*, *IGRS*, *IGALL*, *IGT*, *VBIH*, *CSA*, *IGWS*, and *TSCNSA* (makespan = 821). Fig. 13(b) is the best solution found by

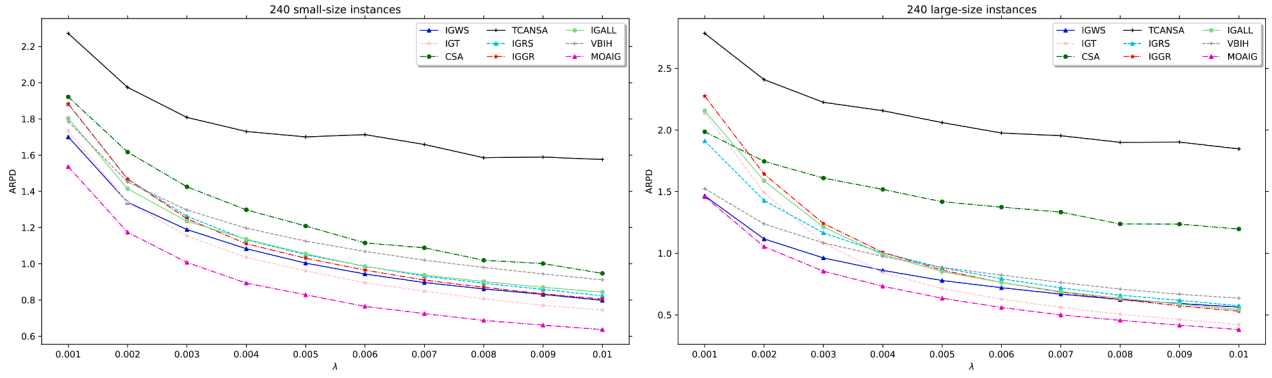
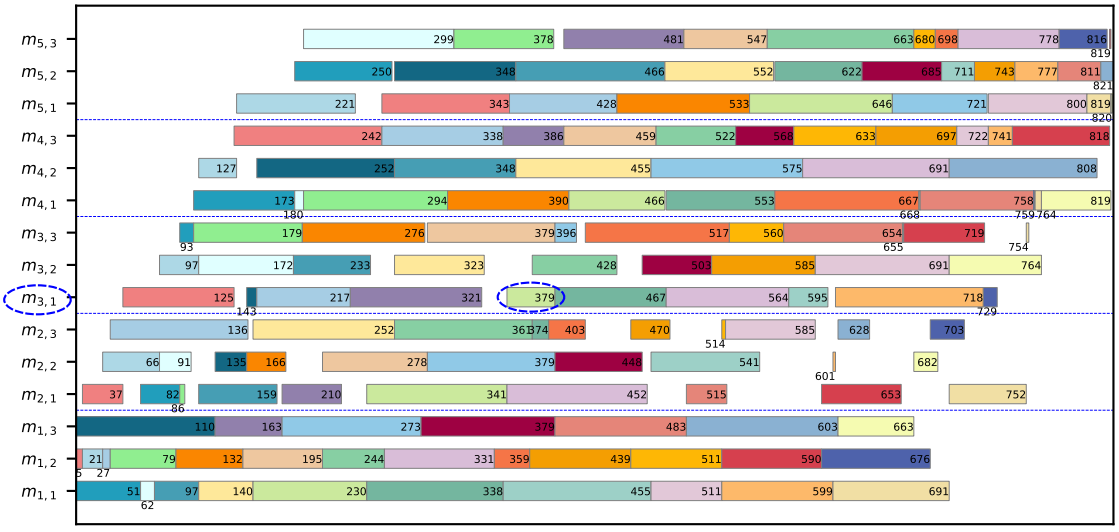
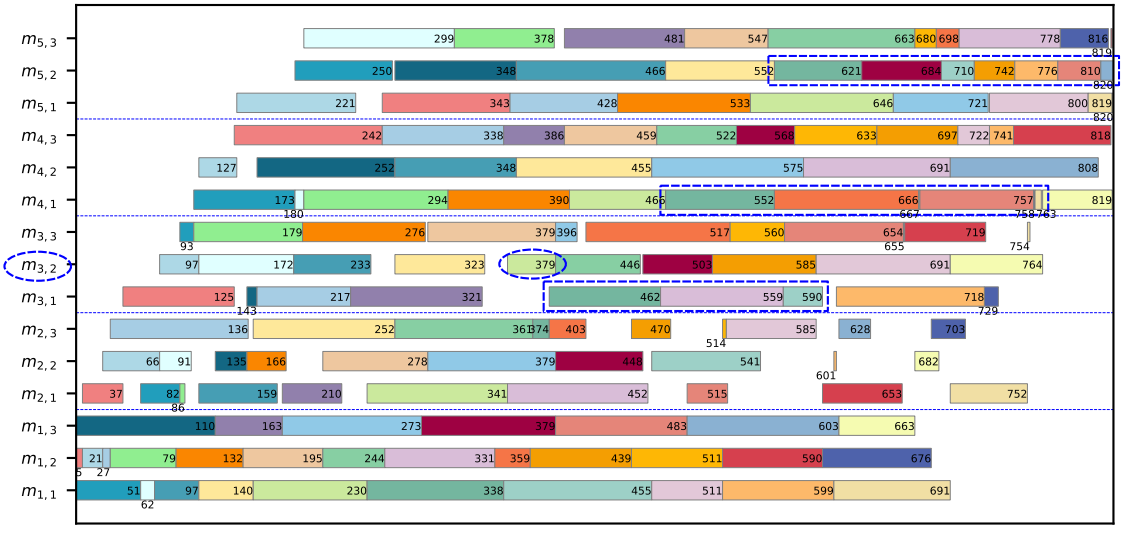


Fig. 12. Convergence evolution curves of the compared algorithms for Jose's instance.



(a) The best solution found by *IGGR*, *IGRS*, *IGALL*, *IGT*, *VBIH*, *CSA*, *IGWS*, and *TSCNSA* (makespan=821)



(b) The best solution found by applying the mandatory operations-based effective insertion local search to Fig. 13. (a) (makespan=820)

Fig. 13. The Gantt chart after applying the mandatory operations-based effective insertion local search.

applying the mandatory operations-based effective insertion local search to Fig. 13(a) (makespan = 820). From Fig. 13(b), it can be observed that the processing machine for the selected mandatory operation changed from $m_{3,1}$ to $m_{3,2}$. As a result, subsequent operations are shifted forward by 1 unit, leading to a decrease in makespan from 821 to 820. Furthermore, it is evident from Fig. 13(b) that no machine allocation rule can generate the Gantt chart after decoding, indicating that the mandatory operations-based local search strategy does not consider any machine allocation rule, thereby expanding the search space and increasing the chances of finding the optimal solution.

In summary, the experimental results reported in Tables 4–7 and Figs. 9–13 demonstrate that the proposed *MOAIG* algorithm is applicable to solving the HFSP with makespan criterion. The superiority is attributed to the mandatory operations-based effective insertion local search. The mandatory operation set is a subset of the critical path operation set. Deleting mandatory operations must make the original

Table 8
ARPD results of the variance analysis under $t = 0.006$ and $t = 0.01$.

$t = 0.006$						$t = 0.01$				
Source	DF	Adj SS	Adj MS	*F-value*	*P-value*	DF	Adj SS	Adj MS	*F-value*	*P-value*
Factor	8	487.5	60.9361	95.57	**0.00**	8	521.6	65.2001	100.54	**0.00**
Error	4311	2748.8	0.6376	/	/	4311	2795.6	0.6485	/	/
Total	4319	3236.3	/	/	/	4319	3317.2	/	/	/

Table 9
ARPD results achieved by Friedman test (confident level $\alpha = 0.05$) under $t = 0.006$ and $t = 0.01$.

Algorithms	$t = 0.006$						$t = 0.01$					
	Ranks	CN	Min	Max	Mean	Std. Dev	Ranks	CN	Min	Max	Mean	Std. Dev
IGWS	4.19	480	0	2.9	0.69	0.6003	4.46	480	0	2.85	0.68	0.5946
IGT	3.86	480	0	4.48	0.62	0.5281	3.81	480	0	4.43	0.583	0.5313
CSA	5.97	480	0	4.64	1.101	1.0961	5.96	480	0	5.73	1.072	1.1188
TCANSA	7.28	480	0	6.24	1.7	1.4933	7.24	480	0	6.08	1.711	1.5274
IGRS	5.26	480	0	4.53	0.748	0.6022	5.12	480	0	4.43	0.698	0.5841
IGGR	4.78	480	0	4.48	0.722	0.5828	4.66	480	0	4.43	0.668	0.568
IGALL	4.96	480	0	4.43	0.732	0.5959	4.96	480	0	4.43	0.693	0.583
VBIH	5.52	480	0	4.58	0.803	0.6468	5.65	480	0	4.48	0.773	0.6398
MOAIG	**3.18**	480	**0**	**2.53**	**0.521**	**0.4384**	**3.15**	480	**0**	**2.53**	**0.509**	**0.4502**
P-value	**0.00**	/	/	/	/	/	**0.00**	/	/	/	/	/

Table 10
Wilcoxon signed-rank test results for all instances under $t = 0.006$.

MOAIG vs.	240 small-size instances			240 large-size instances			96 instances of Liao's benchmarks		
	p-value	Significant ($p \leq 0.05$)	Decision	*p*-value	Significant ($p \leq 0.05$)	Decision	*p*-value	Significant ($p \leq 0.05$)	Decision
IGWS	0.000	Yes	Reject H_0	0.000	Yes	Reject H_0	0.185	No	Fail to reject H_0
IGT	0.005	Yes	Reject H_0	0.005	Yes	Reject H_0	0.032	Yes	Reject H_0
CSA	0.000	Yes	Reject H_0	0.000	Yes	Reject H_0	0.000	Yes	Reject H_0
TCANSA	0.000	Yes	Reject H_0	0.000	Yes	Reject H_0	0.576	No	Fail to reject H_0
IGRS	0.000	Yes	Reject H_0	0.000	Yes	Reject H_0	0.069	No	Fail to reject H_0
IGGR	0.000	Yes	Reject H_0	0.000	Yes	Reject H_0	0.041	Yes	Reject H_0
IGALL	0.000	Yes	Reject H_0	0.000	Yes	Reject H_0	0.060	No	Fail to reject H_0
VBIH	0.000	Yes	Reject H_0	0.000	Yes	Reject H_0	0.028	Yes	Reject H_0

Table 11
Wilcoxon signed-rank test results for all instances under t = 0.01.

MOAIG vs.	240 small-size instances			240 large-size instances			96 instances of Liao's benchmarks		
	p-value	Significant ($p \leq 0.05$)	Decision	*p*-value	Significant ($p \leq 0.05$)	Decision	*p*-value	Significant ($p \leq 0.05$)	Decision
IGWS	0.000	Yes	Reject H_0	0.000	Yes	Reject H_0	0.044	Yes	Reject H_0
IGT	0.005	Yes	Reject H_0	0.005	Yes	Reject H_0	0.008	Yes	Reject H_0
CSA	0.000	Yes	Reject H_0	0.000	Yes	Reject H_0	0.000	Yes	Reject H_0
TCANSA	0.000	Yes	Reject H_0	0.000	Yes	Reject H_0	0.917	No	Fail to reject H_0
IGRS	0.000	Yes	Reject H_0	0.000	Yes	Reject H_0	0.013	Yes	Reject H_0
IGGR	0.000	Yes	Reject H_0	0.000	Yes	Reject H_0	0.007	Yes	Reject H_0
IGALL	0.000	Yes	Reject H_0	0.000	Yes	Reject H_0	0.009	Yes	Reject H_0
VBIH	0.000	Yes	Reject H_0	0.000	Yes	Reject H_0	0.005	Yes	Reject H_0

makespan value smaller. Thus, the mandatory operations-based local search can reduce the number of operations perturbed without losing the opportunity to find the optimal solution. This is because sometimes there are multiple critical paths, no matter how the operations in the critical path but not in the mandatory operations are perturbed, the makespan will not be affected. Thus, this approach reduces the number of operations and insertions involved in the calculation, leading to a significant reduction in the overall computation time.

6.5. Further statistical analysis

To further testify the effective performance of the *MOAIG* on the *ARPD* indicator, we have performed the statistical analysis of significance experiments. Table 8 reports the results of the one-way ANOVA, containing degrees of freedom (DF), adjusted sum of squares (Adj. SS), adjusted mean square (Adj. MS), F-value, and P-value. Table 9 gives the Friedman test of the *ARPD* indicator on 480 instances, containing the rank (Rank), the quantity of instances (CN), the minimum value of *ARPD* (Min), the maximum value of *ARPD* (Max), the mean of the *ARPD* (Mean) and the standard deviation (Std. Dev). The confidence level, α, is equal to 0.05. In Tables 8 and 9, the *P*-value is equal to 0.00. This result demonstrates that our algorithm is significantly different from the eight compared ones on the *ARPD* indicator. The Ranks value 3.18 of MOAIG is the lowest among *IGWS*(4.19), *IGT*(3.86), *CSA*(5.97), *TCANSA*(7.28), *IGRS*(5.26), *IGGR* (4.78), *IGALL* (4.96), and *VBIH* (5.52) under $t = 0.006$. Similarly, the effective performance of the *MOAIG* can be further demonstrated under $t = 0.01$.

To further test the distinctions among the nine algorithms, the results of the Wilcoxon signed-rank test for pairwise comparisons of algorithms are presented in Tables 10 and 11 under $t = 0.006$ and $t = 0.01$. As shown in the Tables 10 and 11, for the 240 small-size and 240 large-size Jose's benchmarks, all pairwise differences are statistically significant at the $\alpha = 0.05$ level. For the 96 instances of Jose's benchmarks, all pairwise differences are statistically significant at the $\alpha = 0.05$ level, with the exception of the *MOAIG* vs *IGWS*, *MOAIG* vs *TCANSA*, *MOAIG* vs *IGRS*, and *MOAIG* vs *IGALL*. In these cases, the differences between the algorithms are not statistically significant at the $\alpha = 0.05$ level, suggesting that algorithms within these pairs exhibit similar performances. The main reason is that these 96 instances are relatively small in scale. Combined with the results in Table 7, it can be seen that some algorithms are able to find optimal or near-optimal solutions for Liao's benchmarks, demonstrating their superiority in handling the problem. However, when $t = 0.01$, except for TCANSA, all pairwise differences are statistically significant at the $\alpha = 0.05$ level for the 96 instances.

6.6. Upper bound update for Jose's instances

Table 12 reports the upper bounds obtained by *IG* (Fernandez-Viagas

Table 12
Upper bounds obtained by all algorithms on Jose's benchmarks.

240 small-size						240 large-size					
Instances	Algs	*MOAIG*	Instances	Algs	*MOAIG*	Instances	Algs	*MOAIG*	Instances	Algs	*MOAIG*
10_5_1	**408**	**408**	25_5_1	**774**	**774**	40_5_1	**976**	**976**	160_5_1	**4141**	**4141**
10_5_2	**384**	387	25_5_2	**696**	**696**	40_5_2	**978**	**978**	160_5_2	**3941**	**3941**
10_5_3	**453**	**453**	25_5_3	**704**	**706**	40_5_3	**870**	**870**	160_5_3	**4365**	**4365**
10_5_4	**452**	**452**	25_5_4	790	**789**	40_5_4	850	**848**	160_5_4	**3938**	**3938**
10_5_5	**389**	**389**	25_5_5	**600**	**600**	40_5_5	**765**	**765**	160_5_5	**4071**	**4071**
10_5_6	**357**	360	25_5_6	**700**	**704**	40_5_6	**887**	888	160_5_6	**4011**	**4011**
10_5_7	**437**	443	25_5_7	**691**	**691**	40_5_7	**779**	**779**	160_5_7	**4391**	**4391**
10_5_8	**433**	436	25_5_8	**692**	**692**	40_5_8	1007	**1006**	160_5_8	**4554**	**4554**
10_5_9	**406**	**406**	25_5_9	**667**	**668**	40_5_9	**932**	933	160_5_9	**4008**	**4008**
10_5_10	**373**	**373**	25_5_10	**646**	**646**	40_5_10	1015	**1013**	160_5_10	**4097**	**4097**
10_10_1	**794**	806	25_10_1	**859**	**861**	40_10_1	**1292**	**1292**	160_10_1	**4270**	**4270**
10_10_2	**785**	**785**	25_10_2	**974**	**974**	40_10_2	1297	**1295**	160_10_2	**3899**	**3899**
10_10_3	**755**	**755**	25_10_3	911	**909**	40_10_3	**1181**	**1181**	160_10_3	**4583**	**4583**
10_10_4	**914**	922	25_10_4	**994**	**994**	40_10_4	**1271**	1272	160_10_4	**4155**	**4155**
10_10_5	**957**	969	25_10_5	**939**	**940**	40_10_5	**1297**	**1297**	160_10_5	4177	**4176**
10_10_6	**1001**	**1001**	25_10_6	**991**	**994**	40_10_6	1286	**1284**	160_10_6	**4323**	**4323**
10_10_7	**939**	944	25_10_7	**875**	**877**	40_10_7	1136	**1131**	160_10_7	**4429**	**4429**
10_10_8	**543**	545	25_10_8	**835**	**835**	40_10_8	**1112**	1113	160_10_8	**4299**	**4299**
10_10_9	**510**	516	25_10_9	1058	**1054**	40_10_9	1279	**1274**	160_10_9	**4616**	**4616**
10_10_10	**684**	**684**	25_10_10	918	**917**	40_10_10	1350	**1349**	160_10_10	**4279**	**4279**
10_15_1	**959**	961	25_15_1	**1204**	**1204**	40_15_1	1487	**1485**	160_15_1	4172	**4137**
10_15_2	**1290**	**1290**	25_15_2	**1136**	**1140**	40_15_2	1572	**1571**	160_15_2	4114	**4095**
10_15_3	**1091**	**1091**	25_15_3	1221	**1219**	40_15_3	**1610**	1611	160_15_3	4085	**4059**
10_15_4	**866**	872	25_15_4	**1352**	**1352**	40_15_4	1377	**1370**	160_15_4	4207	**4190**
10_15_5	**879**	881	25_15_5	**1341**	**1345**	40_15_5	1320	**1317**	160_15_5	3480	**3467**
10_15_6	**836**	841	25_15_6	**1146**	**1147**	40_15_6	1223	**1222**	160_15_6	3420	**3399**
10_15_7	**902**	908	25_15_7	**1056**	**1056**	40_15_7	1293	**1286**	160_15_7	4201	**4179**
10_15_8	**765**	768	25_15_8	1108	**1107**	40_15_8	**1384**	1388	160_15_8	4125	**4078**
10_15_9	**751**	760	25_15_9	1144	**1139**	40_15_9	1712	**1707**	160_15_9	4144	**4140**
10_15_10	**847**	863	25_15_10	1163	**1158**	40_15_10	1622	**1618**	160_15_10	4126	**4090**
10_20_1	**1346**	1353	25_20_1	**1442**	**1443**	40_20_1	1936	**1925**	160_20_1	4538	**4499**
10_20_2	**1155**	**1155**	25_20_2	**1345**	**1350**	40_20_2	1912	**1903**	160_20_2	3727	**3708**
10_20_3	**1503**	**1503**	25_20_3	**1426**	**1428**	40_20_3	1592	**1571**	160_20_3	3901	**3862**
10_20_4	**1459**	1468	25_20_4	**1399**	**1407**	40_20_4	1590	**1588**	160_20_4	3733	**3680**
10_20_5	**1482**	1496	25_20_5	**1401**	**1404**	40_20_5	1621	**1613**	160_20_5	3787	**3744**
10_20_6	**1302**	**1302**	25_20_6	**1365**	**1368**	40_20_6	1971	**1959**	160_20_6	3744	**3718**
10_20_7	**1410**	1415	25_20_7	**1674**	**1679**	40_20_7	2750	**2731**	160_20_7	3778	**3767**
10_20_8	**1517**	1522	25_20_8	1307	**1306**	40_20_8	1689	**1675**	160_20_8	4583	**4544**
10_20_9	**883**	903	25_20_9	**1337**	**1337**	40_20_9	1732	**1719**	160_20_9	4542	**4503**
10_20_10	**1088**	1093	25_20_10	**1226**	**1227**	40_20_10	1562	**1544**	160_20_10	4226	**4189**
15_5_1	**480**	486	30_5_1	649	**648**	80_5_1	**1974**	**1974**	200_5_1	**5020**	**5020**
15_5_2	**421**	422	30_5_2	**789**	**789**	80_5_2	**2098**	**2098**	200_5_2	**5010**	**5010**
15_5_3	**503**	504	30_5_3	**790**	**791**	80_5_3	**2071**	**2071**	200_5_3	**4868**	**4868**
15_5_4	**436**	**436**	30_5_4	**680**	**680**	80_5_4	**1898**	**1898**	200_5_4	**5146**	**5146**
15_5_5	**416**	418	30_5_5	**695**	**696**	80_5_5	**2164**	**2164**	200_5_5	**4996**	**4996**
15_5_6	**411**	414	30_5_6	661	**659**	80_5_6	**2103**	**2103**	200_5_6	**5049**	**5049**
15_5_7	**484**	**484**	30_5_7	**572**	**573**	80_5_7	**2002**	**2002**	200_5_7	**4998**	**4998**
15_5_8	**520**	524	30_5_8	**804**	**804**	80_5_8	**2058**	**2058**	200_5_8	**5159**	**5159**
15_5_9	**547**	556	30_5_9	821	**820**	80_5_9	**2130**	**2130**	200_5_9	**5334**	**5334**
15_5_10	**441**	443	30_5_10	**759**	**759**	80_5_10	**1873**	**1873**	200_5_10	**5214**	**5214**
15_10_1	**748**	756	30_10_1	1057	**1056**	80_10_1	2166	**2155**	200_10_1	**5534**	**5534**
15_10_2	**696**	702	30_10_2	**929**	**929**	80_10_2	2048	**2046**	200_10_2	**4943**	**4943**
15_10_3	**853**	**853**	30_10_3	1126	**1125**	80_10_3	1799	**1795**	200_10_3	**5095**	**5095**
15_10_4	**879**	883	30_10_4	1092	**1091**	80_10_4	1656	**1652**	200_10_4	**5229**	**5229**
15_10_5	**1076**	**1076**	30_10_5	942	**940**	80_10_5	1664	**1659**	200_10_5	**5470**	**5470**
15_10_6	**1042**	**1042**	30_10_6	968	**967**	80_10_6	1827	**1822**	200_10_6	**5169**	**5169**
15_10_7	**1020**	**1020**	30_10_7	976	**974**	80_10_7	**1773**	**1773**	200_10_7	**5171**	**5171**
15_10_8	**1011**	**1011**	30_10_8	**1002**	**1006**	80_10_8	1738	**1729**	200_10_8	**5255**	**5255**
15_10_9	**643**	654	30_10_9	**899**	**904**	80_10_9	1666	**1650**	200_10_9	**4891**	**4891**
15_10_10	**733**	735	30_10_10	1096	**1093**	80_10_10	2153	**2147**	200_10_10	**4878**	**4878**
15_15_1	**1023**	1027	30_15_1	**1198**	**1199**	80_15_1	**2333**	**2333**	200_15_1	5176	**5175**
15_15_2	**1054**	1059	30_15_2	**1270**	**1271**	80_15_2	**2328**	**2328**	200_15_2	**5253**	**5253**
15_15_3	**1144**	1145	30_15_3	**1207**	**1208**	80_15_3	2466	**2462**	200_15_3	**5239**	**5239**
15_15_4	**1160**	1169	30_15_4	1526	**1522**	80_15_4	2565	**2560**	200_15_4	5239	**5238**
15_15_5	**1182**	1188	30_15_5	**1127**	**1133**	80_15_5	2100	**2086**	200_15_5	**5180**	**5178**
15_15_6	**1161**	1168	30_15_6	1433	**1432**	80_15_6	2026	**2014**	200_15_6	**5588**	**5586**
15_15_7	**1549**	1570	30_15_7	**1448**	**1448**	80_15_7	2496	**2485**	200_15_7	**5012**	**5012**
15_15_8	**938**	943	30_15_8	**1413**	**1432**	80_15_8	2495	**2486**	200_15_8	5437	**5432**
15_15_9	**897**	901	30_15_9	2019	**2016**	80_15_9	2250	**2240**	200_15_9	5226	**5225**
15_15_10	**870**	871	30_15_10	**1250**	**1254**	80_15_10	2297	**2291**	200_15_10	4962	**4939**
15_20_1	**1252**	1263	30_20_1	1486	**1483**	80_20_1	2400	**2381**	200_20_1	**5601**	**5601**
15_20_2	**1564**	**1564**	30_20_2	1594	**1591**	80_20_2	2454	**2447**	200_20_2	5432	**5396**
15_20_3	**1206**	1212	30_20_3	**1563**	**1570**	80_20_3	2781	**2746**	200_20_3	5338	**5332**

(*continued on next page*)

Table 12 (*continued*)

240 small-size						240 large-size					
Instances	Algs	*MOAIG*	Instances	Algs	*MOAIG*	Instances	Algs	*MOAIG*	Instances	Algs	*MOAIG*
15_20_4	**1557**	**1557**	30_20_4	**1536**	**1541**	80_20_4	2866	**2835**	200_20_4	4419	**4399**
15_20_5	**1535**	1556	30_20_5	1623	**1622**	80_20_5	2378	**2349**	200_20_5	4459	**4435**
15_20_6	**1684**	1687	30_20_6	1499	**1492**	80_20_6	2380	**2362**	200_20_6	4492	**4434**
15_20_7	**1699**	**1699**	30_20_7	**1516**	**1519**	80_20_7	2440	**2412**	200_20_7	4317	**4305**
15_20_8	**1699**	**1699**	30_20_8	1820	**1819**	80_20_8	2405	**2386**	200_20_8	4457	**4389**
15_20_9	**984**	997	30_20_9	2362	**2351**	80_20_9	2397	**2377**	200_20_9	5392	**5374**
15_20_10	**1085**	1093	30_20_10	2391	**2389**	80_20_10	2548	**2510**	200_20_10	5404	**5381**
20_5_1	**660**	**660**	35_5_1	**1080**	**1080**	120_5_1	**2818**	**2818**	240_5_1	**6346**	**6346**
20_5_2	**583**	585	35_5_2	**1074**	**1074**	120_5_2	**3051**	**3051**	240_5_2	**5964**	**5964**
20_5_3	**555**	557	35_5_3	**888**	**888**	120_5_3	**2932**	**2932**	240_5_3	**6078**	**6078**
20_5_4	**549**	552	35_5_4	**862**	**862**	120_5_4	**3195**	**3195**	240_5_4	**6127**	**6127**
20_5_5	**526**	**526**	35_5_5	**886**	**886**	120_5_5	**2889**	**2889**	240_5_5	**6319**	**6319**
20_5_6	**511**	513	35_5_6	**893**	**894**	120_5_6	**2874**	**2874**	240_5_6	**5845**	**5845**
20_5_7	**672**	677	35_5_7	667	**666**	120_5_7	**2754**	**2754**	240_5_7	**5930**	**5930**
20_5_8	**514**	517	35_5_8	**714**	**714**	120_5_8	**3136**	**3136**	240_5_8	**6217**	**6217**
20_5_9	**679**	681	35_5_9	**840**	**844**	120_5_9	**3066**	**3066**	240_5_9	**5735**	**5735**
20_5_10	**523**	524	35_5_10	**946**	**949**	120_5_10	**2793**	**2793**	240_5_10	**6406**	**6406**
20_10_1	**793**	796	35_10_1	**1085**	**1085**	120_10_1	**3124**	**3124**	240_10_1	**6233**	6234
20_10_2	**841**	843	35_10_2	**1247**	**1247**	120_10_2	**2950**	**2950**	240_10_2	**6096**	**6096**
20_10_3	**850**	854	35_10_3	**1061**	**1061**	120_10_3	**3134**	3135	240_10_3	**6482**	6490
20_10_4	**1011**	**1011**	35_10_4	**1092**	**1092**	120_10_4	**3196**	**3196**	240_10_4	**6140**	**6140**
20_10_5	**971**	973	35_10_5	1267	**1265**	120_10_5	**3236**	**3236**	240_10_5	**6184**	**6184**
20_10_6	**792**	794	35_10_6	**1157**	**1159**	120_10_6	**3131**	**3131**	240_10_6	**5894**	**5894**
20_10_7	**766**	769	35_10_7	**1011**	**1013**	120_10_7	**3131**	**3131**	240_10_7	**6038**	**6038**
20_10_8	**943**	948	35_10_8	**986**	**990**	120_10_8	**3092**	**3092**	240_10_8	**5935**	**5935**
20_10_9	**946**	951	35_10_9	1141	**1140**	120_10_9	**3431**	**3431**	240_10_9	**6382**	6383
20_10_10	**862**	866	35_10_10	**2105**	**2113**	120_10_10	3000	**2975**	240_10_10	**12,744**	**12,744**
20_15_1	**1064**	1065	35_15_1	**1371**	**1371**	120_15_1	3290	**3265**	240_15_1	**6506**	**6506**
20_15_2	**1323**	1332	35_15_2	1351	**1349**	120_15_2	3314	**3276**	240_15_2	**6425**	**6425**
20_15_3	**1283**	1295	35_15_3	1594	**1593**	120_15_3	2696	**2680**	240_15_3	**6029**	**6029**
20_15_4	**1026**	**1026**	35_15_4	**1524**	**1528**	120_15_4	2776	**2754**	240_15_4	**6632**	**6632**
20_15_5	**1010**	1014	35_15_5	1244	**1238**	120_15_5	3299	**3278**	240_15_5	**6414**	**6414**
20_15_6	**1277**	**1277**	35_15_6	**1330**	**1334**	120_15_6	3272	**3246**	240_15_6	**6091**	**6091**
20_15_7	**1268**	1274	35_15_7	1522	**1517**	120_15_7	3320	**3273**	240_15_7	**6145**	6148
20_15_8	**1255**	1259	35_15_8	**1483**	**1486**	120_15_8	3265	**3228**	240_15_8	**6068**	**6068**
20_15_9	**1745**	1747	35_15_9	**2236**	**2236**	120_15_9	3264	**3234**	240_15_9	**6305**	**6305**
20_15_10	**959**	964	35_15_10	**1304**	**1307**	120_15_10	3299	**3265**	240_15_10	**6382**	**6382**
20_20_1	**1322**	1324	35_20_1	1565	**1562**	120_20_1	3638	**3637**	240_20_1	6185	**6148**
20_20_2	**1321**	1325	35_20_2	1693	**1692**	120_20_2	3599	**3555**	240_20_2	5197	**5178**
20_20_3	**1315**	1319	35_20_3	1567	**1559**	120_20_3	3620	**3585**	240_20_3	5087	**5059**
20_20_4	**1572**	1577	35_20_4	**1895**	**1898**	120_20_4	3664	**3627**	240_20_4	6198	**6159**
20_20_5	**1307**	1315	35_20_5	1850	**1848**	120_20_5	3652	**3612**	240_20_5	6183	**6161**
20_20_6	**1280**	**1280**	35_20_6	**1404**	**1406**	120_20_6	3073	**3040**	240_20_6	6225	**6220**
20_20_7	**1623**	1629	35_20_7	1586	**1583**	120_20_7	3036	**3013**	240_20_7	6334	**6303**
20_20_8	1833	**1825**	35_20_8	**1926**	**1929**	120_20_8	3707	**3658**	240_20_8	6166	**6164**
20_20_9	1294	**1291**	35_20_9	1975	**1963**	120_20_9	3693	**3676**	240_20_9	6202	**6159**
20_20_10	1202	**1199**	35_20_10	1834	**1828**	120_20_10	3751	**3730**	240_20_10	6138	**6111**
Num. best	**117**	36	*Num. best*	**77**	75	*Num. best*	47	**113**	*Num. best*	74	**116**
Num.best\same	**131** vs 48					*Num.best\same*	13 vs **121**				

Additional remark of the total running time: IG $t \times \mu^3 \times \sigma/1000$ seconds with $t = 50$ and 0.5 for the small- and large-scale instances; CAS:$t \times 0.3 \times \mu \times \sigma/1000$ seconds with $t = 50$ and 0.5 for the small- and large-scale instances; TSCNSA: $3 \times \mu \times \sigma$ seconds; MOAIG: $3 \times \mu \times \sigma$ seconds.

& Framinan, 2020), *CSA* (Lin et al., 2021), *TSCNSA* (Kuang et al., 2024) and MOAIG, where the column 'Instances' indicates the instance number, the column 'Algs' represents the best upper bounds among *IG*, *CSA*, and *TSCNSA*, the column '*MOAIG*' represents the upper bounds obtained by our algorithm. It should be noted that for *IGGR*, *IGRS*, *IGALL*, *IGT*, *VBIH*, and *IGWS*, no upper bounds were provided in the original paper. To ensure fairness, we select algorithms for which upper bounds are provided. The data in bold in all columns means the best upper bound. The row 'Num.best' is the number of best solutions (including the same best solutions) for 240 small-size and 240 large-size instances, respectively, obtained by 'Algs' and *MOAIG* algorithms. The row 'Num. best\same' is the number of best solutions excluding the same solutions from 'Num.best'.

Among the 240 small-scale instances, the 'Num.best\same' values obtained by 'Algs' and *MOAIG* algorithms are 131 and 48 best upper bounds, respectively. This experimental result indicates that the 'Algs' algorithm performs well in solving small-scale problems. However, it is noteworthy that the result from 'Algs' represents the best value among the three algorithms. In other words, although each algorithm individually may not exhibit superiority in solving small-scale problems, the overall performance is improved by combining their strengths and selecting the best result as the optimal solution for small-scale problems. Nevertheless, when solving large-scale problems, 'Algs' and *MOAIG* algorithms obtain 13 and 121 best upper bounds. This result effectively demonstrates the validity of the proposed algorithm. This is attributed to two main factors: Firstly, conducting an operations-based local search on the graph space can expand solution space of HFSP. Secondly, employing an efficient position insertion based on mandatory operations significantly reduces the algorithm's time complexity. Consequently, the algorithm can adequately explore the local area within a reasonable time to find better solutions.

7. Conclusions and future work

The accelerated local search methods presented to solve flow shop scheduling problems offer significant advantages in terms of computational efficiency and optimization effectiveness. By exploiting problem-specific characteristics and optimizing the evaluation process, the methods significantly reduce computational time and resources required, which is crucial for real-time or large-scale scheduling applications. Based on the above advantages, we have analyzed the search solution space of the hybrid flow shop scheduling problem with makespan criterion, and proposed the theoretical analysis and methods of a mandatory operations-based effective insertion accelerated local search in the graph space. Through many experiments on two widely recognized benchmarks, totaling 576 test instances, a substantial reduction in computational efforts has been demonstrated. In contrast to other methods, the proposed *MOAIG* algorithm shows superiority by reducing the number of operations perturbed and insertion involved in the calculation, leading to a significant reduction in the overall computation time.

In the future, given the increasing complexity of modern manufacturing environments, where factors such as distributed flexible flow or job shop scheduling, customer satisfaction, and flexible delivery schedules are paramount, expanding the application of our method is crucial. Especially, according to the concept of mandatory operations and effective insertion in the graph space, we can extend the proposed method to handle a broader range of constraints and objectives, including multi-objective optimization and additional scheduling constraints, and real-time changes in production scenarios, enabling dynamic adjustments to schedules as conditions evolve. Overall, the rapid evaluation method has the potential to significantly improve scheduling efficiency, reduce production costs, and improve overall operational performance. Continued research and development in this area will undoubtedly lead to innovative solutions that benefit a wide range of industries.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was jointly supported by the Natural Science Foundation of Shandong province under grant numbers ZR2023MF022, National Natural Science Foundation of China under grant numbers 61973203, 61803192, 62106073, and 61966012. We are grateful for Guangyue Young Scholar Innovation Team of Liaocheng University under grant number LCUGYTD2022-03.

References

Aqil, S., & Allali, K. (2020). Local search metaheuristic for solving hybrid flow shop problem in slabs and beams manufacturing. *Expert Systems with Applications, 162*, Article 113716. https://doi.org/10.1016/j.eswa.2020.113716

Carlier, J., & Neron, E. (2000). An Exact Method for Solving the Multi-Processor Flow-Shop. *RAIRO - Operations Research, 34*(1), 1–25. https://doi.org/10.1051/ro:2000103

Chou, F.-D. (2013). Particle swarm optimization with cocktail decoding method for hybrid flow shop scheduling problems with multiprocessor tasks. *International Journal of Production Economics, 141*(1), 137–145. https://doi.org/10.1016/j.ijpe.2012.05.015

Cui, Z., & Gu, X. (2015). An improved discrete artificial bee colony algorithm to minimize the makespan on hybrid flow shop problems. *Neurocomputing, 148*, 248–259. https://doi.org/10.1016/j.neucom.2013.07.056

Dios, M., Fernandez-Viagas, V., & Framinan, J. M. (2018). Efficient heuristics for the hybrid flow shop scheduling problem with missing operations. *Computers & Industrial Engineering, 115*, 88–99. https://doi.org/10.1016/j.cie.2017.10.034

Engin, O., & Döyen, A. (2004). A new approach to solve hybrid flow shop scheduling problems by artificial immune system. *Future Generation Computer Systems, 20*(6), 1083–1095. https://doi.org/10.1016/j.future.2004.03.014

Fan, J., Li, Y., Xie, J., Zhang, C., Shen, W., & Gao, L. (2023). A hybrid evolutionary algorithm using two solution representations for hybrid flow-shop scheduling problem. *IEEE Transactions on Cybernetics, 53*(3), 1752–1764. https://doi.org/10.1109/TCYB.2021.3120875

Fernandez-Viagas, V. (2022). A speed-up procedure for the hybrid flow shop scheduling problem. *Expert Systems with Applications, 187*, Article 115903. https://doi.org/10.1016/j.eswa.2021.115903

Fernandez-Viagas, V., & Framinan, J. M. (2020). Design of a testbed for hybrid flow shop scheduling with identical machines. *Computers & Industrial Engineering, 141*, Article 106288. https://doi.org/10.1016/j.cie.2020.106288

Fernandez-Viagas, V., Molina-Pariente, J. M., & Framinan, J. M. (2018). New efficient constructive heuristics for the hybrid flowshop to minimise makespan: A computational evaluation of heuristics. *Expert Systems with Applications, 114*, 345–356. https://doi.org/10.1016/j.eswa.2018.07.055

Fernandez-Viagas, V., Perez-Gonzalez, P., & Framinan, J. M. (2019). Efficiency of the solution representations for the hybrid flow shop scheduling problem with makespan objective. *Computers & Operations Research, 109*, 77–88. https://doi.org/10.1016/j.cor.2019.05.002

Gümüşçü, A., Kaya, S., Tenekeci, M. E., Karaçizmeli, İ. H., & Aydilek, İ. B. (2022). The impact of local search strategies on chaotic hybrid firefly particle swarm optimization algorithm in flow-shop scheduling. *Journal of King Saud University - Computer and Information Sciences, 34*(8, Part B), 6432–6440. https://doi.org/10.1016/j.jksuci.2021.07.017.

Gupta, J. N. D. (1988). Two-stage, hybrid flowshop scheduling problem. *Journal of the Operational Research Society, 39*(4), 359–364. https://doi.org/10.1057/jors.1988.63

Han, W., Deng, Q., Gong, G., Zhang, L., & Luo, Q. (2021). Multi-objective evolutionary algorithms with heuristic decoding for hybrid flow shop scheduling problem with worker constraint. *Expert Systems with Applications, 168*, Article 114282. https://doi.org/10.1016/j.eswa.2020.114282

Han, Y., Wang, Y., Pan, Q., Wang, L., & Tasgetiren, M. F. (2024). Accelerated evaluation of blocking flowshop scheduling with total flow time criteria using a generalized critical machine-based approach. *European Journal of Operational Research*. https://doi.org/10.1016/j.ejor.2024.05.015

Kuang, Y., Wu, X., Chen, Z., & Li, W. (2024). A two-stage cross-neighborhood search algorithm bridging different solution representation spaces for solving the hybrid flow shop scheduling problem. *Swarm and Evolutionary Computation, 84*, Article 101455. https://doi.org/10.1016/j.swevo.2023.101455

Li, J., Pan, Q., & Wang, F. (2014). A hybrid variable neighborhood search for solving the hybrid flow shop scheduling problem. *Applied Soft Computing, 24*, 63–77. https://doi.org/10.1016/j.asoc.2014.07.005

Liao, C.-J., Tjandradjaja, E., & Chung, T.-P. (2012). An approach using particle swarm optimization and bottleneck heuristic to solve hybrid flow shop scheduling problem. *Applied Soft Computing, 12*(6), 1755–1764. https://doi.org/10.1016/j.asoc.2012.01.011

Lin, S.-W., Cheng, C.-Y., Pourhejazy, P., Ying, K.-C., & Lee, C.-H. (2021). New benchmark algorithm for hybrid flowshop scheduling with identical machines. *Expert Systems with Applications, 183*, Article 115422. https://doi.org/10.1016/j.eswa.2021.115422

Marichelvam, M. K., Geetha, M., & Tosun, Ö. (2020). An improved particle swarm optimization algorithm to solve hybrid flowshop scheduling problems with the effect of human factors – A case study. *Computers & Operations Research, 114*, Article 104812. https://doi.org/10.1016/j.cor.2019.104812

Marichelvam, M. K., Prabaharan, T., & Yang, X. S. (2014). Improved cuckoo search algorithm for hybrid flow shop scheduling problems to minimize makespan. *Applied Soft Computing, 19*, 93–101. https://doi.org/10.1016/j.asoc.2014.02.005

Mastrolilli, M., & Gambardella, L. M. (2000). Effective neighbourhood functions for the flexible job shop problem. *Journal of Scheduling, 3*(1), 3–20. https://doi.org/10.1002/(SICI)1099-1425(200001/02)3:1<3::AID-JOS32>3.0.CO;2-Y

Öztop, H., Fatih Tasgetiren, M., Eliiyi, D. T., & Pan, Q.-K. (2019). Metaheuristic algorithms for the hybrid flowshop scheduling problem. *Computers & Operations Research, 111*, 177–196. https://doi.org/10.1016/j.cor.2019.06.009

Pan, Q.-K., Tasgetiren, M. F., & Liang, Y.-C. (2008). A discrete differential evolution algorithm for the permutation flowshop scheduling problem. *Computers & Industrial Engineering, 55*(4), 795–816. https://doi.org/10.1016/j.cie.2008.03.003

Pan, Q.-K., Wang, L., Li, J.-Q., & Duan, J.-H. (2014). A novel discrete artificial bee colony algorithm for the hybrid flowshop scheduling problem with makespan minimisation. *Omega, 45*, 42–56. https://doi.org/10.1016/j.omega.2013.12.004

Paternina-Arboleda, C. D., Montoya-Torres, J. R., Acero-Dominguez, M. J., & Herrera-Hernandez, M. C. (2008). Scheduling jobs on a k-stage flexible flow-shop. *Annals of Operations Research, 164*(1), 29–40. https://doi.org/10.1007/s10479-007-0257-2

Qin, H., Han, Y., Chen, Q., Wang, L., Wang, Y., Li, J., & Liu, Y. (2023). Energy-efficient iterative greedy algorithm for the distributed hybrid flow shop scheduling with blocking constraints. *IEEE Transactions on Emerging Topics in Computational Intelligence, 1–16*. https://doi.org/10.1109/TETCI.2023.3271331

Qin, H.-X., Han, Y.-Y., Zhang, B., Meng, L.-L., Liu, Y.-P., Pan, Q.-K., & Gong, D.-W. (2022). An improved iterated greedy algorithm for the energy-efficient blocking hybrid flow shop scheduling problem. *Swarm and Evolutionary Computation, 69*, Article 100992. https://doi.org/10.1016/j.swevo.2021.100992

Ribas, I., Leisten, R., & Framiñan, J. M. (2010). Review and classification of hybrid flow shop scheduling problems from a production system and a solutions procedure perspective. *Computers & Operations Research, 37*(8), 1439–1454. https://doi.org/10.1016/j.cor.2009.11.001

Schulz, S., Neufeld, J. S., & Buscher, U. (2019). A multi-objective iterated local search algorithm for comprehensive energy-aware hybrid flow shop scheduling. *Journal of Cleaner Production, 224*, 421–434. https://doi.org/10.1016/j.jclepro.2019.03.155

Shao, W., Shao, Z., & Pi, D. (2021). Multi-objective evolutionary algorithm based on multiple neighborhoods local search for multi-objective distributed hybrid flow shop scheduling problem. *Expert Systems with Applications, 183*, Article 115453. https://doi.org/10.1016/j.eswa.2021.115453

Tarjan, R. (1971). Depth-first search and linear graph algorithms. In *12th Annual Symposium on Switching and Automata Theory (Swat 1971)*, pp. 114–121. https://doi.org/10.1109/SWAT.1971.10.

Tasgetiren, M. F., Pan, Q.-K., Kizilay, D., & Gao, K. (2016). A variable block insertion heuristic for the blocking flowshop scheduling problem with total flowtime criterion. *Algorithms, 9*(4), Article 4. https://doi.org/10.3390/a9040071

Vairaktarakis, G., & Elhafsi, M. (2000). The use of flowlines to simplify routing complexity in two-stage flowshops. *IIE Transactions, 32*(8), 687–699. https://doi.org/10.1023/A:1007652626510

Wang, S., Wang, L., Liu, M., & Xu, Y. (2013). An enhanced estimation of distribution algorithm for solving hybrid flow-shop scheduling problem with identical parallel machines. *The International Journal of Advanced Manufacturing Technology, 68*(9), 2043–2056. https://doi.org/10.1007/s00170-013-4819-y

Wang, S.-Y., Wang, L., Xu, Y., & Zhou, G. (2012). An estimation of distribution algorithm for solving hybrid flow-shop scheduling problem: An estimation of distribution algorithm for solving hybrid flow-shop scheduling problem. *Acta Automatica Sinica, 38*(3), 437–443. https://doi.org/10.3724/SP.J.1004.2012.00437

Wang, Y., Han, Y., Li, H., Pan, Q., & Gong, D. (2024). Theoretical analysis and methods of rapid evaluation: Solving the blocking flowshop group scheduling problem with the total flow time and total tardiness time criteria. *Swarm and Evolutionary Computation, 88*, Article 101604. https://doi.org/10.1016/j.swevo.2024.101604

Wang, Y., Han, Y., Pan, Q., Li, H., & Wang, Y. (2023). Redefining hybrid flow shop group scheduling: Unveiling a novel hybrid modeling paradigm and assessing 48 MILP and CP models. *Swarm and Evolutionary Computation, 83*, Article 101416. https://doi.org/10.1016/j.swevo.2023.101416

Wang, Y., Han, Y., Wang, Y., Pan, Q., & Wang, L. (2023). Sustainable scheduling of distributed flow shop group: A collaborative multi-objective evolutionary algorithm driven by indicators. *IEEE Transactions on Evolutionary Computation, 1–1*. https://doi.org/10.1109/TEVC.2023.3339558

Wang, Y., Han, Y., Wang, Y., Tasgetiren, M. F., Li, J., & Gao, K. (2023). Intelligent optimization under the makespan constraint: Rapid evaluation mechanisms based on the critical machine for the distributed flowshop group scheduling problem. *European Journal of Operational Research, 311*(3), 816–832. https://doi.org/10.1016/j.ejor.2023.05.010

Wang, Y., Wang, Y., Han, Y., Li, J., Gao, K., & Nojima, Y. (2023). Intelligent optimization under multiple factories: Hybrid flow shop scheduling problem with blocking constraints using an advanced iterated greedy algorithm. *Complex System Modeling and Simulation, 3*(4), 282–306. https://doi.org/10.23919/CSMS.2023.0016

Yu, C., Andreotti, P., & Semeraro, Q. (2020). Multi-objective scheduling in hybrid flow shop: Evolutionary algorithms using multi-decoding framework. *Computers & Industrial Engineering, 147*, Article 106570. https://doi.org/10.1016/j.cie.2020.106570

Zhao, F., Zhang, H., & Wang, L. (2023). A Pareto-based discrete Jaya algorithm for multiobjective carbon-efficient distributed blocking flow shop scheduling problem. *IEEE Transactions on Industrial Informatics, 19*(8), 8588–8599. https://doi.org/10.1109/TII.2022.3220860

Zhao, F., Zhu, B., & Wang, L. (2023). An estimation of distribution algorithm-based hyper-heuristic for the distributed assembly mixed no-idle permutation flowshop scheduling problem. *IEEE Transactions on Systems, Man, and Cybernetics: Systems, 53*(9), 5626–5637. https://doi.org/10.1109/TSMC.2023.3272311

Zhao, F., Zhuang, C., Wang, L., & Dong, C. (2024). An iterative greedy algorithm with Q-Learning Mechanism for the multiobjective distributed no-idle permutation flowshop scheduling. *IEEE Transactions on Systems, Man, and Cybernetics: Systems, 1–13*. https://doi.org/10.1109/TSMC.2024.3358383

Zhao, Y., Luo, X., & Zhang, Y. (2024). The application of heterogeneous graph neural network and deep reinforcement learning in hybrid flow shop scheduling problem. *Computers & Industrial Engineering, 187*, Article 109802. https://doi.org/10.1016/j.cie.2023.109802

Zohali, H., Naderi, B., Mohammadi, M., & Roshanaei, V. (2019). Reformulation, linearization, and a hybrid iterated local search algorithm for economic lot-sizing and sequencing in hybrid flow shop problems. *Computers & Operations Research, 104*, 127–138. https://doi.org/10.1016/j.cor.2018.12.008